ADVANCES IN MARGINAL INFERENCE FOR CHAIN-GRAPH LOGICAL CREDAL NETWORKS

Anonymous authors
Paper under double-blind review

ABSTRACT

Logical Credal Networks (LCNs) are an expressive probabilistic logic framework that combines propositional formulas with imprecise probability bounds to represent sets of probability distributions. LCNs allow cyclic dependencies and impose few syntactic restrictions, but inference is hard: the only exact method optimizes a nonconvex program over all 2^n joint states, and the only approximate method is the ARIEL message-passing algorithm. We introduce Credal Junction Tree (CJT), an exact inference algorithm for LCNs whose structure is a chain graph. CJT replaces the joint distribution by local marginals on the clusters of a junction tree of the chain graph, imposes each probabilistic sentence and independence assertion on a cluster that contains it, and solves a nonconvex program with $O(n\,2^{w+1})$ variables for treewidth w. We prove that, under conditions that can be checked mechanically, its global optimum gives the exact lower and upper probabilities. CJT uses only the structure of the chain graph and requires no compilation into a credal network. As a complementary approach, we compile chain-graph LCNs into credal networks, characterize the family-realizable LCNs on which this compilation preserves the exact bounds, and adapt three credal-network algorithms — variable elimination, interval belief propagation and coordinate descent — with their exactness and bound guarantees. We evaluate all methods on synthetic and realistic LCN instances.

1 INTRODUCTION

Combining logical and probabilistic reasoning is a long-standing goal of Artificial Intelligence. Frameworks such as Markov Logic Networks (Richardson & Domingos, 2006), ProbLog (De Raedt et al., 2007), and Probabilistic Soft Logic (Bach et al., 2017) encode domain knowledge as logical rules augmented with uncertainty, but most of them require *precise* probability values or impose strong structural assumptions such as acyclicity. In many real applications, however, domain expertise yields only *ranges* of plausible probabilities and the rules of interest are neither directed nor acyclic.

Logical Credal Networks (LCNs) (Marinescu et al., 2022) were introduced to address this gap: they allow *imprecise* specifications of the form $\alpha \leq P(\phi \,|\, \varphi) \leq \beta$ over arbitrary propositional formulas, with no acyclicity restriction. An LCN defines a set of probability distributions consistent with its constraints and a local Markov condition, and inference amounts to computing tight lower and upper bounds on the probabilities of query formulas. This flexibility makes LCNs well suited to causal reasoning with partial knowledge, robust decision making, and trustworthy neuro-symbolic systems, but it also makes inference considerably harder than in Bayesian or classical credal networks (Pearl, 1988; Cozman, 2000). The only exact method optimizes a nonconvex program over the full joint distribution, with one variable for each of the 2^n truth assignments (Marinescu et al., 2022), which confines it to a handful of propositions; the only approximate method is the ARIEL message-passing scheme (Marinescu et al., 2023).

Contribution. We show that exact LCN inference need not pay the 2^n cost. Our main contribution is *Credal Junction Tree* (CJT), an exact algorithm for LCNs whose structure is a chain graph. Building on the chain-graph factorization of LCNs established by Cozman (2023), CJT moralizes and triangulates the chain graph, builds a junction tree, and replaces the global joint distribution by one local marginal per cluster, tied together by separator-consistency constraints. The LCN's

probabilistic sentences are imposed on the clusters that contain them, and its independence assertions are either entailed by the tree's topology, reduced to equivalent smaller assertions that fit in a cluster, or implied by one constraint per node of a topologically ordered tree. The resulting nonlinear program has $O(n\,2^{w+1})$ variables for treewidth w instead of 2^n, and we prove that, under conditions we characterize and check mechanically, its global optimum is the exact LCN bound. CJT needs only the chain graph's *structure*: unlike the approaches below, it never compiles the LCN into a credal network, never solves per-family local problems and never enumerates extreme points, so it is not affected by the relaxation those steps introduce.

Our second contribution is a systematic *credal-network compilation* of chain-graph LCNs, which makes the well-developed machinery of credal networks (Cozman, 2000; Mauá & Cozman, 2020; Ide & Cozman, 2008; Antonucci et al., 2015) available for LCN inference. The compilation replaces the LCN's distribution set by the strong extension of a credal network over the chain components, which contains it; we identify a class of *family-realizable* LCNs on which the two have the same bounds. On this basis we adapt three credal-network algorithms: credal variable elimination (CVE), exact for family-realizable LCNs with or without evidence, with its fully polynomial-time approximation scheme CVE$_\varepsilon$; interval belief propagation (IVBP), a credal sum-product algorithm, exact on family-realizable LCNs whose compiled credal network is a polytree, with or without evidence; and a coordinate-descent method (CDVE) that computes inner bounds. We evaluate all methods on synthetic and realistic LCN benchmarks against CJT as the exact reference.

The Appendix contains the proofs, detailed descriptions of all algorithms with worked examples, and the full experimental results.

2 Background

2.1 Credal Networks

A *credal network* (CN) (Cozman, 2000) is defined by a tuple $\langle \mathbf{V}, \mathbf{D}, \mathbf{K}, G\rangle$, where $\mathbf{V} = \{V_1, \ldots, V_m\}$ is a set of discrete (not necessarily binary) variables with finite domains $\mathbf{D} = \{D_1, \ldots, D_m\}$, G is a directed acyclic graph (DAG) over $\mathbf{V}$ as nodes, and $\mathbf{K} = \{K(V_i \mid \mathrm{pa}(V_i) = \pi_{ik})\}$ is a set of separately specified conditional credal sets for each variable V_i and each k-th configuration π_{ik} of its parents $\mathrm{pa}(V_i)$ in G. A credal set is a set of probability distributions (Levi, 1980) and we consider credal sets that are closed and convex with a finite number of vertices (e.g., probability intervals). Let $\mathcal{P}_{\mathrm{SE}}$ denote the set of joint distributions that factorize along G with every local conditional chosen, separately for each variable and parent configuration, from its credal set:

$$\mathcal{P}_{\mathrm{SE}} = \Big\{ P(\mathbf{V}) \;:\; P(\mathbf{V}) = \prod_{i=1}^{m} P(V_i \mid \mathrm{pa}(V_i)),\; P(V_i \mid \pi_{ik}) \in K(V_i \mid \pi_{ik})\; \forall i, k \Big\}. \tag{1}$$

The *strong extension* of the credal network is its convex hull (Cozman, 2000; Mauá & Cozman, 2020),

$$\mathcal{D}_{\mathrm{SE}} = \mathrm{CH}\big(\mathcal{P}_{\mathrm{SE}}\big), \tag{2}$$

whose extreme points are products of vertices of the local credal sets. $\mathcal{D}_{\mathrm{SE}}$ may contain mixtures violating the independences of G, but both sets have the same lower and upper probabilities, as marginals and (positive-denominator) conditionals are linear-fractional in P (Lemma 2). Set-level results below concern $\mathcal{P}_{\mathrm{SE}}$.

2.2 Logical Credal Networks

Credal networks attach credal sets to the families of a fixed DAG. A *Logical Credal Network* (LCN) (Marinescu et al., 2022) instead attaches probability bounds to arbitrary propositional formulas, and derives its independences from the formulas rather than from a given graph. Formally, an LCN consists of a set of binary variables (or propositions) $\mathbf{X} = \{X_1, \ldots, X_n\}$ and a set of constraints or probabilistic sentences $\mathcal{S}$. Each sentence $s \in \mathcal{S}$ is a constraint of a Type-1 form $\ell \le P(\varphi) \le u$, or a Type-2 form $\ell \le P(\varphi \mid \psi) \le u$ over propositional formulas. Since a Type-2 sentence is a ratio constraint which is undefined when its conditioning event has probability zero, we consider hereafter the algebraically *cleared* form:

$$\ell \cdot P(\psi) \;\le\; P(\varphi \wedge \psi) \;\le\; u \cdot P(\psi), \tag{3}$$

$0.1 \leq P(B) \leq 0.2$

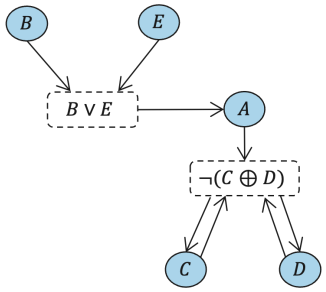
$0.05 \leq P(E) \leq 0.1$

$0.6 \leq P(A|B \vee E) \leq 0.9$

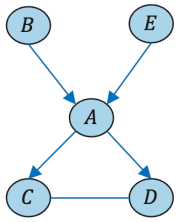
$0.5 \leq P(\neg(C \oplus D)|A) \leq 0.7$

$0.05 \leq P(A) \leq 0.1$

(a) LCN constraints



(b) LCN primal graph



(c) LCN structure (chain graph)

Figure 1: A simple LCN instance with its primal graph and corresponding structure.

which is total (defined for all p) and *bilinear* in p.

The *primal graph* G of an LCN contains a *proposition-node* for each of its propositions (Fig. 1). For each Type-1 sentence $\ell \leq P(\varphi) \leq u$, G contains a *formula-node* labeled by φ and bi-directed edges from φ to each proposition in φ. For each Type-2 constraint $\ell \leq P(\varphi \mid \psi) \leq u$, G contains formula nodes labeled by φ and ψ, directed edges from each proposition in ψ to ψ, a directed edge from ψ to φ, and bi-directed edges from each proposition in φ to φ, respectively (Marinescu et al., 2022).

The *structure* of an LCN is a graph that can be extracted from G as follows: (i) for each formula-node φ appearing in either a Type-1 or a Type-2 constraint, place an undirected edge between any two propositions in φ; (ii) for each pair of formula-nodes φ and ψ in a Type-2 constraint, add a directed edge from each proposition in ψ to each proposition in φ; (iii) if there is now a pair of edges $A \leftrightarrows B$ for some propositions A, B, replace both by an undirected edge; (iv) remove formula-nodes and all their edges as well as any duplicate edges (Cozman, 2023).

The parents of a proposition node X_i in G, denoted $\mathrm{pa}(X_i)$, are the proposition nodes that reach X_i via a directed path whose intermediate nodes (if any) are formula nodes. The descendants of X_i, denoted $\mathrm{de}(X_i)$, are the proposition nodes reachable from X_i via a directed path whose intermediate nodes do not belong to $\mathrm{pa}(X_i)$. An LCN is endowed with a Local Markov Condition (LMC(LCN)).

Definition 1 (LMC(LCN)). *Given an LCN $\mathcal{L}$ with primal graph G, every proposition X_i in $\mathcal{L}$ is independent of its non-parent non-descendant propositions conditioned on X_i's parents in G.*

Each LMC assertion $\iota : (X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ with X a singleton imposes, for *every* configuration $(\mathbf{y}, \mathbf{z})$ of $\mathbf{Y} \cup \mathbf{Z}$, the bilinear equality $P(x, \mathbf{y}, \mathbf{z})\, P(\mathbf{z}) = P(x, \mathbf{z})\, P(\mathbf{y}, \mathbf{z})$. Writing g_s for the functional of a sentence s and h_ι for the residual of an assertion ι, the *LCN distribution set* $\mathcal{D}_{\mathrm{LCN}}$ is:

$$\mathcal{D}_{\mathrm{LCN}} = \left\{ p \in \Delta(2^{\mathbf{X}}) \;:\; \ell_s \leq g_s(p) \leq u_s \;\forall s, \qquad h_\iota(p) = 0 \;\forall \iota \right\}, \tag{4}$$

where $\Delta(2^{\mathbf{X}})$ is the simplex of joint distributions of the 2^n truth assignments to $\mathbf{X}$. The *marginal inference* task for a query atom a is defined by:

$$\underline{P}(a) = \min_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a), \qquad \overline{P}(a) = \max_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a),$$

and analogously for a conditional $P(a \mid \mathbf{e})$ under evidence $\mathbf{e}$. The current state of the art consists of two approaches: the exact nonlinear optimization method of (Marinescu et al., 2022) and the approximate message-passing algorithm called ARIEL (Marinescu et al., 2023), respectively.

Example 1. *Figure 1 shows a simple LCN with 5 propositions and 5 constraints. The corresponding primal graph and structure are given in Figures 1b and 1c, respectively. The LMC implies that: B and E are marginally independent, C is conditionally independent of B and E given C's parents $\{A, D\}$, and D is conditionally independent of B and E given $\{A, C\}$, respectively.*

2.3 Chain Graphs and Factorization

Chain graphs (Cowell et al., 1999) generalize both DAGs and undirected graphs. Let $\mathbf{Y} = \{Y_1, \ldots, Y_n\}$ be a set of discrete random variables and $G = (\mathcal{N}, E)$ a graph whose nodes $\mathcal{N}$ are in one-to-one correspondence with the variables in $\mathbf{Y}$; we denote a configuration of $\mathbf{Y}$ by $\mathbf{y}$.

Definition 2 (chain graph). *Let $G = (\mathcal{N}, E)$ be a mixed graph with $E = E_{\rightarrow} \cup E_{-}$, where $E_{\rightarrow}$ is a set of directed edges and E_{-} is a set of undirected edges. The graph G is a* chain graph *if there exists a partition $\mathcal{N} = \mathcal{T}_1 \dot\cup \mathcal{T}_2 \dot\cup \cdots \dot\cup \mathcal{T}_m$ such that: (i) for each i, the subgraph induced by $\mathcal{T}_i$ contains only undirected edges; (ii) for $i \neq j$, every edge between $\mathcal{T}_i$ and $\mathcal{T}_j$ is directed, and all such edges have the same orientation; (iii) the directed graph obtained by contracting each chain component $\mathcal{T}_i$ into a single node is acyclic. The sets $\mathcal{T}_1, \ldots, \mathcal{T}_m$ are called the* chain components *of G.*

A DAG is a chain graph whose components are all singletons, and an undirected graph is a chain graph with a single component. If there is a directed edge from A to B in G then A is a *parent* of B and B is a *child* of A; the parents of A are denoted $\mathrm{pa}(A)$. If there is an undirected edge $A - B$ then both nodes are said to be *neighbors*; the neighbors of A are denoted by $\mathrm{ne}(A)$. The *boundary* of a node A is $\mathrm{bd}(A) = \mathrm{pa}(A) \cup \mathrm{ne}(A)$, and the boundary of a set $\mathcal{B}$ of nodes is $\mathrm{bd}(\mathcal{B}) = \bigcup_{A \in \mathcal{B}} \mathrm{bd}(A) \setminus \mathcal{B}$. A node B is a *descendant* of A if there is a path from A to B that follows every directed edge in its direction, uses undirected edges in either direction, and contains at least one directed edge. The Local Markov Condition (LMC(C)) for chain graphs is defined as follows:

Definition 3 (LMC(C)). *Let G be a chain graph. A node $A \in G$ is independent, given its boundary $\mathrm{bd}(A)$, of all nodes that are not A itself nor descendants of A nor boundary nodes of A.*

For strictly positive distributions the local and global Markov conditions of a chain graph are equivalent (Cowell et al., 1999), and a strictly positive joint distribution $P(\mathbf{Y})$ satisfying LMC(C) factorizes according to G as:

$$P(\mathbf{Y} = \mathbf{y}) = \prod_{i=1}^{m} P(\mathcal{T}_i = \mathbf{y}_{\mathcal{T}_i} \mid \mathrm{pa}(\mathcal{T}_i) = \mathbf{y}_{\mathrm{pa}(\mathcal{T}_i)}), \tag{5}$$

where the chain components are ordered so that directed edges can only point from earlier to later components, $\mathrm{pa}(\mathcal{T}_i)$ denotes the set of nodes outside $\mathcal{T}_i$ with directed edges into $\mathcal{T}_i$, and $\mathbf{y}_{\mathcal{T}_i}$ and $\mathbf{y}_{\mathrm{pa}(\mathcal{T}_i)}$ are the projections of $\mathbf{y}$ onto the corresponding variables. When G is a DAG positivity is not needed: its local Markov property implies equation 5 for every distribution. For components with several variables it is, since deriving the independence of a whole component from the node-wise conditions requires the intersection property (Appendix E).

Each chain component $\mathcal{T}_i$ can be regarded as a single variable V_i taking values in the joint states of its members; V_i is *compound*, and in general multi-valued, when $|\mathcal{T}_i| \geq 2$. Contracting the components turns G into a DAG D over $\mathbf{V} = \{V_1, \ldots, V_m\}$, the *component DAG*, and equation 5 is the Bayesian-network factorization of $P(\mathbf{V})$ along D. For a node $v \in \mathbf{V}$ we write $\mathrm{ch}_v \subseteq \mathbf{Y}$ for the variables of its component and pa_v for its parents in D; the pairs $(\mathrm{ch}_v, \mathrm{pa}_v)$ are the *families* of G, and the sets $\mathrm{ch}_v \cup \mathrm{pa}_v$ its *family scopes*.

Chain-graph LCNs. Cozman (2023) show that when the structure of an LCN is a chain graph over its propositions, $\mathbf{Y} = \mathbf{X}$, the LMC of the LCN (Definition 1) coincides with LMC(C), so every strictly positive distribution in $\mathcal{D}_{\mathrm{LCN}}$ factorizes as in equation 5. We call such an LCN a *chain-graph LCN* and consider only these hereafter; its components $\mathbf{V}$ and families are those of its structure. Results relying on equation 5 are stated for *factorizing* chain-graph LCNs: those whose structure is a DAG, or whose $\mathcal{D}_{\mathrm{LCN}}$ is restricted to strictly positive distributions.

3 Exact LCN Inference at Treewidth Cost

Exact inference in an LCN computes $\underline{P}(a) = \min_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a)$ and $\overline{P}(a) = \max_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a)$. Optimizing over $\mathcal{D}_{\mathrm{LCN}}$ directly requires a nonlinear program with one variable per joint state, 2^n in total (Marinescu et al., 2022). We introduce *Credal Junction Tree* (CJT), which solves the same problem exactly with a program whose size is exponential only in the treewidth of the chain graph.

CJT uses the chain graph only through its family scopes (Section 2). It moralizes and triangulates the chain graph, producing a chordal graph G^*, and builds a junction tree T whose clusters $\mathcal{C}$ satisfy the running-intersection property (Dechter, 2003). Instead of one variable per joint state, it introduces a local marginal $q_C \in \Delta(2^{|C|})$ for each cluster $C \in \mathcal{C}$ and solves (see the Appendix for details and a

worked example):

$$
\begin{array}{lll}
\min / \max & \rho_a(q) & \\
\text{s.t.} & \sum_i q_{C,i} = 1,\ q_{C,i} \geq 0 & \forall C \in \mathcal{C}, \\
& q_C \restriction S = q_D \restriction S & \forall (C, D) \in T,\ S = C \cap D, \\
& \ell \langle A_\psi, q_{C_h} \rangle \leq \langle A_{\varphi \wedge \psi}, q_{C_h} \rangle \leq u \langle A_\psi, q_{C_h} \rangle & \forall \text{ sentence hosted on } C_h, \\
& \langle A_a, q \rangle \langle A_b, q \rangle = \langle A_c, q \rangle \langle A_d, q \rangle & \forall \text{ LMC assertion hosted,}
\end{array}
$$

where A_φ is the $0/1$ indicator vector of φ over the host cluster's states, a constraint is *hosted* on any cluster containing its scope, $\langle A_\varphi, q \rangle$ is the dot-product of two vectors, and ρ_a reads $P(a)$ (or $P(a \mid \mathbf{e})$, via one auxiliary variable) off a cluster containing a and the evidence. For every pair of adjacent clusters C and D, the marginals induced on their separator S, denoted by $q_C \restriction S$ and $q_D \restriction S$, are required to coincide, ensuring global consistency. The rows for the independence assertions need care, since an assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ may involve more atoms than any cluster holds. CJT discharges each assertion in one of three ways (Appendix B.2). If $\mathbf{Z}$ separates X from $\mathbf{Y}$ in G^* the assertion is *separator-entailed*: every joint reconstructed from consistent cluster marginals satisfies it, so no row is needed. Otherwise $\mathbf{Y}$ can be replaced by its part $\mathbf{Y}_{\text{in}}$ still connected to X once $\mathbf{Z}$ is removed; the resulting assertion is equivalent to the original for every such joint, and is usually small enough to be hosted, if necessary after widening the tree. Alternatively, building the tree along a reverse topological order of the component DAG makes a single row per node imply all assertions at once.

Example 2. *For the LCN of Figure 1, G^* has the clusters $\{B, E, A\}$, $\{A, C, D\}$, $\{E, A\}$ and $\{E\}$. The assertions $(C \perp\!\!\!\perp B, E \mid A, D)$ and $(D \perp\!\!\!\perp B, E \mid A, C)$ are separator-entailed, because every path from C or D to B or E passes through A; the assertion $(B \perp\!\!\!\perp E)$ is not, since B and E are adjacent in G^*, and is imposed on $\{B, E, A\}$ as two bilinear rows. The program has 22 variables, against $2^5 = 32$ for the joint (Appendix C.1).*

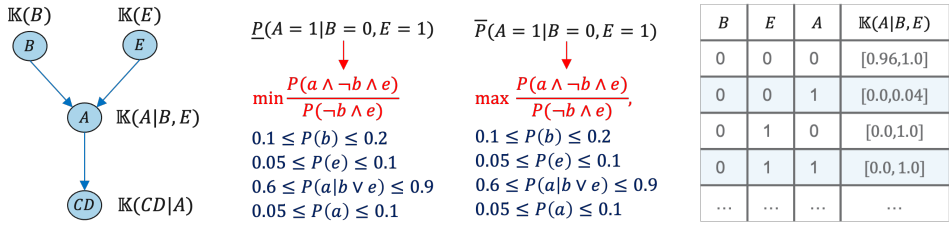
We now state when this program is exact. Conditions H1 and H2 concern the tree and the sentences: the clusters must satisfy the running-intersection property, and every sentence must fit in some cluster. Condition H3 concerns the independence assertions: the imposed rows must follow from the LMC, so that no member of $\mathcal{D}_{\text{LCN}}$ is excluded, and must in turn imply every LMC assertion on any joint assembled from the clusters, so that no distribution outside $\mathcal{D}_{\text{LCN}}$ is admitted. H4 asks that a conditional query and its evidence share a cluster; when they do not, CJT builds for that query a tree augmented by their node set. All four are checked by the algorithm for each instance. The program is nonconvex through the bilinear assertion rows; each endpoint is solved by spatial branch-and-bound (SCIP), and an endpoint the solver does not certify within its time limit is reported by its dual bound, so that every reported interval is an outer bound, and an exact one when H1–H4 hold and both endpoints are certified (Appendix B.3).

Theorem 1 (exactness)**.** *Let $\mathcal{L}$ be a chain-graph LCN and T a junction tree of its chordal graph G^*. Assume that* (H1) *T satisfies the running-intersection property;* (H2) *every probabilistic sentence is hosted by some cluster of T;* (H3) *the independence rows imposed on T are implied by the LMC and in turn imply every LMC assertion on every joint that factorizes over G^*;* (H4) *the query and evidence variables are contained in a common cluster. Then the global optimum of the CJT program is the exact lower and upper probability over $\mathcal{D}_{\text{LCN}}$ (for a conditional query, over the members of $\mathcal{D}_{\text{LCN}}$ that give the evidence positive probability, the conditional query being solved in Charnes–Cooper form). Without H2 and the second half of H3 it is still an outer bound.*

The proof (Appendix C.2) shows the two inclusions separately. Projecting any $p \in \mathcal{D}_{\text{LCN}}$ onto the clusters yields a feasible point with the same objective value, which gives the outer bound. Conversely, any feasible point is the set of cluster marginals of some joint distribution; that joint satisfies every sentence by H2 and every LMC assertion by H3, so it belongs to $\mathcal{D}_{\text{LCN}}$. The second step needs care when separator marginals have zeros, where the joint is not unique; the proof handles this case and uses no positivity assumption.

The size of the program is governed by the largest cluster of the tree.

Theorem 2 (size)**.** *Let $w + 1$ be the number of atoms in the largest cluster of T. The CJT program has $\sum_C 2^{|C|} = O(n\, 2^{w+1})$ variables, $O(n\, 2^w)$ separator-consistency constraints, two rows per sentence and at most 2^{w+1} rows per imposed independence assertion.*



B	E	A	𝕂(A|B,E)
0	0	0	[0.96,1.0]
0	0	1	[0.0,0.04]
0	1	0	[0.0,1.0]
0	1	1	[0.0, 1.0]
…	…	…	…

(a) Credal network (b) Local fractional LPs (c) Local credal set

Figure 2: The credal network for the chain-graph factorization of the LCN from Figure 1.

Here w is the width of the tree actually built, an upper bound on the treewidth of the moralized chain graph; widening (Appendix B.2) can increase it. The bound concerns the formulation, not the solving time: the program is nonconvex, and its global optimization is not polynomial in its size. Its benefit is that the program is exponential in w rather than in n, so exact inference becomes possible on instances where the 2^n formulation cannot even be written down. CJT never compiles a credal network: it uses neither per-family local credal sets nor their extreme points, and its target is $\mathcal{D}_{\text{LCN}}$ itself. Section 4 shows what is gained, and what is lost, by compiling instead.

4 Approximate Inference via Credal Network Compilation

An alternative to solving the LCN program, exactly as CJT does or approximately, is to compile the LCN into a credal network (Section 2.1) and use the algorithms developed for those. We describe the compilation, characterize when it is exact, and adapt three credal-network algorithms to it.

4.1 The Chain Graph Approximation for LCNs

Given a chain-graph LCN $\mathcal{L}$ over propositions $\mathbf{X} = \{X_1, \ldots, X_n\}$, we approximate $\mathcal{D}_{\text{LCN}}$ by the strong extension $\mathcal{D}_{\text{SE}}$ of an induced credal network over the component DAG D (Section 2). Each node takes values in the joint state space of its chain component, so a compound node is a multi-valued variable. Let v be a family, with child node ch_v and parent set pa_v. For each parent configuration $pa_v = \pi$, we define a local credal set $K(ch_v \mid pa_v = \pi)$ (or K_v for short) by computing lower and upper bounds on the conditional probabilities of ch_v given π. Following the standard LCN encoding of Marinescu et al. (2022), these bounds are obtained by solving a local optimization problem involving only the probabilistic sentences whose variables belong to $ch_v \cup pa_v$. The resulting fractional linear programs (LPs) are solved using the Charnes–Cooper transformation (Charnes & Cooper, 1962).

Example 3. *Figure 2a shows the credal network induced by the chain-graph factorization of the LCN from Figure 1. Any joint distribution in $\mathcal{D}_{\text{SE}}$ factorizes as $P(B) \cdot P(E) \cdot P(A \mid B, E) \cdot P(C, D \mid A)$. In Figure 2b we outline the fractional LPs for computing lower and upper bounds on $P(A = 1 \mid B = 0, E = 1)$, respectively. A partial local credal set $K(A \mid B, E)$ is given in Figure 2c.*

Given a reference credal set $\mathcal{R}$ with true query bounds $[\underline{q}^\star, \overline{q}^\star]$, an estimated interval is *exact* when it equals the true bounds, an *outer bound* when it encloses them, and an *inner bound* when it is enclosed by them. Intervals that satisfy neither property are termed *unreliable*. We can show that:

Proposition 1 (inclusion)**.** *For any factorizing chain-graph LCN, $\mathcal{D}_{\text{LCN}} \subseteq \mathcal{P}_{\text{SE}} \subseteq \mathcal{D}_{\text{SE}}$. Consequently, any inference algorithm that is* exact *or* outer *for $\mathcal{D}_{\text{SE}}$ computes valid* outer bounds *for $\mathcal{D}_{\text{LCN}}$.*

4.2 Family-Realizable Sentences

The proposed approximation introduces two looseness gaps. First, each K_v is computed per family in isolation, so it may admit conditionals that no global $p \in \mathcal{D}_{\text{LCN}}$ realizes – it ignores sentences on other families and the cross-family LMC equalities. Formally $K_v^{\text{true}} \subseteq K_v$, sometimes strictly. Second, even with $K_v = K_v^{\text{true}}$, the free product of equation 1 lets each family pick its row independently, which can violate a sentence whose scope spans several families. We therefore

introduce a class of LCN sentences, called *family-realizable sentences*, and identify conditions under which $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$, so that $\mathcal{D}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{LCN}}$ have the same bounds.

Definition 4 (family-realizable sentence)**.** *Let $\mathcal{L}$ be a chain-graph LCN with families $\{(\mathrm{ch}_v, \mathrm{pa}_v)\}$, and let s be a sentence with functional g_s. Say s is* family-realizable in family v *if* $\mathrm{atoms}(s) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$ *and the value $g_s(p)$ is determined by the conditional table $P(\mathrm{ch}_v \mid \mathrm{pa}_v)$* alone *— i.e. for any two joints p, p' that induce the same conditional table at v, $g_s(p) = g_s(p')$, whatever the parents' marginals $P(\mathrm{pa}_v)$ and $P'(\mathrm{pa}_v)$ may be. Concretely, s is family-realizable in v if and only if it has one of the two forms:*

(R1) Type-1 on a root block: *s is $\ell \le P(\varphi) \le u$ with $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$ and $\mathrm{pa}_v = \varnothing$. Then $P(\varphi)$ is a function of $P(\mathrm{ch}_v)$, which* is *the family's table.*

(R2) Type-2 pinning one full parent configuration: *s is $\ell \le P(\varphi \mid \psi) \le u$ with $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$, $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$, and ψ satisfied by* exactly one *assignment $\pi^\star$ of pa_v. Then $P(\varphi \mid \psi) = P(\varphi \mid \mathrm{pa}_v{=}\pi^\star)$ is a single row of the table.*

A sentence is non-realizable *if it is family-realizable in no family. The LCN is* fully realizable *if every sentence is family-realizable in some family*[1].

Proposition 2 (sufficiency)**.** *Let $\mathcal{L}$ be a consistent, factorizing chain-graph LCN with families $\{(\mathrm{ch}_v, \mathrm{pa}_v)\}$ and local credal sets built by the chain-graph approximation. If every sentence of $\mathcal{L}$ is family-realizable in some family (Definition 4), then $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$; hence $\mathcal{D}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{LCN}}$ have the same lower and upper probabilities for every marginal and every conditional query with positive denominator.*

The condition $K_{v,\pi} = K_{v,\pi}^{\mathrm{true}}$ (no local-set widening) is not an extra hypothesis: under full realizability it follows from consistency (Lemma 6 in the Appendix).

We next present three algorithms that compute, respectively, exact, outer and inner bounds for $\mathcal{D}_{\mathrm{SE}}$. By Proposition 1 the first two are outer bounds for $\mathcal{D}_{\mathrm{LCN}}$; an inner bound for $\mathcal{D}_{\mathrm{SE}}$ carries no guarantee for $\mathcal{D}_{\mathrm{LCN}}$ unless the two have the same bounds (Proposition 2).

4.3 Credal Variable Elimination

Credal Variable Elimination (CVE) extends Variable Elimination (VE) for Bayesian networks (Dechter, 2003) to credal networks by propagating finite sets of *pairs* of non-negative functions instead of single functions (Mauá et al., 2012; Mauá & Cozman, 2020).

Definition 5 (pair potential)**.** *A* potential $\phi(\mathbf{Y})$ *over variables $\mathbf{Y}$ is a finite set of pairs (p, q) of non-negative functions on the joint domain of $\mathbf{Y}$. The* product *of two potentials is $\phi \cdot \psi = \{(pr, qs) : (p, q) \in \phi,\ (r, s) \in \psi\}$ over $\mathbf{Y} \cup \mathbf{Z}$, and the* sum-marginal $\sum_X \phi = \{(\sum_x p, \sum_x q) : (p, q) \in \phi\}$ *sums both components.* Pruning *keeps the pairs that are maximal for the lower or for the upper bound: (p, q) is removed when pairs $(r, s) \neq (p, q)$ and $(r', s') \neq (p, q)$ of ϕ satisfy $r \le p,\ s \ge q$ and $r' \ge p,\ s' \le q$ pointwise.*

For an atom a of a variable Z, let t be the $0/1$ indicator of a over the states of Z. Every CPT enters as $\{(f, f)\}$, one f per choice of extreme points of the local credal sets, and the evidence as indicator pairs $\{(\delta_{\mathbf{y}}, \delta_{\mathbf{y}})\}$. The query potential $\{(1 - t,\ t)\}$ splits the mass, so that after all variables are eliminated each pair carries $p = P(\neg a, \mathbf{y})$ and $q = P(a, \mathbf{y})$ for one selection of extreme points. Then $\underline{P}(a \mid \mathbf{y})$ and $\overline{P}(a \mid \mathbf{y})$ are the minimum and maximum of $q/(p + q)$ over the final pairs. Since $q/(p + q)$ increases in q and decreases in p, and dominance is preserved by products and sums, pruning is lossless, with or without evidence. CVE eliminates Z last, so every pair has $p = q$ until the query enters. Each atom marginal then needs a single elimination per variable Z, shared by all of Z's atoms; compound variables need no projection. Before the query enters, a potential can moreover be reduced to the vertices of the convex hull of its (normalized) functions: $q/(p + q)$ is linear-fractional in each such function, so its extrema are attained at vertices (Appendix F).

FPTAS. Since the intermediate potentials can grow exponentially, we use the fully polynomial-time approximation scheme of Mauá et al. (2012). For $\alpha > 1$, two pairs are *α-equivalent* when every

[1] Note that a sentence can be attached to several families while being realizable in only one of them.

coordinate of both components is zero in both, or positive in both with the same $\lfloor \log_\alpha \cdot \rfloor$. The α-*quotient* keeps one real pair per equivalence class and is applied after every product and every summation. We denote this variant CVE$_\varepsilon$.

Theorem 3 (complexity)**.** *The time and space complexity of CVE is $O(m \cdot C \cdot d^{w^*})$ where m is the number of variables, d bounds the domain size, w^* is the treewidth, and C is the maximum potential cardinality (which can be exponential). For CVE$_\varepsilon$, C is polynomial in the input size and in $1/\varepsilon$ when w^* and d are bounded (Mauá et al., 2012).*

Theorem 4 (exactness)**.** *Let $\mathcal{L}$ be a consistent, factorizing, family-realizable chain-graph LCN. Then, for every atom a and evidence $\mathbf{y}$ with $\underline{P}(\mathbf{y}) > 0$, CVE computes the* exact bounds *on $P(a \mid \mathbf{y})$ with respect to $\mathcal{D}_{\mathrm{LCN}}$.*

Proposition 3 (FPTAS guarantee)**.** *Under the assumptions of Theorem 4, let N be the number of α-quotients in the elimination for a's variable, and let $\alpha = (1+\varepsilon)^{1/(2N)}$. Then CVE$_\varepsilon$ returns* inner *bounds within a factor $1+\varepsilon$: $\overline{P}/(1+\varepsilon) \le \overline{P}_\varepsilon \le \overline{P}$ and $\underline{P} \le \underline{P}_\varepsilon \le (1+\varepsilon)\,\underline{P}$.*

4.4 Interval Belief Propagation

Our second approach, Interval Belief Propagation (IVBP), runs the sum-product algorithm (Kschischang et al., 2001) on the *factor graph* of the credal network. There is one variable node per credal-network variable V, one family factor $f_V(V, \mathrm{pa}(V))$ holding the local credal sets $K(V \mid \pi)$, and one evidence factor holding the $0/1$ indicator of the observed atoms of V. For a fixed choice of local conditionals, sum-product sends

$$\mu_{v\to f}(x) = \prod_{g\in\mathcal{N}(v)\setminus f} \mu_{g\to v}(x), \qquad \mu_{f\to v}(x) = \sum_{\mathbf{x}_{\mathcal{N}(f)\setminus v}} \phi_f(x, \mathbf{x}_{\mathcal{N}(f)\setminus v}) \prod_{w\in\mathcal{N}(f)\setminus v} \mu_{w\to f}(x_w), \tag{6}$$

and the belief $b_v \propto \prod_{g\in\mathcal{N}(v)} \mu_{g\to v}$ equals $P(V \mid \mathbf{y})$ on a tree. As the local conditionals range over the strong extension, each message ranges over a set. Messages only matter up to scale, so IVBP propagates a *credal message*: the finite set of vertices of the convex hull of the normalized message vectors. For a binary variable this is an interval, and IVBP is the factor-graph form of 2U (Fagiuoli & Zaffalon, 1998): the messages to a child are 2U's π-messages, the messages to a parent its λ-messages.

Every update is a projective map (linear, then normalized) of each incoming message and of each row $P(V \mid \pi)$ separately. A projective map with a positive denominator maps the convex hull of a set onto the convex hull of its image, so the vertices are enough. The variable-to-factor update multiplies the vertices of the incoming sets. The update towards a child is linear in the rows, so for each combination of parent vertices it is a Minkowski sum of the rows' vertex sets. The update towards a parent U is a ratio in which each coordinate u depends only on the rows with $\pi_U = u$, so for each combination of incoming vertices it ranges over a box, whose normalized corners are the candidates. Belief vertices with zero mass are dropped (regular conditioning). The bounds of an atom a are the minimum and maximum of the linear functional $\sum_{s\models a} b_V(s)$ over the vertices of the belief. Messages are swept in topological and reverse topological order until they no longer change. On a loopy graph the sweeps stop when the change falls below a tolerance or after T sweeps, as in loopy 2U (Ide & Cozman, 2008).

Theorem 5 (complexity)**.** *One sweep of IVBP costs $O\big(\sum_V C_V\,(R_V\, e_V\, d + |\mathrm{pa}(V)|\, 2^d)\big)$ arithmetic operations, where C_V is the number of combinations of incoming message vertices at the factor f_V, R_V its number of rows, e_V the maximum number of vertices of a row, and d the maximum domain size, plus the cost of reducing each message to its hull vertices. On a polytree at most $D+1$ sweeps reach the fixed point, where D is the diameter of the factor graph.*

Theorem 6 (exactness)**.** *Let $\mathcal{L}$ be a consistent, factorizing, family-realizable chain-graph LCN whose compiled credal network is a polytree. Then, for every atom a and evidence $\mathbf{y}$ with $\underline{P}(\mathbf{y}) > 0$, IVBP computes the* exact bounds *on $P(a \mid \mathbf{y})$ with respect to $\mathcal{D}_{\mathrm{LCN}}$, for binary and multi-valued (compound) variables alike.*

On a polytree the inputs of every update depend on disjoint sets of local conditionals, so they vary independently, and every update is linear in each input before normalization; the hull vertices of the message sets are therefore propagated without loss. On loopy structures the inputs share parameters,

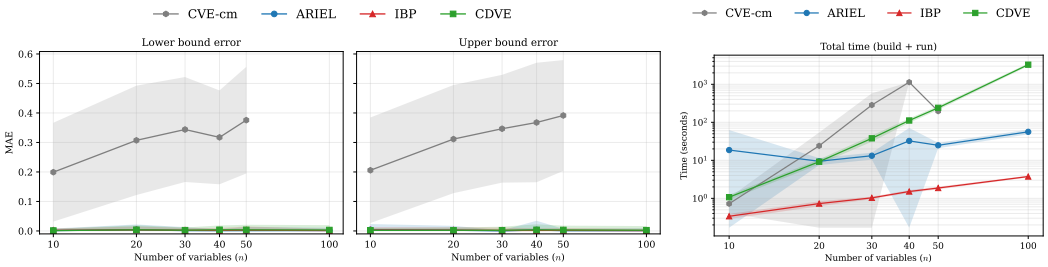


Figure 3: Large random `dag` LCNs: (left) MAEs against the CJT reference, with bands giving one standard deviation over 10 instances per size – curves stopping where the budget was exceeded; (right) total time in seconds.

and IVBP is a heuristic with no guarantee in either direction. With multi-valued variables the vertex sets can grow, consistently with the NP-hardness of exact polytree inference for ternary variables (Mauá et al., 2014). For binary variables every message has at most two vertices.

4.5 Coordinate Descent with Variable Elimination

Our third approach, Coordinate Descent with Variable Elimination (CDVE), follows ApproxLP (Antonucci et al., 2015). It optimizes the multilinear objective by coordinate descent over the local credal sets, one family at a time, and solves each local problem exactly. For an atom a of a variable Z and evidence $\mathbf{y}$, the objective is $P(a \mid \mathbf{y}) = \sum_{s \models a} P(Z{=}s, \mathbf{y}) / \sum_s P(Z{=}s, \mathbf{y})$. With every family but V fixed, both sums are linear in the rows of V: $P(\cdot, \mathbf{y}) = \sum_{x,\pi} P(x \mid \pi)\, T_V(x, \pi, Z)$, where T_V is the product of all other factors and of the evidence indicators, summed by variable elimination onto $\{V\} \cup \mathrm{pa}(V) \cup \{Z\}$. The local problem is therefore the linear-fractional program $\max_r \sum_\pi \langle n_\pi, r_\pi \rangle / \sum_\pi \langle d_\pi, r_\pi \rangle$ over the product of the rows' credal sets. Dinkelbach's method solves it exactly: for a given ratio t, every row independently takes the vertex that maximizes $\langle n_\pi - t\, d_\pi, v \rangle$, and t is updated until it no longer increases. A sweep applies this step to every family that is an ancestor of Z or of the evidence; the other families are barren and do not affect $P(Z, \mathbf{y})$. Sweeps repeat until none improves, at most T times, for the senses $\min$ and $\max$ and for every non-evidence atom, atoms of compound variables included.

Descent starts at the centroids of the local credal sets. From a centroid a family moves to a best vertex choice even when the ratio does not improve: the symmetric start is often a saddle point, where an uninformative child row hides its parent. To escape local optima, CDVE also restarts from R random vertex choices and keeps the best result. Every point visited is a member of $\mathcal{P}_{\mathrm{SE}}$ with $P(\mathbf{y}) > 0$, so the result is an *inner* bound for $\mathcal{D}_{\mathrm{SE}}$, with or without evidence. It is an inner bound for $\mathcal{D}_{\mathrm{LCN}}$ when $\mathcal{D}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{LCN}}$ have the same bounds (Proposition 2); otherwise its relation to the LCN interval is not determined.

Theorem 7 (complexity)**.** *CDVE runs in time* $O\big(A\,(R{+}1)\,T \sum_V (n\,k^{w^*+1} + I\,E_V\,k)\big)$ *and space* $O(n\,k^{w^*+1})$*, where* A *is the number of atoms,* n *the number of variables,* k *the maximum domain size,* w^* *the induced width of the elimination orders,* E_V *the number of extreme points of all rows of* V*, and* I *the number of Dinkelbach iterations per step, which is finite.*

5 Experiments

In this section, we evaluate the proposed marginal inference schemes on several classes of LCNs, comparing them with the exact method of Marinescu et al. (2022) and the approximate ARIEL algorithm (Marinescu et al., 2023). All algorithms are implemented in Python 3.12 and use `ipopt` 3.14 (Wächter & Biegler, 2006) for the nonlinear programs. Every run has a limit of 3600 seconds and 30 GB of memory; a dash in a table means the algorithm did not finish inside that budget.

Benchmarks. We consider two types of LCN instances. *Random LCNs* come in four topologies, each generated so that every sentence is *family-realizable*: `tree`, `polytree`, `ktree` (k=3) and `dag` (moralized treewidth ≤ 3, at most two parents per atom). Each topology is generated at two scales, *small* ($n \in \{5, \ldots, 10\}$) and *large* ($n \in \{10, 20, 30, 40, 50, 100\}$), with 10 instances per size, giving $8 \times 60 = 480$ instances. Sentence bounds are drawn as $[\max(0.01, v{-}\varepsilon), \min(1, v{+}\varepsilon)]$ with

Instance	Algorithm	MAE_l	MAE_u	Time (s)
Alarm (n=10), CJT ref 0.72s	ARIEL	0.000	0.000	3.92
	IVBP	0.101	0.015	0.13
	CDVE	0.101	0.015	0.69
	CVE_{cm}	0.087	0.067	0.24
Credit (n=11), CJT ref 1.15s	ARIEL	0.050	0.000	3.18
	IVBP	0.257	0.000	0.24
	CDVE	0.257	0.000	0.69
	CVE_{cm}	0.108	0.165	1.73
Hepatitis (n=10), CJT ref 0.56s	ARIEL	0.008	0.008	3.91
	IVBP	0.069	0.080	0.75
	CDVE	0.070	0.085	1.15
	CVE_{cm}	0.114	0.184	16.53
MScaling (n=12), CJT ref 0.66s	ARIEL	0.028	0.000	8.13
	IVBP	0.129	0.493	0.35
	CDVE	0.129	0.501	1.23
	CVE_{cm}	0.079	0.227	0.57
BScaling (n=32), CJT ref 2.42s	ARIEL	0.000	0.000	19.80
	IVBP	0.115	0.225	0.69
	CDVE	0.115	0.225	20.55
	CVE_{cm}	0.064	0.072	9.18
PScaling (n=40), CJT ref 3.27s	ARIEL	0.000	0.000	34.72
	IVBP	0.129	0.263	0.91
	CDVE	0.129	0.263	51.25
	CVE_{cm}	0.070	0.057	658.84

Table 1: Realistic LCNs: three instances derived from real Bayesian networks (top) and three Supreme Court chain-graph instances (bottom), all against the CJT reference. Time is total time including the credal-network build; the reference's own time on each instance is given with the instance name.

$v \sim U(0,1)$ and $\varepsilon = 0.3$; formulas span at most three atoms; two extra marginals per instance are placed on root atoms only; the seed is 42. Every instance is accepted only after a product-distribution consistency witness is found, so all 480 are consistent by construction. CJT is exact on each (Appendix I.1). ***Realistic LCNs*** are of two kinds: ten instances derived from real Bayesian networks (Constantinou et al., 2020) (n = 4–11, 6–24 sentences), and sixteen instances from the causal chain graphs of the US Supreme Court analysis of Ogburn et al. (2020). The latter stress two different axes: the `scaling` instances are large and sparse (n = 6–40, 11–79 sentences), whereas the `supreme_court` instances are small but extremely dense (n=10 with 345–1009 sentences).

Algorithms, reference and metrics. We compare four approximate algorithms: ARIEL (Marinescu et al., 2023), which operates directly on the LCN factor graph (10 iterations) and needs no credal-network build; IVBP, interval belief propagation on the credal network (100 iterations); CDVE, coordinate descent over extreme points (50 iterations), which by construction returns *inner* bounds; and CVE_{cm}, Credal Variable Elimination with its intermediate potentials clustered to at most 10 functions using the mean representative. Plain CVE (exact over the strong extension, no clustering) is included where it terminates. All credal-network methods share the same build: the chain-graph factorization, no scope merging, and per-family local credal sets solved as linear programs. As the reference we use CJT, the junction-tree NLP formulation (using the `ipopt` backend): it is the tightest engine tractable at every size here, whereas exact inference of Marinescu et al. (2022) over the 2^n joint solves only $15/60$ of `tree_small` and none of the Supreme Court instances. We report the mean absolute error of the lower and upper marginal bounds against the reference, $\mathrm{MAE}_l = \frac{1}{n}\sum_i |\underline{P}^*(x_i) - \underline{P}(x_i)|$ and MAE_u analogously, and wall-clock time in seconds including the credal-network build. Note that CJT's build is not comparable to the others': it needs only the chain-graph structure and skips the vertex enumeration entirely. All queries are marginals without evidence, the setting of Theorems 4 and 6: on these family-realizable instances CVE is exact on every topology and IVBP on the singly connected ones (`tree`, `polytree`).

Random LCNs. Figure 3 shows the loopy `dag` LCNs at the large scale, with shaded bands giving one standard deviation over the ten instances per size (full results on all LCNs are included in the Appendix). Three findings hold across all four topologies. First, IVBP dominates: its error against the reference is at or below 4×10^{-3} on every scale (small, large) and size — indistinguishable

from CJT on `tree` and `polytree`, as the exactness characterization predicts — while being the fastest method everywhere, by one to three orders of magnitude at n=100. Second, CDVE matches that accuracy but not that cost: its mean error stays in the 10^{-3} range, yet its runtime grows from under a second on the small instances to 273–614 seconds on the large ones, making it slower than ARIEL at n=100. ARIEL is likewise accurate (exactly zero error on `tree` and `polytree`) but pays 25–35 seconds at scale. Third, approximating the potentials by clustering buys speed at a real cost in quality: CVE$_{cm}$ runs in a fraction of a second on the small instances but its error grows with n, from 0.08–0.11 on the small instances to 0.31–0.35 on the large ones. Plain CVE is the scalability bottleneck: it is essentially exact where it runs but costs 226–1141 seconds, times out on all of `ktree_small` and `polytree_large`, and is intractable on the large loopy LCNs.

Realistic LCNs. Table 1 reports six representative instances. These instances are chain-graph LCNs but they are not family-realizable and therefore Proposition 2 does not hold. The ordering is different here: ARIEL is clearly the tightest method, at or near zero error on four of the six instances, because it reasons on the original LCN rather than on its credal-network relaxation. IVBP and CDVE agree closely and produce widths 1.4–7.7× the reference — genuine but loose outer bounds, the gap being the cost of the relaxation rather than a defect of the message passing. On running time the picture is again favourable to IVBP, which is the fastest method on every instance and the only one that stays under a second at n=40, where CDVE needs 51 seconds and CVE$_{cm}$ 659.

6 Related Work

Exact inference in LCNs has so far been limited to constrained nonlinear optimization over the full joint distribution (Marinescu et al., 2022), and approximate inference to the ARIEL message-passing algorithm (Marinescu et al., 2023); related work has addressed abductive reasoning (Marinescu et al., 2024) and the chain-graph factorization of LCNs (Cozman, 2023). CJT turns that factorization into an exact algorithm. It is related to junction-tree methods for Bayesian networks (Lauritzen & Spiegelhalter, 1988; Jensen et al., 1990), which compute marginals by propagating messages over a cluster tree; CJT uses the same running-intersection structure, but, since an LCN defines a nonconvex set of distributions rather than a single one, it imposes calibration as constraints of one optimization problem rather than computing it by message passing, and it must additionally discharge the LCN's independence assertions on the tree. Credal networks (CNs) (Cozman, 2000) generalize Bayesian networks by replacing precise conditional probabilities with credal sets; despite the hardness of exact inference (Mauá & Cozman, 2020; Mauá et al., 2014), the area has produced a rich algorithmic toolbox, including 2U/L2U (Ide & Cozman, 2008), GL2U (Antonucci et al., 2010) and ApproxLP (Antonucci et al., 2015). Our compilation makes this toolbox applicable to LCNs, and the family-realizability condition identifies when doing so loses no precision. More broadly, LCNs are related to probabilistic logic frameworks such as ProbLog (De Raedt et al., 2007), DeepProbLog (Manhaeve et al., 2021), MLNs (Richardson & Domingos, 2006) and PSL (Bach et al., 2017), but are distinguished by their support for imprecise probability assessments and minimal structural assumptions.

7 Conclusion

We introduced Credal Junction Tree (CJT), an exact inference algorithm for chain-graph Logical Credal Networks that replaces the 2^n-variable joint formulation by a nonlinear program over junction-tree cluster marginals, with size exponential in the treewidth rather than in the number of propositions. CJT needs only the chain graph's structure; it discharges the LCN's independence assertions on the tree by separation, exact reduction or an ordered construction, and we proved that under conditions it checks mechanically its certified optimum gives the exact lower and upper probabilities. As a complementary route, we compiled chain-graph LCNs into credal networks, characterized the family-realizable LCNs on which this compilation preserves the exact bounds, and adapted variable elimination, interval belief propagation and coordinate descent to it, with exactness guarantees for marginal queries without evidence. CJT also provides an exact reference against which the approximate methods can be measured. CJT already handles evidence that shares a cluster with the query; removing that requirement without widening the tree, and extending the compiled algorithms to evidence via the generalized Bayes rule, are natural next steps.

AI USE STATEMENT

We used generative AI tools to assist with language editing, grammar correction, and code development. In particular, AI-assisted coding tools were used during the implementation of the proposed algorithms. All AI-generated content was reviewed, verified, and, where applicable, extensively tested by the authors. For example, AI-generated code was manually inspected for correctness and validated through unit and integration testing. The authors take full responsibility for the accuracy, validity, and integrity of all content presented in this paper.

ETHICS STATEMENT

This work did not involve human subjects.

REPRODUCIBILITY STATEMENT

To ensure reproducibility, we have included the Python code and benchmark problem instances in the supplementary material. We will open-source the code and benchmarks if the paper is accepted.

REFERENCES

A. Antonucci, C. P. de Campos, D. Huber, and M. Zaffalon. Approximate credal network updating by linear programming with applications to decision making. *International Journal of Approximate Reasoning*, 58:25–38, 2015.

Alessandro Antonucci, Yi Sun, Cassio De Campos, and Marco Zaffalon. Generalized loopy 2U: A new algorithm for approximate inference in credal networks. *International Journal of Approximate Reasoning*, 51(5):474–484, 2010.

Stephen Bach, Matthias Broecheler, Bert Huang, and Lise Getoor. Hinge-loss markov random fields and probabilistic soft logic. *Journal of Machine Learning Research (JMLR)*, 18(1):1–67, 2017.

A. Charnes and W. W. Cooper. Programming with linear fractional functionals. *Naval Research Logistics Quarterly*, 9(3-4):181–186, 1962.

Anthony Constantinou, Yang Liu, Kiattikun Chobtham, Zhigao Guo, and Neville Kitson. The bayesys data and bayesian network repository. Technical report, Bayesian Artificial Intelligence research lab, Queen Mary University of London, London, UK, 2020.

Robert Cowell, Philip Dawid, Steffen Lauritzen, and David Spiegelhalter. *Probabilistic Networks and Expert Systems*. Springer-Verlag, New York, 1999.

Fabio Cozman. Credal networks. *Artificial Intelligence*, 120(2):199–233, 2000.

Fabio Gagliardi Cozman. Markov conditions and factorization in logical credal networks. In Enrique Miranda, Ignacio Montes, Erik Quaeghebeur, and Barbara Vantaggi (eds.), *Proceedings of the Thirteenth International Symposium on Imprecise Probability: Theories and Applications*, volume 215 of *Proceedings of Machine Learning Research*, pp. 130–140. PMLR, 11–14 Jul 2023. URL `https://proceedings.mlr.press/v215/cozman23a.html`.

Luc De Raedt, Angelika Kimmig, and Hannu Toivonen. Problog: A probabilistic prolog and its application in link discovery. In *Proceedings of the International Joint Conference on Artificial Intelligence*, volume 7, pp. 2462–2467, 2007.

R. Dechter. Bucket elimination: A unifying framework for probabilistic inference algorithms. In *Uncertainty in Artificial Intelligence (UAI'96)*, pp. 211–219, 1996.

R. Dechter. *Constraint Processing*. Morgan Kaufmann Publishers, 2003.

Enrico Fagiuoli and Marco Zaffalon. 2U: An exact interval propagation algorithm for polytrees with binary variables. *Artificial Intelligence*, 106(1):77–107, 1998.

Jaime Ide and Fabio Cozman. Approximate algorithms for credal networks with binary variables. *International Journal of Approximate Reasoning*, 48(1):275–296, 2008.

F.V. Jensen, S.L Lauritzen, and K.G. Olesen. Bayesian updating in causal probabilistic networks by local computation. *Computational Statistics Quarterly*, 4:269–282, 1990.

Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.

S.L. Lauritzen and D.J. Spiegelhalter. Local computation with probabilities on graphical structures and their application to expert systems. *Journal of the Royal Statistical Society, Series B*, 50(2): 157–224, 1988.

I. Levi. *The Enterprise of Knowledge*. MIT Press, 1980.

Robin Manhaeve, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. Neural probabilistic logic programming in deepproblog. *Artificial Intelligence*, 298:103504, 2021. ISSN 0004-3702. doi: https://doi.org/10.1016/j.artint.2021.103504. URL https://www.sciencedirect.com/science/article/pii/S0004370221000552.

Radu Marinescu, Haifeng Qian, Alexander Gray, Debarun Bhattacharjya, Francisco Barahona, Tian Gao, Ryan Riegel, and Pravinda Sahu. Logical credal networks. In *36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

Radu Marinescu, Haifeng Qian, Alexander Gray, Debarun Bhattacharjya, Francisco Barahona, Tian Gao, and Ryan Riegel. Approximate inference in logical credal networks. In *32nd International Joint Conference on Artificial Intelligence (IJCAI)*, 2023.

Radu Marinescu, Fabio Cozman, Debarun Bhattacharjya, Junkyu Lee, and Alexander Gray. Abductive reasoning in logical credal networks. In *38th Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

Denis Mauá and Fabio Cozman. Thirty years of credal networks: Specifications, algorithms and complexity. *International Journal of Approximate Reasoning*, 1(126):133–137, 2020.

Denis D. Mauá, Cassio P. de Campos, and Marco Zaffalon. Updating credal networks is approximable in polynomial time. *International Journal of Approximate Reasoning*, 53(8):1183–1199, 2012.

Denis D. Mauá, Cassio P. de Campos, Alessio Benavoli, and Alessandro Antonucci. Probabilistic inference in credal networks: New complexity results. *Journal of Artificial Intelligence Research*, 50, 2014.

Elizabeth L. Ogburn, Ilya Shpitser, and Youjin Lee. Causal inference, social networks, and chain graphs, 2020. URL https://arxiv.org/abs/1812.04990.

Judea Pearl. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann, 1988.

Matthew Richardson and Pedro Domingos. Markov logic networks. *Machine Learning*, 62(1-2): 107–136, 2006.

Andreas Wächter and Lorenz Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.

A PRELIMINARIES

A Logical Credal Network specifies imprecise probabilistic knowledge by bounding the probabilities of propositional formulas: a *sentence* is either $\ell \leq P(\varphi) \leq u$ (Type-1) or $\ell \leq P(\varphi \mid \psi) \leq u$ (Type-2), over propositional formulas φ, ψ on binary atoms. Together with the independence assumptions that the Local Markov Condition derives from the program's structure, a set of sentences determines a closed set $\mathcal{D}_{\mathrm{LCN}}$ of joint distributions, and the inference task is to compute $\underline{P}(a) = \min_{p \in \mathcal{D}_{\mathrm{LCN}}} P_p(a)$ and $\overline{P}(a) = \max_{p \in \mathcal{D}_{\mathrm{LCN}}} P_p(a)$ for each atom a.

This paper relates three ways of computing these bounds. The *direct* exact method of Marinescu et al. (2022) writes one variable per joint state and optimizes over $\mathcal{D}_{\mathrm{LCN}}$ itself; it is exact when

its solver certifies global optimality, but its 2^n formulation confines it to small instances. CJT, the main contribution (Section 3), also optimizes over $\mathcal{D}_{\mathrm{LCN}}$, but through cluster marginals of a junction tree, at cost $\sum_C 2^{|C|} = O(n\, 2^{w+1})$; it consumes only the chain graph's *structure* and the LCN's own sentences and independence assertions, and under the hypotheses of Theorem 1 it is exact (Appendices B–C). The *credal-network* methods (CVE, IVBP, CDVE; Section 4) first compile the LCN into a credal network with interval local credal sets and then propagate over the strong extension $\mathcal{D}_{\mathrm{SE}} \supseteq \mathcal{D}_{\mathrm{LCN}}$; they scale differently, and are exact for $\mathcal{D}_{\mathrm{LCN}}$ only on family-realizable instances (Appendices D–E).

A.1 Notation

Throughout, boldface $\mathbf{X} = \{X_1, \ldots, X_n\}$ denotes a set of *binary* variables — the propositions, or atoms, of an LCN — and boldface $\mathbf{V} = \{V_1, \ldots, V_m\}$ a set of discrete variables that need not be binary, such as the variables of a compiled credal network, one per chain component (a compound node takes values in the joint states of its component's atoms). $\Delta(2^{\mathbf{X}})$ is the simplex of joint distributions over the 2^n truth assignments of $\mathbf{X}$, and $p \restriction A$ the marginal of p on an atom subset $A \subseteq \mathbf{X}$. For a propositional formula φ we write $\mathrm{atoms}(\varphi)$ for its atoms and $\omega \models \varphi$ for satisfaction.

A.2 The credal set $\mathcal{D}_{\mathrm{LCN}}$

A Type-1 sentence $\ell \le P(\varphi) \le u$ contributes the two linear rows $\ell \le \sum_{\omega \models \varphi} p(\omega) \le u$. A Type-2 sentence $\ell \le P(\varphi \mid \psi) \le u$ is read in *cleared* form,

$$\ell \cdot P(\psi) \;\le\; P(\varphi \wedge \psi) \;\le\; u \cdot P(\psi), \tag{7}$$

which avoids a division and is the form the implementation imposes.

Remark 1 (The cleared reading is the one in force)**.** *The cleared rows equation 7 are vacuously satisfied on $\{p : P(\psi) = 0\}$, so they admit joints that the ratio reading $\ell \le P(\varphi \wedge \psi)/P(\psi) \le u$ excludes. The two readings therefore define different credal sets. Every statement in this paper is made for the cleared reading, matching our implementation as well.*

The Local Markov Condition attaches to the program a set of conditional independence assertions $(X \perp\!\!\!\perp Y \mid S)$ with X a singleton. Each is imposed as the family of *joint* equalities

$$P(\mathbf{x}, \mathbf{y}, \mathbf{s})\, P(\mathbf{s}) = P(\mathbf{x}, \mathbf{s})\, P(\mathbf{y}, \mathbf{s}), \tag{8}$$

one for each configuration $(\mathbf{y}, \mathbf{s})$ of $Y \cup S$. Writing g_s for the functional of a sentence s and h_ι for the residual of an assertion ι,

$$\mathcal{D}_{\mathrm{LCN}} = \big\{\, p \in \Delta(2^{\mathbf{X}}) \;:\; \ell_s \le g_s(p) \le u_s \;\forall s, \qquad h_\iota(p) = 0 \;\forall \iota \,\big\}, \tag{9}$$

and the inference targets are $\underline{P}(a) = \min_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a)$, $\overline{P}(a) = \max_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a)$.

The following elementary lemma is the workhorse of our theoretical contribution: it is what licenses imposing a constraint on a single cluster.

Lemma 1 (Locality)**.** *Every constraint in equation 9 depends on p only through its marginal on a fixed atom set, its* scope*: a sentence s through $p \restriction \mathrm{atoms}(s)$, and an LMC assertion $(X \perp\!\!\!\perp Y \mid S)$ through $p \restriction (X \cup Y \cup S)$. Consequently, if p and p' satisfy $p \restriction A = p' \restriction A$ for the scope A of a constraint c, then c holds for p iff it holds for p'.*

Proof. Let c be a constraint with scope A. There are three cases.

Type-1. $P(\varphi) = \sum_{\omega \models \varphi} p(\omega)$. Since $\mathrm{atoms}(\varphi) \subseteq A$, the truth value of φ at ω is determined by $\omega \restriction A$; grouping the sum by the value of $\omega \restriction A$ gives $P(\varphi) = \sum_{\alpha \models \varphi} (p \restriction A)(\alpha)$, where α ranges over assignments of A. This is a function of $p \restriction A$ alone.

Type-2. By the previous case both $P(\varphi \wedge \psi)$ and $P(\psi)$ are functions of $p \restriction A$ with $A \supseteq \mathrm{atoms}(\varphi) \cup \mathrm{atoms}(\psi)$; the rows equation 7 are therefore functions of $p \restriction A$.

LMC. Each factor in equation 8 is a probability of a conjunction of literals over $X \cup Y \cup S$, hence by the Type-1 case a function of $p \restriction (X \cup Y \cup S)$; a product of such functions is again one.

In each case the constraint's truth value is determined by $p \restriction A$, which is precisely the stated consequence. □

A.3 CHAIN GRAPH, STRONG EXTENSION, AND THE TWO GAPS

The LCN pipeline moralizes and simplifies the program's structure graph into a *chain graph* whose nodes are chain components; an undirected component collapses into a single *compound* node (rendered with a −-joined name, e.g. `C−D`). This yields a factorization

$$P(\mathbf{X}) = \prod_v P(\mathrm{ch}_v \mid \mathrm{pa}_v), \tag{10}$$

one factor per node v, where ch_v is the (possibly multi-atom) chain component at v and pa_v its parents. The *family scope* at v is $\mathrm{ch}_v \cup \mathrm{pa}_v$; root families have $\mathrm{pa}_v = \varnothing$.

The credal-network based methods replace each factor by an interval local credal set $K_{v,\pi}$ per parent configuration π and work with the *product set*

$$\mathcal{P}_{\mathrm{SE}} = \left\{ \textstyle\prod_v P_v \ : \ P_v(\cdot \mid \pi) \in K_{v,\pi} \text{ chosen independently for each } v \text{ and parent configuration } \pi \right\} \tag{11}$$

and its convex hull, the *strong extension* $\mathcal{D}_{\mathrm{SE}} = \mathrm{CH}(\mathcal{P}_{\mathrm{SE}})$. Mixtures of products need not factorize, so $\mathcal{D}_{\mathrm{SE}}$ may contain distributions violating the independences of the chain graph; but the two sets have the same bounds.

Lemma 2 (Hull invariance of bounds)**.** *Let S be a compact set of distributions and $f(p) = \langle a, p\rangle / \langle b, p\rangle$ with $\langle b, p\rangle > 0$ on $\mathrm{CH}(S)$ (for a marginal, $b = \mathbf{1}$). Then $\min_{\mathrm{CH}(S)} f = \min_S f$ and $\max_{\mathrm{CH}(S)} f = \max_S f$.*

Proof. Let $p = \sum_k \lambda_k p_k$ be a convex combination of points of S. Then $f(p) = \sum_k \mu_k f(p_k)$ with $\mu_k = \lambda_k \langle b, p_k\rangle / \sum_j \lambda_j \langle b, p_j\rangle \geq 0$ summing to 1, so $f(p)$ lies between $\min_k f(p_k)$ and $\max_k f(p_k)$. The extrema over S exist by compactness and continuity. □

Every statement below about *sets* is made for $\mathcal{P}_{\mathrm{SE}}$; by Lemma 2 it transfers to the *bounds* of $\mathcal{D}_{\mathrm{SE}}$. The inclusion $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{P}_{\mathrm{SE}}$ is generally strict, for two reasons that it is worth naming since they are exactly what CJT avoids:

- **Gap A (local-set widening).** Each K_v is computed per family in isolation, so it may admit conditionals that no global $p \in \mathcal{D}_{\mathrm{LCN}}$ realizes: it ignores sentences on *other* families and the cross-family LMC equalities.
- **Gap B (strong-extension widening).** Even with exact local sets, the free product in equation 11 lets each family pick its row independently, which can violate a sentence whose scope spans several families.

CJT reads neither the interval bounds nor the extreme points of any $K_{v,\pi}$; it uses equation 10 only for its *scopes*. Neither gap therefore arises, and the engine's target is $\mathcal{D}_{\mathrm{LCN}}$ itself. This is the structural reason it is the only engine in the credal-network directory whose bound character is stated against $\mathcal{D}_{\mathrm{LCN}}$ rather than $\mathcal{D}_{\mathrm{SE}}$.

Definition 6 (Bound character)**.** *Let $\mathcal{R}$ be a reference set (here $\mathcal{D}_{\mathrm{LCN}}$ or $\mathcal{D}_{\mathrm{SE}}$) with true bounds $[\underline{q}^\star, \overline{q}^\star]$ for the query at hand. An engine's interval $[\underline{q}, \overline{q}]$ is* exact w.r.t. $\mathcal{R}$ *if $[\underline{q}, \overline{q}] = [\underline{q}^\star, \overline{q}^\star]$; a valid* outer *bound if $[\underline{q}, \overline{q}] \supseteq [\underline{q}^\star, \overline{q}^\star]$; an* inner *bound if $[\underline{q}, \overline{q}] \subseteq [\underline{q}^\star, \overline{q}^\star]$; and* unreliable *if neither containment is guaranteed. An outer bound is sound as a guarantee; an inner bound is not.*

B DETAILED DESCRIPTION OF THE CJT ALGORITHM

CJT proceeds in four steps. It builds a cluster tree over the atoms from the family scopes of the chain graph equation 10; it assigns every sentence to a cluster containing its atoms, and discharges every independence assertion by separation, by an equivalent reduced assertion hosted on a cluster, or by the rows of an ordered tree; it writes a single constraint system whose variables are one distribution per cluster; and it reads the query off a cluster containing it, minimizing and maximizing over that system. Algorithm 1 states the procedure; Section C writes the constraint system formally.

The only input the algorithm takes from the chain graph is the collection of family *scopes* $\{\mathrm{ch}_v \cup \mathrm{pa}_v\}_v$, together with the LCN's own sentences and assertions. It never forms the local credal sets $K_{v,\pi}$ of

Algorithm 1 CJT: marginal bounds at treewidth cost

Require: sentences Σ, LMC assertions Λ, component DAG D with family scopes $\mathcal{F}$, evidence e, width budget b
Ensure: $[\underline{P}(a \mid e),\ \overline{P}(a \mid e)]$ for every atom $a \notin \mathrm{atoms}(e)$

$\mathcal{S} \leftarrow \mathcal{F} \cup \{\mathrm{atoms}(s) : s \in \Sigma \text{ cross-family}\}$
$T_o \leftarrow \textsc{BucketTree}(\mathcal{S},\ \text{reverse topological order of } D);\ \Lambda_o \leftarrow \textsc{Ordered}(T_o)$ ▷ Prop. 4
$T_m \leftarrow \textsc{BucketTree}(\mathcal{S}, \textsc{MinFillOrder}(\mathcal{S}));\ (T_m, \Lambda_m) \leftarrow \textsc{Triage}(\Lambda, T_m, b)$ ▷ §B.2
$(T, \Lambda_T) \leftarrow$ narrowest candidate on which H3 holds ▷ RIP holds [H1]
for all $s \in \Sigma$ **do** ▷ every sentence must be hosted [H2]
  host s on a cluster C with $\mathrm{atoms}(s) \subseteq A(C)$, if one exists
if $\max_C |A(C)| > b$ **or** some $s \in \Sigma$ was not hosted **then**
  return $\textsc{Direct}(a, e)$ for every atom $a \notin \mathrm{atoms}(e)$ ▷ 2^n, uncertified; Rem. 4
build $\mathcal{G}$: $q_C \in \Delta(2^{|A(C)|}),\ q_C \restriction S_{CD} = q_D \restriction S_{CD}$, equation 18–equation 19 on each host, equation 20 for Λ_T ▷ once [H3]
for all atoms $a \notin \mathrm{atoms}(e)$ **do**
  if $\{a\} \cup \mathrm{atoms}(e) \subseteq A(C_a)$ for some C_a **then** ▷ objective only [H4]
    $\underline{P} \leftarrow \min_{q \in \mathcal{G}} \rho_a(q),\ \overline{P} \leftarrow \max_{q \in \mathcal{G}} \rho_a(q)$ ▷ certified or dual bound, Rem. 2
  else
    $[\underline{P}, \overline{P}] \leftarrow$ solve on a tree augmented by $\{a\} \cup \mathrm{atoms}(e)$ ▷ or DIRECT
  clamp $[\underline{P}, \overline{P}]$ to $[0, 1]$

equation 11, and never enumerates their extreme points. This is the structural reason its target is $\mathcal{D}_{\mathrm{LCN}}$ rather than $\mathcal{D}_{\mathrm{SE}}$: neither Gap A nor Gap B can arise from a construction that does not build the objects in which those gaps live.

Throughout, $\mathcal{C}$ denotes the clusters, $A(C) \subseteq \mathbf{X}$ the atoms of cluster C, $S_{CD} = A(C) \cap A(D)$ the separator on a tree edge, and

$$w + 1 = \max_{C \in \mathcal{C}} |A(C)|, \tag{12}$$

so w is the width of T — an upper bound on the treewidth of the moralized chain graph — and $w + 1$ the largest cluster measured in atoms. Cluster sizes are always reported in atoms below.

B.1 Constructing the cluster tree

The scopes to be covered are the family scopes, augmented by one clique per *cross-family sentence* — a sentence whose atoms fit inside no single family scope. Augmenting forces such a sentence's atoms to co-occur in a cluster, which is what guarantees it a host in Section B.2.

Independence assertions are *not* used to augment wholesale: most are entailed by the tree's topology (Section B.2), and adding their full scopes as cliques would inflate the width for no gain — a Markov chain would collapse from width 2 to a single cluster containing all n atoms.

Two elimination orders are tried. The *min-fill* order is computed greedily on the interaction graph — the graph in which two nodes are adjacent when they co-occur in some scope — eliminating at each step a vertex whose elimination adds fewest fill-in edges. The *ordered* order eliminates the nodes of the chain-graph component DAG D in *reverse topological* order; on the resulting tree every LMC assertion is discharged by one hosted row per node, with no widening (Proposition 4). The algorithm keeps the narrower of the trees on which H3 holds. For either order, buckets are formed along the order, each scope placed in the bucket of the first order variable it contains; a bucket's cluster is the union of its scopes, and its parent is the first later order variable appearing in the cluster after the bucket variable is removed. This is standard bucket elimination, so **the resulting tree has the running-intersection property by construction**, discharging hypothesis H1 of Theorem 8 without further argument.

Each cluster's atom set $A(C)$ is the union of the atoms of its nodes; a compound node contributes all atoms of its chain component at once, so a cluster may be wider in atoms than in nodes. Finally, any *leaf* whose atom set is contained in its parent's is removed, repeatedly: such a leaf is redundant, since the parent already carries that joint, and removing it is safe because a leaf has no children to reattach. Only subsumed leaves are removed, so the tree is not claimed to be minimal — a non-maximal cluster may survive in the interior.

B.2 HOSTING AND THE ASSERTION TRIAGE

Say a constraint is *hosted* on a cluster C when $\mathrm{scope}(c) \subseteq A(C)$; by Lemma 1 its value is then a function of q_C alone. Hosting each constraint on a single cluster suffices: separator consistency propagates its effect to the rest of the tree, as the reconstruction argument of Theorem 8 makes precise.

Every sentence is hosted if it can be. A sentence with no host cannot be represented exactly on this tree, and the algorithm falls back to the direct formulation (Section B.3).

Independence assertions are discharged in one of two ways, depending on the tree.

Min-fill tree: reduce, host, widen, relax. For an assertion $(X \perp\!\!\!\perp Y \mid Z)$ let K be the set of atoms reachable from X in G^* after deleting Z, and $Y_{\mathrm{in}} = Y \cap K$.

1. **Reduce.** Replace the assertion by $(X \perp\!\!\!\perp Y_{\mathrm{in}} \mid Z)$. By Lemma 4 the two are *equivalent* for every joint the cluster program can reconstruct, so nothing is lost. If $Y_{\mathrm{in}} = \varnothing$ the assertion is *separator-entailed* — Z separates X from Y in G^* — and nothing is imposed; this is what keeps a chain at width 2.
2. **Host.** If $X \cup Y_{\mathrm{in}} \cup Z \subseteq A(C)$ for some cluster C, impose the reduced assertion there as the bilinear equalities equation 8.
3. **Widen.** Otherwise add $X \cup Y_{\mathrm{in}} \cup Z$ as an augmenting clique and rebuild the tree; repeat to a fixed point while the largest cluster stays within the budget b.
4. **Relax.** An assertion still unhostable is replaced by its hostable consequences $(X \perp\!\!\!\perp y \mid Z)$, $y \in Y_{\mathrm{in}}$. Each is implied by the original (decomposition), so this is sound; but H3 fails and the bound is a valid outer bound that may be loose.

Ordered tree: one row per node. For every node x with elimination clique $\{x\} \cup N_x$, impose the single assertion $(x \perp\!\!\!\perp N_x \setminus \mathrm{pa}(x) \mid \mathrm{pa}(x))$ on x's cluster. Its scope is that cluster, so it is always hosted; it is checked to follow from an LCN assertion by symmetry and decomposition (soundness); and by Proposition 4 these rows imply *every* LMC assertion, so H3 holds by construction. A compound child $x = (x_1, \ldots, x_k)$ is imposed through the equivalent chain $x_i \perp\!\!\!\perp N_x \setminus \mathrm{pa}(x) \mid \mathrm{pa}(x), x_{<i}$, $i = 1, \ldots, k$ (contraction one way, weak union and decomposition the other; no positivity).

Example 4 (Reduction)**.** *Consider an LCN over $x_0, \ldots, x_5$ in which x_2 and x_4 are roots, x_3 has parents x_2, x_4 with $0.096543 \le P(x_3 \mid x_2 \wedge x_4) \le 0.696543$, and x_0, x_1 form a separate component. The LMC gives the assertion $(x_2 \perp\!\!\!\perp x_0, x_1, x_4)$, whose four atoms exceed the largest cluster (3 atoms). In G^* the atoms x_0, x_1 lie in a different connected component from x_2, so the reduction yields the equivalent $(x_2 \perp\!\!\!\perp x_4)$, which is hosted on $\{x_2, x_3, x_4\}$. With $0.673737 \le P(x_2)$ and $0.349412 \le P(x_4)$ the exact bound is $\underline{P}(x_3) = 0.673737 \cdot 0.349412 \cdot 0.096543 = 0.022727$; without the assertion the program could correlate x_2 and x_4 negatively and would return only $0.096543\,(0.673737 + 0.349412 - 1) = 0.0022$.*

B.3 SOLVING

The constraint system of Step 3 mentions no query, and every atom lies in some cluster because each family scope is itself a clique. One system therefore serves all marginals: per atom, only the objective changes, and two optimizations are solved. This is the sense in which the construction is *query-independent*.

Global solution and certification. The system is nonconvex: the assertion equalities equation 20 are bilinear, while the conditional read-off is linear after the scaling equation 22. Exactness in Theorem 8 refers to the global optimum, so each endpoint is computed by a spatial branch-and-bound solver (SCIP) and recorded with its certificate.

Remark 2 (Uncertified endpoints are reported by their dual bound)**.** *When branch-and-bound stops before closing the gap (e.g. at its time limit), the incumbent of a minimization lies* above *the true minimum, so reporting it would make the interval an unsound inner bound.* CJT *reports the solver's* dual *bound instead, which is $\le$ the true minimum (resp. $\ge$ the true maximum). An endpoint is* certified *when the solver proves optimality or its primal and dual bounds agree to within 10^{-4}; the reported*

value is the dual bound in either case. Every reported interval is therefore an outer bound, and it is exact when H1–H4 hold and both endpoints are certified.

Remark 3 (A local solver yields an inner bound)**.** *Solved instead by a local method from a finite set of starting points — keeping the smallest minimum and largest maximum found — the procedure returns a value no more extreme than the true optimum in each direction. The resulting interval is therefore contained in the exact one: an* inner *bound in the sense of Definition 6, and not a guarantee. Global solution is what makes Theorem 8 applicable.*

Query and evidence in different clusters. When a query atom and the evidence share no cluster of the shared tree (H4 fails for that atom), CJT builds for that atom a tree whose scopes are augmented by the query and evidence node set, which restores H4, runs the same triage on it, and solves the conditional query there.

Fallback. The direct 2^n formulation is used only when a tree — the shared one, or the augmented one of a query — exceeds the width budget b, or a sentence has no host.

Remark 4 (The fallback is exact in formulation, not in certificate)**.** *The fallback optimizes the true 2^n program over $\mathcal{D}_{\mathrm{LCN}}$, but by a local method, so its output carries no global guarantee and can be over-tight. Conversely, even at n small enough for the joint to be written down, the cluster program is often the easier one to certify: on the worked example of Section C.1 it certifies all endpoints where the 2^n formulation certifies only seven of ten.*

A failed optimization is reported as the corresponding vacuous endpoint (0 for a minimum, 1 for a maximum): uninformative, never unsound. If every marginal returns vacuous, the instance or the evidence is likely inconsistent.

B.4 Complexity

Variables. One per cluster state, so by equation 12

$$\sum_{C\in\mathcal{C}} 2^{|A(C)|} \;=\; O\big(n\,2^{w+1}\big), \tag{13}$$

since bucket elimination creates at most one cluster per node of the component DAG, $|\mathcal{C}| \le m \le n$; under evidence add the single scale r. Against 2^n for the direct formulation, this is the algorithm's reason for being: the cost is exponential in the treewidth, not in the number of atoms.

Constraints. There are $|\mathcal{C}|$ normalization equalities; $\sum_{(C,D),\,S_{CD}\neq\varnothing} 2^{|S_{CD}|} = O(n2^w)$ separator equalities, since a tree has $|\mathcal{C}|-1$ edges and a separator, being a proper subset of a cluster, has at most w atoms; two rows per hosted sentence; and, per hosted assertion $(X \perp\!\!\!\perp Y \mid S)$,

$$2^{|Y|}\cdot\max\big(1, 2^{|S|}\big) \quad \text{equalities.} \tag{14}$$

Remark 5 (Which rows are nonconvex, and why the width bound survives)**.** *(a) Only the assertion equalities equation 20 are nonconvex; the conditional objective is linear after the scaling equation 22. The cleared Type-2 rows equation 19 are* linear *in q, since ℓ and u are constants. Consequently an instance with no evidence and no hosted assertion yields a* linear *program, even when it is loopy; the example of Section C.1 has two bilinear rows, from its one hosted assertion. (b) The count equation 14 appears unbounded in $|Y|$, but a* hosted *assertion satisfies $X\cup Y\cup S\subseteq A(C)$, so it contributes at most 2^{w+1} rows and equation 13 is unaffected. An assertion with a wide Y is reduced, by Lemma 4, to the part of Y connected to X; what remains is hosted, possibly after widening, which increases w itself, or relaxed to hostable consequences.*

Per atom the cost is two optimizations over the shared system, hence $2|\mathbf{X}|$ in total.

C The nonlinear program

Fix a cluster C and index the $2^{|A(C)|}$ assignments of $A(C)$ by i, writing ω_i^C for the i-th. For a formula φ with $\mathrm{atoms}(\varphi)\subseteq A(C)$ let $A_\varphi^{(C)} \in \{0,1\}^{2^{|A(C)|}}$ be its indicator vector, $A_\varphi^{(C)}[i] = \mathbf{1}[\omega_i^C \models \varphi]$, and abbreviate $\langle v, q_C\rangle = \sum_i v_i q_{C,i}$; thus $\langle A_\varphi^{(C)}, q_C\rangle = P(\varphi)$ whenever q_C is the marginal of a joint on $A(C)$.

Variables and normalization. One distribution per cluster,

$$q_C \in [0,1]^{2^{|A(C)|}}, \qquad \sum_{i=0}^{2^{|A(C)|}-1} q_{C,i} = 1 \qquad \forall C \in \mathcal{C}. \tag{15}$$

There are no separator variables, no per-family conditional variables and no global joint variable; the tree structure enters only through the equalities below.

Marginalization. Let $\pi_{C\to S}$ map a state of C to its restriction to $S \subseteq A(C)$, and define the $0/1$ aggregation matrix

$$M^{C\to S}_{r,j} = \mathbf{1}\big[\pi_{C\to S}(j) = r\big], \qquad \text{so} \qquad \big(M^{C\to S} q_C\big)[r] = (q_C \restriction S)(r). \tag{16}$$

Separator consistency. For every edge with a non-empty separator, the two endpoints must agree on it:

$$\sum_{j:\, \pi_{C\to S}(j)=r} q_{C,j} \;=\; \sum_{k:\, \pi_{D\to S}(k)=r} q_{D,k} \qquad \forall (C,D) \in E,\ \forall r \in \{0,\dots,2^{|S_{CD}|}-1\}, \tag{17}$$

equivalently $q_C \restriction S_{CD} = q_D \restriction S_{CD}$: linear equalities, $2^{|S_{CD}|}$ per edge. No messages are computed; the consistency conditions are imposed simultaneously with all others, which is what distinguishes this formulation from message passing.

Sentences. A Type-1 sentence hosted on $C_h \supseteq \mathrm{atoms}(\varphi)$ contributes

$$\ell \;\le\; \big\langle A^{(C_h)}_{\varphi},\, q_{C_h} \big\rangle \;\le\; u, \tag{18}$$

and a Type-2 sentence contributes the cleared pair

$$\ell \,\big\langle A^{(C_h)}_{\psi}, q_{C_h}\big\rangle \;\le\; \big\langle A^{(C_h)}_{\varphi\wedge\psi}, q_{C_h}\big\rangle \;\le\; u\,\big\langle A^{(C_h)}_{\psi}, q_{C_h}\big\rangle. \tag{19}$$

No division appears, and consistently with Remark 1 the rows equation 19 are vacuously satisfied when $P(\psi) = 0$: a Type-2 sentence constrains nothing on a null conditioning event.

Independence assertions. For a hosted $(X \perp\!\!\!\perp Y \mid S)$ with $X = \{x\}$, one equality per configuration $(\mathbf{y}, \mathbf{s})$. With $S \neq \varnothing$, writing the four indicators for $\{x{=}1, Y{=}\mathbf{y}, S{=}\mathbf{s}\}$, $\{S{=}\mathbf{s}\}$, $\{x{=}1, S{=}\mathbf{s}\}$, $\{Y{=}\mathbf{y}, S{=}\mathbf{s}\}$ as A_a, A_b, A_c, A_d:

$$\langle A_a, q_{C_h}\rangle\,\langle A_b, q_{C_h}\rangle - \langle A_c, q_{C_h}\rangle\,\langle A_d, q_{C_h}\rangle = 0, \tag{20}$$

which is equation 8 verbatim; and with $S = \varnothing$ the marginal form $\langle A_a, q\rangle - \langle A_b, q\rangle\langle A_c, q\rangle = 0$ for $P(x,\mathbf{y}) = P(x)P(\mathbf{y})$. These rows are bilinear, hence nonconvex, and are the reason a global solver is required.

Two features of this encoding matter. First, only the $x{=}1$ slice is needed: with $(\mathbf{y}, \mathbf{s})$ fixed a binary X has one degree of freedom, so the $x{=}0$ equality follows. Second, the configurations $\mathbf{y}$ must range over *whole* assignments of Y. Replacing $(X \perp\!\!\!\perp Y \mid S)$ by the pairwise family $\{(X \perp\!\!\!\perp t \mid S)\}_{t\in Y}$ is strictly weaker for $|Y| \ge 2$ — pairwise independence does not imply joint independence — and under-constrains the program, yielding spuriously loose bounds.

Objective. Let C_a be a cluster containing the query atom a and all evidence atoms (hypothesis H4). Without evidence the read-off is linear,

$$\rho_a(q) = \big\langle A^{(C_a)}_{a=1},\, q_{C_a}\big\rangle = P(a{=}1). \tag{21}$$

With evidence e the read-off $P(a \mid e) = \langle A_{a\wedge e}, q\rangle / \langle A_e, q\rangle$ is linear-fractional, and is made linear by the Charnes–Cooper transformation (Charnes & Cooper, 1962): substitute $y_C = r\, q_C$ for every cluster, with a scale $r \ge 0$, and require

$$\textstyle\sum_i y_{C,i} = r \quad \forall C, \qquad \big\langle A^{(C_a)}_{e}, y_{C_a}\big\rangle = 1, \qquad \rho_a(y) = \big\langle A^{(C_a)}_{a=1}\cdot A^{(C_a)}_{e},\, y_{C_a}\big\rangle. \tag{22}$$

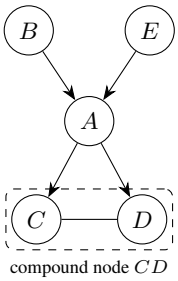
Each row is made homogeneous by multiplying its lower-degree terms by powers of r. The separator equalities, the cleared Type-2 rows and the conditional assertion rows equation 20 are already homogeneous and are imposed on y unchanged; the Type-1 rows become $\ell\, r \le \langle A_\varphi, y\rangle \le u\, r$, and the marginal assertion rows $\langle A_a, q\rangle = \langle A_b, q\rangle\langle A_c, q\rangle$ become $r\,\langle A_a, y\rangle = \langle A_b, y\rangle\langle A_c, y\rangle$. A feasible y corresponds to the feasible $q = y/r$ with $P_q(e) = 1/r > 0$, and $\rho_a(y) = P_q(a \mid e)$; conversely every feasible q with $P_q(e) > 0$ yields such a y. The program is therefore exact for the conditional query without any denominator floor, and it is infeasible exactly when $P(e) = 0$ for every feasible q, in which case the conditional bound is undefined and is reported as vacuous.

(a) LCN

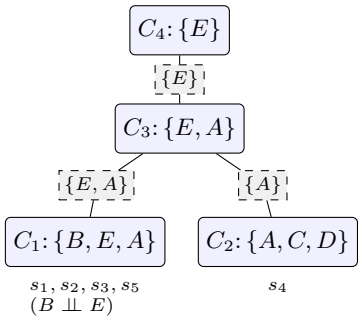
s_1: $0.1 \le P(B) \le 0.2$
s_2: $0.05 \le P(E) \le 0.1$
s_3: $0.6 \le P(A \mid B \vee E) \le 0.9$
s_4: $0.5 \le P(\neg(C \oplus D) \mid A) \le 0.7$
s_5: $0.05 \le P(A) \le 0.1$

LMC: $(B \perp\!\!\!\perp E)$,
$(C \perp\!\!\!\perp B, E \mid A, D)$,
$(D \perp\!\!\!\perp B, E \mid A, C)$

(b) chain graph



(c) junction tree



(d) nonlinear program for $\underline{P}(A)$ / $\overline{P}(A)$, with $q_1 \in \Delta(8)$ over (B, E, A), $q_2 \in \Delta(8)$ over (A, C, D), $q_3 \in \Delta(4)$ over (E, A), $q_4 \in \Delta(2)$ over (E):

$$\min / \max \; P_{q_1}(A)$$

$$\begin{aligned}
\text{s.t. } & q_k \in \Delta(2^{|C_k|}),\ k = 1, \dots, 4 && \text{(4 simplex)} \\
& q_1 \restriction \{E, A\} = q_3, \quad q_2 \restriction \{A\} = q_3 \restriction \{A\}, \quad q_3 \restriction \{E\} = q_4 && \text{(4+2+2 separator)} \\
& 0.1 \le P_{q_1}(B) \le 0.2,\ \ 0.05 \le P_{q_1}(E) \le 0.1,\ \ 0.05 \le P_{q_1}(A) \le 0.1 && \text{(6 sentence, linear)} \\
& 0.6\, P_{q_1}(B \vee E) \le P_{q_1}(A \wedge (B \vee E)) \le 0.9\, P_{q_1}(B \vee E) && \text{(2 sentence, linear)} \\
& 0.5\, P_{q_2}(A) \le P_{q_2}(A \wedge \neg(C \oplus D)) \le 0.7\, P_{q_2}(A) && \text{(2 sentence, linear)} \\
& P_{q_1}(B{=}1, E{=}y) = P_{q_1}(B{=}1)\, P_{q_1}(E{=}y), \quad y \in \{0, 1\} && \text{(2 assertion, bilinear)}
\end{aligned}$$

Figure 4: The worked example. (a) The LCN of Figure 1 and the assertions of its LMC. (b) Its structure: C and D form an undirected component, i.e. a compound node. (c) The junction tree, with separators dashed on the edges and the constraints hosted by each cluster below it. (d) The resulting program: 22 variables and 24 constraints, of which only the two rows of $(B \perp\!\!\!\perp E)$ are nonlinear; $P_{q_k}(\varphi)$ abbreviates $\langle A_\varphi^{(C_k)}, q_k \rangle$.

The program. Collecting equation 15–equation 21, for a marginal query,

$$\begin{aligned}
\underline{P}(a),\ \overline{P}(a) \ = \ \min / \max \quad & \rho_a(q) && \\
\text{s.t.} \quad & \textstyle\sum_i q_{C,i} = 1, \quad q_{C,i} \in [0, 1] && \forall C \in \mathcal{C} \\
& q_C \restriction S_{CD} = q_D \restriction S_{CD} && \forall (C, D) \in E,\ S_{CD} \neq \varnothing \\
& \text{equation 18, equation 19} && \forall \text{ hosted sentence} \\
& \text{equation 20} && \forall \text{ hosted assertion.}
\end{aligned} \tag{23}$$

For a conditional query the same constraints are imposed on y as described above, together with equation 22.

C.1 A WORKED EXAMPLE

We work the LCN of Figure 1 completely. It is small enough that every variable and constraint can be written down, yet it exercises every part of the construction: its structure is loopy after moralization, it has a compound node, two of its three LMC assertions are separator-entailed, and the third must be imposed as bilinear rows. Figure 4 summarizes the instance, its structure, the junction tree and the program.

C.1.1 THE INSTANCE

The sentences are shown in Figure 4(a). The structure (Figure 4(b)) has the directed edges $B \to A$, $E \to A$, $A \to C$, $A \to D$ and the undirected edge $C - D$, so it is a chain graph with components $\{B\}$, $\{E\}$, $\{A\}$ and $\{C, D\}$. The component DAG has the families

$$B \mid \varnothing, \qquad E \mid \varnothing, \qquad A \mid B, E, \qquad CD \mid A,$$

and the LMC yields the three assertions listed in the figure. Moralization connects the co-parents B and E of A, so the moral graph contains the triangle B, E, A and is not singly connected.

C.1.2 THE JUNCTION TREE

Bucket elimination along a min-fill order produces the four clusters

$$C_1 = \{B, E, A\}, \qquad C_2 = \{A, C, D\}, \qquad C_3 = \{E, A\}, \qquad C_4 = \{E\},$$

with tree edges $C_1 - C_3$, $C_2 - C_3$ and $C_3 - C_4$ and separators $\{E, A\}$, $\{A\}$ and $\{E\}$ (Figure 4(c)). The largest cluster has three atoms, so $w = 2$. The compound node CD contributes both of its atoms to C_2 at once, which is why that cluster has three atoms although it comes from a family with only two nodes. The tree has the running-intersection property by construction (H1). Every sentence fits in a cluster (H2): s_1, s_2, s_3 and s_5 mention only B, E and A and are hosted on C_1, and s_4 is hosted on C_2.

The chordal graph G^* joins two atoms when they share a cluster; its edges are BE, BA, EA, AC, AD and CD. The ordered construction of Proposition 4 does not apply here, because the assertions on C and on D each condition on the other member of the compound node; the min-fill triage of Section B.2 is used instead.

C.1.3 DISCHARGING THE ASSERTIONS

$(C \perp\!\!\!\perp B, E \mid A, D)$ and $(D \perp\!\!\!\perp B, E \mid A, C)$: separator-entailed. Every path in G^* from C or D to B or E passes through A, so deleting the conditioning set $\{A, D\}$ (resp. $\{A, C\}$) leaves C (resp. D) connected only to the other member of $\{C, D\}$. The reduction of Lemma 4 is therefore empty, and no row is imposed: every joint reconstructed from consistent cluster marginals satisfies these assertions.

$(B \perp\!\!\!\perp E)$: hosted. B and E are adjacent in G^*, so the empty set does not separate them and the reduction keeps $\mathbf{Y}_{\text{in}} = \{E\}$. Both atoms lie in C_1, so the assertion is imposed there in the marginal form of equation 20, one row per state of E (only the $B{=}1$ slice is needed):

$$P_{q_1}(B{=}1, E{=}y) = P_{q_1}(B{=}1)\, P_{q_1}(E{=}y), \qquad y \in \{0, 1\}.$$

These two rows are the only nonlinear constraints of the program. Without them the program could correlate B and E, which the LCN forbids, and the bounds on A would widen. With them H3 holds: the imposed rows are those of an LMC assertion, and by Lemma 4 the hosted assertion together with the two separator-entailments gives every assertion of the LMC on every reconstructed joint.

C.1.4 THE PROGRAM

The program is shown in Figure 4(d). Its $8 + 8 + 4 + 2 = 22$ variables are the cells of $q_1, \ldots, q_4$. The constraints are 4 simplex equalities; 8 separator equalities, 4 for $\{E, A\}$, 2 for $\{A\}$ and 2 for $\{E\}$, for example

$$q_1[B{=}0, E{=}e, A{=}a] + q_1[B{=}1, E{=}e, A{=}a] = q_3[E{=}e, A{=}a], \qquad e, a \in \{0, 1\};$$

10 sentence rows, all linear since the cleared Type-2 rows equation 19 are linear in q; and the 2 bilinear assertion rows: 24 constraints in all. The direct formulation over the joint has $2^5 = 32$ variables; the saving grows exponentially with n at fixed width (Theorem 2).

C.1.5 RESULTS

Solving the program for each atom with SCIP certifies all ten endpoints, in 3.5 s in total:

engine	$P(A)$	$P(B)$	$P(C)$	$P(D)$	$P(E)$
CJT (SCIP)	[0.0870, 0.1000]	[0.1000, 0.1228]	[0, 1]	[0, 1]	[0.0500, 0.0741]
direct 2^n (SCIP)	[–, 0.1000]	[0.1000, –]	[0, 1]	[0, 1]	[0.0500, –]

The direct global formulation over the 2^5 joint certifies only 7 of its 10 endpoints within a time limit of 120 s per solve (381 s in total), and those 7 agree with CJT; a dash marks an endpoint it did not certify. The three remaining CJT values can be checked by hand. By s_3 and the independence of B and E, $P(A) \geq 0.6\, P(B \vee E) = 0.6\,(b + e - be)$, minimized at $b = 0.1$, $e = 0.05$, which gives $\underline{P}(A) = 0.6 \cdot 0.145 = 0.087$. Conversely s_5 forces $0.6\, P(B \vee E) \leq P(A) \leq 0.1$, i.e.

$b + e - be \leq 1/6$; with $e = 0.05$ this gives $b \leq 0.1228 = \overline{P}(B)$, and with $b = 0.1$ it gives $e \leq 0.0741 = \overline{P}(E)$. The atoms C and D are vacuous because s_4 constrains only whether C and D agree.

Table 2 lists the separator marginals at the minimizer of $P(A)$. Both clusters incident to an edge realize the same vector, as the separator equalities require; and the marginal of q_1 on $\{B, E\}$ is the product of its marginals on B and on E, $P(B{=}1) = 0.1$ and $P(E{=}1) = 0.05$, as the hosted assertion requires. Lemma 3 reassembles these cluster marginals into a joint distribution in $\mathcal{D}_{\mathrm{LCN}}$ attaining $\underline{P}(A)$.

Table 2: Separator marginals at the minimizer of $P(A)$, by assignment of the separator (bits in the listed order). Both clusters incident to each edge realize the same vector.

edge	separator	marginal			
$C_1 - C_3$	$\{E, A\}$	00: 0.8898	01: 0.0602	10: 0.0232	11: 0.0268
$C_2 - C_3$	$\{A\}$	0: 0.9130	1: 0.0870		
$C_3 - C_4$	$\{E\}$	0: 0.9500	1: 0.0500		

C.1.6 A SECOND STRUCTURE

Table 3 compares the program size with that of the five-atom `asia` example, a DAG on which the ordered construction of Proposition 4 applies: there two ordered rows, $(X \perp\!\!\!\perp D \mid C)$ and $(C \perp\!\!\!\perp B \mid S)$, imply all four LMC assertions.

Table 3: Program size on the worked example and on `asia`.

instance	tree	clus.	$w{+}1$	edges	vars	constr.	simplex	sep.	sent.	assert.
alarm (Fig. 4)	min-fill	4	3	3	22	24	4	8	10	2
`asia`	ordered	5	3	4	30	37	5	14	10	8

C.2 EXACTNESS

This section states and proves the guarantees. The development is closed: it uses only Lemma 1 from Section A and Lemma 3 below, and in particular needs no notion of realizability, no property of the strong extension, and no result about any other engine.

Definition 7 (Cluster-feasible set, hosting, separator-entailment, read-off)**.** *Let T be the cluster tree built by bucket elimination (Section B.1) over clusters $\mathcal{C}$ with atom sets $A(C)$, and let G^* be the graph on the atoms in which two atoms are adjacent iff they co-occur in a cluster; it is the triangulated moral graph of the chain graph, augmented by the cross-family sentence scopes. Every cluster is a clique of G^* and every maximal clique of G^* is a cluster; we identify a cluster with its atom set.*

- *The* cluster-feasible set *is*

$$\mathcal{G} = \big\{(q_C)_{C\in\mathcal{C}} \;:\; q_C \in \Delta(2^{|C|}) \text{ for each } C;\; q_C \restriction S = q_D \restriction S \text{ for every edge } (C, D) \in T,\; S = C \cap D;\; \text{every hosted constraint holds on its host cluster}\big\}.$$

- *A constraint c is* hosted *if some cluster $C_h \supseteq \mathrm{scope}(c)$ exists; its row is then imposed on q_{C_h}'s marginal.*
- *An LMC assertion $(X \perp\!\!\!\perp Y \mid S)$ is* separator-entailed *if S separates X from Y in G^* (every path from X to Y meets S).*
- *The* imposed independence rows Λ_T *are the independence statements actually hosted on T: reduced LMC assertions (Lemma 4) on a min-fill tree, or ordered rows (Proposition 4) on an ordered tree. The LCN's own LMC set is denoted Λ.*

- *For a query atom a and evidence e lying together in a cluster C_a, the* read-off *is $\rho_a(q) = \sum_{i:a=1} q_{C_a}[i]$ (no evidence), or $\rho_a(q) = \big(\sum_{i:a=1,e} q_{C_a}[i]\big)/\big(\sum_{i:e} q_{C_a}[i]\big)$ (with evidence).*
- *$\pi(p) = (p \restriction C)_{C\in\mathcal{C}}$ is the* projection *of a joint onto the clusters.*

The reconstruction direction rests on the following lemma. We state and prove it for distributions with zeros, where the joint is not unique and the familiar quotient formula involves $0/0$.

Lemma 3 (RIP reconstruction, with zeros)**.** *Let T be a clique tree with the running-intersection property over clusters $\mathcal{C}$, and let $(q_C) \in \prod_C \Delta(2^{|C|})$ be separator-consistent: $q_C \restriction S = q_D \restriction S$ for every edge (C, D) with $S = C \cap D$. Then there exists a joint distribution $p \in \Delta(2^{\mathbf{X}})$ with $p \restriction C = q_C$ for every $C \in \mathcal{C}$, and p factorizes over the cliques of G^*. Such p is unique if every separator marginal is strictly positive, but* not *in general.*

Proof. Induction on $|\mathcal{C}|$. If $|\mathcal{C}| = 1$ take $p = q_C$.

Let $|\mathcal{C}| \geq 2$ and pick a leaf cluster C of T with neighbour D and separator $S = C \cap D$. Let $T' = T - C$. Removing a leaf preserves the running-intersection property, the remaining clusters are still separator-consistent, and by running intersection $C \setminus S$ meets no cluster of T'; so the inductive hypothesis gives a joint p' over $\mathbf{X}' = \bigcup_{C'\in T'} C'$ with $p' \restriction C' = q_{C'}$ for all $C' \in T'$. Note $p' \restriction S = (p' \restriction D) \restriction S = q_D \restriction S = q_C \restriction S =: q_S$, using separator consistency.

Define a conditional kernel on the fresh atoms $C \setminus S$: for each assignment s of S,

$$\kappa(\cdot \mid s) = \begin{cases} q_C(\cdot, s)/q_S(s), & q_S(s) > 0, \\ \delta_{c_0}, & q_S(s) = 0, \end{cases}$$

where c_0 is any fixed assignment of $C \setminus S$. This is well defined: when $q_S(s) > 0$ the right-hand side is non-negative and sums over $C \setminus S$ to $q_S(s)/q_S(s) = 1$, so it is a distribution. Now set $p(x) = p'(x_{\mathbf{X}'}) \cdot \kappa(x_{C\setminus S} \mid x_S)$.

p is a distribution: it is non-negative, and summing over $x_{C\setminus S}$ first gives p', which sums to 1.

The arbitrary choice is irrelevant. On $\{x : q_S(x_S) = 0\}$ we have $p'(x_{\mathbf{X}'}) = 0$, because $p' \restriction S = q_S$ assigns zero mass to that event and $p' \geq 0$. Hence p vanishes there regardless of κ, so p does not depend on c_0. (This is also the precise sense in which the familiar quotient formula $p = \prod_C q_C / \prod_{(C,D)} q_S$ is legitimate: a zero separator cell forces every contributing cluster cell to zero, since $q_S = q_C \restriction S$ is a sum of non-negative q_C entries, so numerator and denominator vanish together and the term is assigned 0.)

Cluster marginals. For $C' \in T'$: summing p over $x_{C\setminus S}$ gives p', so $p \restriction C' = p' \restriction C' = q_{C'}$. For C itself: $p \restriction C(c, s) = \big(\sum \text{ of } p \text{ over } \mathbf{X}' \setminus S \text{ at fixed } x_S = s\big) \cdot \kappa(c \mid s) = p' \restriction S(s)\, \kappa(c \mid s) = q_S(s)\kappa(c \mid s)$, which equals $q_C(c, s)$ when $q_S(s) > 0$ and equals $0 = q_C(c, s)$ when $q_S(s) = 0$. So $p \restriction C = q_C$.

Factorization. Unwinding the induction, p is a product of one non-negative factor per cluster (namely q_C or the kernel κ derived from it, each a function of the atoms of that cluster only), hence factorizes over the cliques of G^*.

Non-uniqueness. If some separator cell $q_S(s) = 0$, then $\kappa(\cdot \mid s)$ was chosen arbitrarily; any other choice yields a joint with the same cluster marginals, since all such joints vanish on that event. Hence p is not unique. When all separator marginals are positive the kernels are forced and the induction determines p uniquely. □

The two triages of Section B.2 discharge H3 through the following two results.

Lemma 4 (Exact reduction of an assertion)**.** *Let p factorize over the cliques of G^*, let $(X \perp\!\!\!\perp Y \mid Z)$ be an assertion, let K be the set of atoms reachable from X in G^* after deleting Z, and let $Y_{\text{in}} = Y \cap K$, $Y_{\text{out}} = Y \setminus K$. Then $X \perp\!\!\!\perp Y \mid Z$ holds under p if and only if $X \perp\!\!\!\perp Y_{\text{in}} \mid Z$ does. In particular the assertion is separator-entailed, and holds automatically, when $Y_{\text{in}} = \varnothing$.*

Proof. ($\Rightarrow$) is decomposition. ($\Leftarrow$) By construction of K, every path from K to Y_{out} meets Z, so Z separates $K \supseteq X \cup Y_{\text{in}}$ from Y_{out} in G^*. Factorization over an undirected graph implies its global Markov property without any positivity assumption, so $(X \cup Y_{\text{in}}) \perp\!\!\!\perp Y_{\text{out}} \mid Z$, and by

weak union $X \perp\!\!\!\perp Y_{\text{out}} \mid Z \cup Y_{\text{in}}$. Contraction with $X \perp\!\!\!\perp Y_{\text{in}} \mid Z$ yields $X \perp\!\!\!\perp Y_{\text{in}} \cup Y_{\text{out}} \mid Z$. Decomposition, weak union and contraction hold for every probability distribution (they are the semi-graphoid axioms). □

Proposition 4 (Ordered tree: H3 by construction)**.** *Let D be the component DAG of the chain-graph factorization equation 10, let T be built by bucket elimination along a reverse topological order of D, and for every node x let $\{x\} \cup N_x$ be its elimination clique. Then N_x consists of topological predecessors of x and contains* $\mathrm{pa}(x)$*. If p factorizes over G^* and satisfies, for every node x,*

$$x \perp\!\!\!\perp N_x \setminus \mathrm{pa}(x) \mid \mathrm{pa}(x), \tag{24}$$

then $p = \prod_x p(x \mid \mathrm{pa}(x))$, so p satisfies every d-separation of D. Consequently, if every LMC assertion of the LCN is a d-separation of D between unions of nodes (in particular, no assertion splits a compound node), p satisfies every LMC assertion. Conversely each row equation 24 follows from the local Markov property of D, since $N_x \setminus \mathrm{pa}(x)$ consists of non-descendants of x.

Proof. Work at the level of nodes of D: let G_D^* be the graph on the nodes in which two nodes are adjacent iff their atoms co-occur in a cluster. A G^*-factorizing p is also G_D^*-factorizing, because every clique of G^* lies inside a cluster and every cluster is the union of the atoms of a clique of G_D^*. Bucket elimination along $x_n, \ldots, x_1$ (reverse topological) removes x only after all its topological successors, so the neighbours of x at that moment — N_x — are predecessors; $\mathrm{pa}(x) \subseteq N_x$ because the family scope $\{x\} \cup \mathrm{pa}(x)$ is one of the covered scopes and none of $\mathrm{pa}(x)$ has been eliminated yet. Since G_D^* is the graph of the elimination cliques, the order is a perfect elimination order of it: when x is eliminated it is simplicial in the graph induced on the remaining nodes, with neighbourhood N_x. (Pruning a subsumed leaf does not change G_D^*, since the leaf's clique lies inside its parent's.) Summing a G_D^*-factorizing p over a simplicial vertex leaves a distribution that factorizes over the induced subgraph (all factors containing x lie in its clique), so by induction the marginal of p on the nodes remaining at that moment factorizes over their induced subgraph; its global Markov property gives that x is independent of the other remaining nodes given N_x. The chain rule along the order then writes $p = \prod_x p(x \mid N_x)$, each conditional defined arbitrarily on $\{p(N_x) = 0\}$, where it is multiplied by zero. By equation 24, $p(x \mid N_x) = p(x \mid \mathrm{pa}(x))$ wherever $p(N_x) > 0$, hence $p = \prod_x p(x \mid \mathrm{pa}(x))$. A distribution that factorizes along a DAG satisfies its global (d-separation) Markov property, again without positivity. The converse is the local Markov property of D followed by decomposition. □

The implementation checks, per instance, the two conditions that make Proposition 4 applicable: that each row equation 24 is implied by a single LCN assertion (by symmetry and decomposition), and that every LCN assertion is a d-separation of D between unions of nodes. If either fails the ordered tree is discarded and the min-fill triage is used. They fail when an assertion splits a compound node, as in the worked example of Section C.1 and in the `smokers` example; for the latter the min-fill triage widens to a single 9-atom cluster, which is exact but no cheaper than the direct program.

Theorem 1 of the main text is the following theorem, with the same hypotheses H1–H4; Theorem 2 is the count of Section B.4.

Theorem 8 (CJT exactness)**.** *Suppose* (H1) *T has the running-intersection property (true by construction);* (H2) *every LCN sentence is hosted;* (H3) *every imposed row in Λ_T is implied by Λ, and every assertion of Λ holds for every joint that factorizes over G^* and satisfies Λ_T (this is H3 of Theorem 1, the rows of Λ_T being part of the program);* (H4) *the query atom and all evidence atoms lie together in some cluster. Then for every such atom a,*

$$\min_{q \in \mathcal{G}} \rho_a(q) = \min_{p \in \mathcal{D}_{\text{LCN}}} \rho_a(\pi(p)) = \underline{P}(a), \qquad \max_{q \in \mathcal{G}} \rho_a(q) = \max_{p \in \mathcal{D}_{\text{LCN}}} \rho_a(\pi(p)) = \overline{P}(a),$$

For a conditional query $P(a \mid e)$, solved in the scaled form equation 22, the same equalities hold for $\underline{P}(a \mid e)$ and $\overline{P}(a \mid e)$ taken over $\mathcal{D}_{\text{LCN}}^+ = \{p \in \mathcal{D}_{\text{LCN}} : P_p(e) > 0\}$, the standard definition of conditional lower and upper probabilities. Moreover the projection *half holds without* (H2) *and without the second half of* (H3)*, so* CJT *is a valid* outer *bound whenever every imposed row is implied by Λ — which the relaxation step of Section B.2 preserves.*

Proof. First note the read-off identity: for any joint p, $\rho_a(\pi(p)) = P_p(a)$ (resp. $P_p(a \mid e)$), because ρ_a sums the cells of $\pi(p)_{C_a} = p \restriction C_a$ in which a (and e) hold, which by definition of a marginal equals $P_p(a)$ (resp. the ratio). So both sides optimize the *same* functional.

Projection $\mathcal{D}_{\mathrm{LCN}} \to \mathcal{G}$ (needs H1, H4 and the first half of H3). Let $p \in \mathcal{D}_{\mathrm{LCN}}$ and set $q_C := p \restriction C$, i.e. $q = \pi(p)$. Each q_C is a distribution (a marginal of one). Each separator equality holds because both $q_C \restriction S$ and $q_D \restriction S$ equal $p \restriction S$. Each hosted constraint c holds: by Lemma 1 its value depends on p only through $p \restriction \mathrm{scope}(c)$, and since $\mathrm{scope}(c) \subseteq C_h$ we have $q_{C_h} \restriction \mathrm{scope}(c) = p \restriction \mathrm{scope}(c)$, so c holds on the host exactly because it holds for p. For a sentence this is immediate; for an imposed row of Λ_T it holds for p because, by the first half of H3, it is implied by an assertion of Λ, which $p \in \mathcal{D}_{\mathrm{LCN}}$ satisfies. Hence $q \in \mathcal{G}$, with $\rho_a(q) = P_p(a)$. Therefore every value achievable over $\mathcal{D}_{\mathrm{LCN}}$ is achievable over $\mathcal{G}$, giving $\min_{\mathcal{G}} \rho_a \le \min_{\mathcal{D}_{\mathrm{LCN}}}$ and $\max_{\mathcal{G}} \rho_a \ge \max_{\mathcal{D}_{\mathrm{LCN}}}$: the cluster interval *contains* the true interval. This is the outer-bound claim; it used neither H2 nor the second half of H3.

Reconstruction $\mathcal{G} \to \mathcal{D}_{\mathrm{LCN}}$ (needs H2, H3). Let $q \in \mathcal{G}$. By H1 and Lemma 3 there is a joint p with $p \restriction C = q_C$ for every cluster, factorizing over the cliques of G^*. We check $p \in \mathcal{D}_{\mathrm{LCN}}$.

Sub-scope consistency: if $A \subseteq C$ for some cluster C, then $p \restriction A = (p \restriction C) \restriction A = q_C \restriction A$, since marginalizing in stages agrees with marginalizing at once.

Hosted constraints (H2 and the hosted part of H3): let c be hosted on C_h. By the previous paragraph $p \restriction \mathrm{scope}(c) = q_{C_h} \restriction \mathrm{scope}(c)$, and q satisfies c on C_h by definition of $\mathcal{G}$; by Lemma 1 c therefore holds for p.

LMC assertions (the second half of H3): p factorizes over the cliques of G^* and, by the previous paragraph, satisfies every imposed row of Λ_T; by H3 it therefore satisfies every assertion of Λ, i.e. equation 8 holds for each. On a min-fill tree this is Lemma 4 (each assertion is equivalent to its imposed reduction, or separator-entailed); on an ordered tree it is Proposition 4. Neither argument uses positivity. Note that no assertion of Λ need be imposed verbatim.

Hence $p \in \mathcal{D}_{\mathrm{LCN}}$, and $\rho_a(q) = \rho_a(\pi(p)) = P_p(a)$ because $\pi(p) = q$ on the host cluster. So every value achievable over $\mathcal{G}$ is achievable over $\mathcal{D}_{\mathrm{LCN}}$, giving the reverse inequalities. Combined with the projection half, the optima coincide.

Conditional queries. By H4 the read-off is a functional of the single cluster C_a, and both the projection $p \mapsto \pi(p)$ and the reconstruction of Lemma 3 preserve q_{C_a}, hence $P(e)$. So projection maps $\mathcal{D}^+_{\mathrm{LCN}}$ into $\mathcal{G}^+ = \{q \in \mathcal{G} : P_q(e) > 0\}$, reconstruction maps $\mathcal{G}^+$ into $\mathcal{D}^+_{\mathrm{LCN}}$, both preserve $P(a \mid e)$, and the argument above gives equality of the optima over these two sets. The scaling equation 22 maps $\mathcal{G}^+$ onto the feasible set of the scaled program, $q \mapsto (y, r) = (q/P_q(e), 1/P_q(e))$, with inverse $q = y/r$, under which $P_q(a \mid e)$ equals the linear read-off of y. Every constraint of $\mathcal{G}$ is preserved: each scaled row is the original row multiplied by a power of r (by r for the linear rows, by r^2 for the assertion rows), and $r > 0$. □

Remark 6 (Reading the proof)**.** *(a) The argument never mentions chains: exactness is a property of the* junction tree*, via RIP, hosting, separator-entailment and locality. It is in particular* not *a statement about singly-connected topologies — the worked example of Section C.1 is loopy and exact. (b) The projection half needs only that every imposed row follows from the LMC, which the triage guarantees by construction; so for marginal queries* CJT *is never unsound — its interval always contains the truth — and H2 and the second half of H3 decide only tightness; the same holds for conditional queries, over $\mathcal{D}^+_{\mathrm{LCN}}$. (c) H3 is* sufficient, not necessary*, and it can be checked mechanically: the implementation records for each instance whether H3 holds, how many assertions were separator-entailed, reduced and hosted, implied by ordered rows, or relaxed.*

Remark 7 (Why the conditional query is scaled)**.** *A natural alternative linearizes $P(a \mid e)$ by an auxiliary t with $t\,P(e) = P(a \wedge e)$ and a floor $P(e) \ge \varepsilon$ to exclude the degenerate $P(e) = 0$. The floor removes the members of $\mathcal{D}_{\mathrm{LCN}}$ with $P(e) < \varepsilon$, and in floating point it fails in a worse way: once ε is near the solver's feasibility tolerance, $t\,P(e) = P(a \wedge e)$ holds within tolerance for every t. On the instance $0 \le P(E) \le 0.3$, $0.2 \le P(A \mid E) \le 0.4$, $0.6 \le P(A \mid \neg E) \le 0.8$ with $\varepsilon = 10^{-6}$, SCIP certifies $P(A \mid E) \in [0, 1]$, whereas the exact bound is $[0.2, 0.4]$. The scaled form equation 22 has neither defect and returns $[0.2, 0.4]$.*

D FAMILY-REALIZABLE SENTENCES

Which sentences can a *single* family's local set enforce? A family's degrees of freedom are the conditional probabilities $P(\mathrm{ch}_v{=}c \mid \mathrm{pa}_v{=}\pi)$; the parents' marginal $P(\mathrm{pa}_v)$ is not one of them — it is determined elsewhere in the network, and is a free variable during the local solve. A sentence that

constrains a quantity depending on $P(\mathrm{pa}_v)$ therefore cannot be enforced locally, and the free product equation 11 may violate it. This is the mechanism behind Gap B, and the following definition isolates it.

Definition 8 (Family-realizable sentence)**.** *Let $\mathcal{L}$ be a chain-graph LCN with families $\{(\mathrm{ch}_v, \mathrm{pa}_v)\}$, and let s be a sentence with functional g_s. Say s is* family-realizable in family v *if* $\mathrm{atoms}(s) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$ *and the value $g_s(p)$ is determined by the conditional table $P(\mathrm{ch}_v \mid \mathrm{pa}_v)$* alone *— i.e. for any two joints p, p' that induce the same conditional table at v, $g_s(p) = g_s(p')$, whatever the parents' marginals $P(\mathrm{pa}_v)$ and $P'(\mathrm{pa}_v)$ may be. Concretely, s is family-realizable in v if and only if it has one of the two forms:*

(R1) Type-1 on a root block: *s is $\ell \le P(\varphi) \le u$ with $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$ and $\mathrm{pa}_v = \varnothing$. Then $P(\varphi)$ is a function of $P(\mathrm{ch}_v)$, which* is *the family's table.*

(R2) Type-2 pinning one full parent configuration: *s is $\ell \le P(\varphi \mid \psi) \le u$ with $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$, $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$, and ψ satisfied by* exactly one *assignment $\pi^\star$ of pa_v. Then $P(\varphi \mid \psi) = P(\varphi \mid \mathrm{pa}_v{=}\pi^\star)$ is a single row of the table.*

A sentence is non-realizable *if it is family-realizable in no family. The LCN is* fully realizable *if every sentence is family-realizable in some family.*

The next lemma is what makes Definition 8 a *criterion* rather than a description: it shows the two forms (R1)–(R2) exhaust the $P(\mathrm{pa}_v)$-invariant sentences, so realizability can be decided syntactically.

Lemma 5 (Invariance criterion)**.** *Let v be a family with $\mathrm{pa}_v \neq \varnothing$ and let s be a sentence with $\mathrm{atoms}(s) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$. Write π for assignments of pa_v, $w_\pi = P(\mathrm{pa}_v{=}\pi)$ for the parents' marginal, and $T(\cdot \mid \pi) = P(\mathrm{ch}_v{=}\cdot \mid \mathrm{pa}_v{=}\pi)$ for the table.*

(i) *If s is Type-1, $\ell \le P(\varphi) \le u$, then $P(\varphi) = \sum_\pi w_\pi\, T(\varphi_\pi \mid \pi)$ where $T(\varphi_\pi \mid \pi) := \sum_{c\,:\,(c,\pi)\models\varphi} T(c \mid \pi)$. This is $P(\mathrm{pa}_v)$-invariant for all tables iff the map $\pi \mapsto T(\varphi_\pi \mid \pi)$ is constant for every table — which fails whenever $|\{\pi\}| \ge 2$ and φ is non-trivial. Hence no Type-1 sentence is realizable in a non-root family.*

(ii) *If s is Type-2 with $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$ and $\Pi_\psi := \{\pi : \pi \models \psi\}$, then on $\{P(\psi) > 0\}$*

$$P(\varphi \mid \psi) = \sum_{\pi \in \Pi_\psi} \lambda_\pi\, T(\varphi_\pi \mid \pi), \qquad \lambda_\pi = \frac{w_\pi}{\sum_{\pi' \in \Pi_\psi} w_{\pi'}}, \tag{25}$$

a convex combination of table rows with weights λ determined by $P(\mathrm{pa}_v)$. If $|\Pi_\psi| = 1$ this is one row, hence invariant; if $|\Pi_\psi| \ge 2$ it is invariant for all tables iff the rows agree, so s is not *realizable.*

Proof. (i) Condition the Type-1 probability on the parent block. Since $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$, the truth value of φ at a joint assignment is determined by $(\mathrm{ch}_v, \mathrm{pa}_v)$, so by Lemma 1 $P(\varphi) = \sum_\pi P(\mathrm{pa}_v{=}\pi)\, P(\varphi \mid \mathrm{pa}_v{=}\pi) = \sum_\pi w_\pi T(\varphi_\pi \mid \pi)$, which is the stated formula. This is an affine function of the weight vector w on the simplex, and an affine function on a simplex is constant iff all its coefficients coincide; hence invariance for *every* table forces $\pi \mapsto T(\varphi_\pi \mid \pi)$ to be constant. Choose a table with $T(\varphi_{\pi_1} \mid \pi_1) = 1$ and $T(\varphi_{\pi_2} \mid \pi_2) = 0$ for two distinct parent configurations $\pi_1 \neq \pi_2$ (possible whenever φ is satisfiable and refutable within ch_v, i.e. non-trivial); then the coefficients differ and invariance fails. So s is not realizable in v.

(ii) With $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$, the event ψ is a union of parent configurations, $\{\psi\} = \bigcup_{\pi \in \Pi_\psi} \{\mathrm{pa}_v{=}\pi\}$, a disjoint union. Hence $P(\psi) = \sum_{\pi \in \Pi_\psi} w_\pi$ and, by the computation in (i) restricted to Π_ψ, $P(\varphi \wedge \psi) = \sum_{\pi \in \Pi_\psi} w_\pi T(\varphi_\pi \mid \pi)$. Dividing gives equation 25, and λ is a probability vector supported on Π_ψ.

If $|\Pi_\psi| = 1$, say $\Pi_\psi = \{\pi^\star\}$, then $\lambda_{\pi^\star} = 1$ and $P(\varphi \mid \psi) = T(\varphi_{\pi^\star} \mid \pi^\star)$, a single row — invariant, and this is (R2). If $|\Pi_\psi| \ge 2$, then equation 25 is a non-degenerate convex combination: pick $\pi_1 \neq \pi_2$ in Π_ψ and a table with $T(\varphi_{\pi_1} \mid \pi_1) = 1$, $T(\varphi_{\pi_2} \mid \pi_2) = 0$. Taking w concentrated near π_1 gives $P(\varphi \mid \psi) \to 1$, and near π_2 gives $\to 0$, so the value genuinely depends on $P(\mathrm{pa}_v)$ and s is not realizable. Equivalently, $\partial P(\varphi \mid \psi)/\partial \lambda_\pi = T(\varphi_\pi \mid \pi)$ is not constant across Π_ψ. □

D.1 A FIVE-VARIABLE CHAIN-GRAPH EXAMPLE

We show next and example of family-realizability on a five-atom chain graph.

Example 5. *Consider the following family-realizable k-tree LCN with n=5 atoms k=3:*

$s_0 : 0.01 \le P(x_3) \le 0.399975$ $\quad$ $s_4 : 0.673756 \le P(\neg x_0 \mid x_3 \wedge x_4 \wedge \neg x_1) \le 1.0$

$s_1 : 0.01 \le P(x_4 \mid x_3) \le 0.442867$ $\quad$ $s_5 : 0.748845 \le P(\neg x_3) \le 0.954666$

$s_2 : 0.638553 \le P(x_2 \mid \neg x_3 \wedge x_4) \le 1.0$ $\quad$ $s_6 : 0.141404 \le P(x_3) \le 0.247704$

$s_3 : 0.01 \le P(\neg x_1 \mid x_3 \wedge x_4 \wedge x_2) \le 0.307066$

The chain graph has five singleton-block families and there is a single LMC assertion:

$$x_3 \mid \varnothing, \quad x_4 \mid x_3, \quad x_2 \mid x_3, x_4, \quad x_1 \mid x_2, x_3, x_4, \quad x_0 \mid x_1, x_3, x_4; \qquad (x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3, x_4).$$

Table 4 classifies every sentence. Two points are worth drawing out. First, the attached *column (what the scope-subset test collects) is much larger than the* realizable-in *column: the root marginals s_0, s_5, s_6 are attached to all five families but realizable only in the root family x_3, which is exactly why Definition 8 quantifies existentially over families. Second, each of s_1, s_2, s_3, s_4 conditions on a* full *conjunction of signed literals over all of its family's parents, so $|\Pi_\psi| = 1$ and (R2) applies — this is what the random problem generator guarantees for every family-realizable problem class.*

Table 4: Sentence classification. Every sentence is family-realizable in exactly one family, so the instance is fully realizable.

sentence	form	attached to families	realizable in	why
s_0, s_5, s_6	Type-1 $P(x_3)$, $P(\neg x_3)$	all five	$x_3 \mid \varnothing$	(R1) root block
s_1	Type-2, $\psi = x_3$	x_4, x_2, x_1, x_0	$x_4 \mid x_3$	(R2) $\lvert\Pi_\psi\rvert$=1
s_2	Type-2, $\psi = \neg x_3 \wedge x_4$	x_2, x_1	$x_2 \mid x_3, x_4$	(R2) $\lvert\Pi_\psi\rvert$=1
s_3	Type-2, $\psi = x_3 \wedge x_4 \wedge x_2$	x_1	$x_1 \mid x_2, x_3, x_4$	(R2) $\lvert\Pi_\psi\rvert$=1
s_4	Type-2, $\psi = x_3 \wedge x_4 \wedge \neg x_1$	x_0	$x_0 \mid x_1, x_3, x_4$	(R2) $\lvert\Pi_\psi\rvert$=1

Measured bounds. *The exact method with a global nonlinear solver (e.g., SCIP) certifies all ten solves (gap 0), and the new `CJT` reproduces every endpoint:*

engine	$P(x_0)$	$P(x_1)$	$P(x_2)$	$P(x_3)$	$P(x_4)$
`Exact` *(global)*	$[0, 1]$	$[0, 1]$	$[0, 1]$	$[0.141404, 0.247704]$	$[0.001414, 0.921219]$
`CJT` *(SCIP)*	$[0, 1]$	$[0, 1]$	$[0, 1]$	$[0.141404, 0.247704]$	$[0.001414, 0.921219]$

The vacuous $[0, 1]$ on x_0, x_1, x_2 is genuine, not a solver artifact: CJT exhibits explicit feasible witnesses attaining both endpoints (e.g. setting $P(x_2 \mid \neg x_3 \wedge x_4) = 0$ makes s_3's conditioning event unreachable, freeing x_1). The junction tree has maximal cliques $\{x_0, x_1, x_3, x_4\}$ and $\{x_1, x_2, x_3, x_4\}$ — treewidth 3 — and the separator between them is exactly $\{x_1, x_3, x_4\}$, the conditioning set of the lone LMC assertion; this is the certificate that hypotheses H1–H4 of Theorem 1 hold.

E THE LCN SET VERSUS THE STRONG EXTENSION

We now relate $\mathcal{D}_{\mathrm{LCN}}$ and $\mathcal{D}_{\mathrm{SE}}$.

Call a chain-graph LCN *factorizing* if every $p \in \mathcal{D}_{\mathrm{LCN}}$ satisfies equation 10 with the conditionals induced by p. This holds for every DAG-structured LCN (all chain components are single atoms), because the local Markov property of a DAG implies its factorization for every distribution (chain rule along a topological order, then the local independences). It holds for general chain graphs if $\mathcal{D}_{\mathrm{LCN}}$ is restricted to strictly positive distributions (Cozman, 2023). Positivity cannot simply be dropped for multi-atom components: the atom-wise LMC assertions $(C \perp\!\!\!\perp \mathbf{Y} \mid D, \mathbf{Z})$, $(D \perp\!\!\!\perp \mathbf{Y} \mid C, \mathbf{Z})$ of a component $\{C, D\}$ give the component-wise $(\{C, D\} \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ only through the intersection property, which may fail when p has zeros.

Proposition 5 (Inclusion)**.** *Let $\mathcal{L}$ be a factorizing chain-graph LCN. Then $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{P}_{\mathrm{SE}} \subseteq \mathcal{D}_{\mathrm{SE}}$. Consequently, for every query, $[\underline{q}_{\mathrm{LCN}}, \overline{q}_{\mathrm{LCN}}] \subseteq [\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}]$: any engine that is exact or outer w.r.t. $\mathcal{D}_{\mathrm{SE}}$ is automatically a valid* outer *bound w.r.t. $\mathcal{D}_{\mathrm{LCN}}$.*

Proof. Let $p \in \mathcal{D}_{\mathrm{LCN}}$. Since $\mathcal{L}$ is factorizing, p satisfies equation 10: $p = \prod_v P_p(\mathrm{ch}_v \mid \mathrm{pa}_v)$, where $P_p(\cdot \mid \cdot)$ denotes the conditionals induced by p (defined arbitrarily on parent configurations of zero probability, which do not affect the product).

Fix a family v and a parent configuration π with $P_p(\mathrm{pa}_v{=}\pi) > 0$. The row $P_p(\mathrm{ch}_v \mid \mathrm{pa}_v{=}\pi)$ is a conditional table attainable by the joint p, and p satisfies every sentence of $\mathcal{L}$ — in particular every sentence whose atom scope lies inside $\mathrm{ch}_v \cup \mathrm{pa}_v$. But $K_{v,\pi}$ was defined as the set of rows attainable by *some* joint satisfying exactly those in-scope sentences. Hence $P_p(\mathrm{ch}_v \mid \mathrm{pa}_v{=}\pi) \in K_{v,\pi}$: the local set is a *relaxation* of the projection, so it contains it. For a configuration with $P_p(\mathrm{pa}_v{=}\pi) = 0$ the row is unconstrained by p and may be chosen to be any element of $K_{v,\pi}$ (non-empty, since p itself witnesses feasibility of the in-scope sentences) without changing the product.

Thus p is exhibited as a product $\prod_v \prod_\pi$ of rows drawn from the respective $K_{v,\pi}$, which is precisely membership in $\mathcal{P}_{\mathrm{SE}}$ as defined in equation 11; and $\mathcal{P}_{\mathrm{SE}} \subseteq \mathrm{CH}(\mathcal{P}_{\mathrm{SE}}) = \mathcal{D}_{\mathrm{SE}}$. The bound inclusion follows because minimizing and maximizing a fixed functional over a larger set yields a wider interval. □

The sufficiency result needs every local row to be realized inside $\mathcal{D}_{\mathrm{LCN}}$ ("no Gap A"). Under full realizability this is not an extra assumption.

Lemma 6 (Realizability closes Gap A)**.** *Let $\mathcal{L}$ be a consistent, factorizing chain-graph LCN in which every sentence is family-realizable in some family. Then for every family v and parent configuration π, $K_{v,\pi} = K^{\mathrm{true}}_{v,\pi}$, the projection of $\mathcal{D}_{\mathrm{LCN}}$ onto the row $P(\mathrm{ch}_v \mid \mathrm{pa}_v{=}\pi)$.*

Proof. $K^{\mathrm{true}}_{v,\pi} \subseteq K_{v,\pi}$ is the inclusion argument of Proposition 5. Conversely, let $r \in K_{v,\pi}$. By consistency pick $p^0 \in \mathcal{D}_{\mathrm{LCN}}$, and let q be the product of the conditional rows of p^0 with the single row (v, π) replaced by r. We show $q \in \mathcal{D}_{\mathrm{LCN}}$; then r is the (v, π) row of a member of $\mathcal{D}_{\mathrm{LCN}}$ (on $\{q(\mathrm{pa}_v{=}\pi) = 0\}$ the row is immaterial, and the cleared sentences are vacuous there), so $r \in K^{\mathrm{true}}_{v,\pi}$.

Sentences. By realizability each sentence s is a function of one row of one family (forms R1/R2). If that row is not (v, π), q and p^0 share it, so $g_s(q) = g_s(p^0) \in [\ell_s, u_s]$. If it is (v, π), then $\mathrm{atoms}(s) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$, so s is among the in-scope sentences of the local program defining $K_{v,\pi}$. That program ranges over joints on the family scope; by R1/R2 the value of s on any such joint is a function of its (v, π) row alone, so every row in the projection $K_{v,\pi}$ — in particular r — satisfies s. The other in-scope sentences of that program are realizable in *other* families and constrain only rows of those families (e.g. a root marginal of a parent); they restrict the local program's parent marginal, not its (v, π) row, and are satisfied at q because q inherits those rows from p^0.

LMC. q is a product of the form equation 10; factorization implies the global Markov property of the chain graph without positivity, so q satisfies every LMC assertion. Hence $q \in \mathcal{D}_{\mathrm{LCN}}$. □

Proposition 6 (Sufficiency)**.** *Let $\mathcal{L}$ be a consistent, factorizing chain-graph LCN with local sets built by chain-graph factorization, in which every sentence is family-realizable in some family (Definition 8). Then $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$, and by Lemma 2 $\mathcal{D}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{LCN}}$ have the same lower and upper probabilities.* Single-connectedness of the primal graph is not required.

Proof. Proposition 5 gives $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{P}_{\mathrm{SE}}$, so only $\mathcal{P}_{\mathrm{SE}} \subseteq \mathcal{D}_{\mathrm{LCN}}$ needs proof. Let $q = \prod_v \prod_\pi P_{v,\pi}$ with each $P_{v,\pi} \in K_{v,\pi}$ be an arbitrary element of $\mathcal{P}_{\mathrm{SE}}$. We verify that q satisfies both families of constraints in equation 9.

LMC equalities. By construction q is a product of the form equation 10 over the families of the chain graph. By the converse direction of the chain-graph factorization theorem (Lauritzen, Thm. 3.34: factorization implies the global Markov property of the chain graph, and this direction needs no positivity assumption), q satisfies every conditional independence entailed by the chain graph — in particular every LMC assertion, since the LMC is by definition read off that same graph. Hence $h_\iota(q) = 0$ for all ι. Note this step is where the "loopiness contributes nothing" claim comes from: a product over the chain-graph families satisfies the LMC whether or not the graph is singly connected.

Sentences. Let s be a sentence. By (B) it is family-realizable in some family v, so by Definition 8 the value $g_s(q)$ is determined by the conditional table of q at v alone. That table is the collection of rows $\{P_{v,\pi}\}_\pi$, and by Lemma 6 each $P_{v,\pi} \in K^{\mathrm{true}}_{v,\pi}$, i.e. each row is realized by *some* $p^{(\pi)} \in \mathcal{D}_{\mathrm{LCN}}$. It remains to see that this forces $g_s(q) \in [\ell_s, u_s]$. Two cases, matching the two clauses of Definition 8.

If s has form (R2), then $g_s(q) = P_{v,\pi^\star}(\varphi_{\pi^\star})$ is the *single* row $\pi^\star$. That row equals the corresponding row of some $p^{(\pi^\star)} \in \mathcal{D}_{\mathrm{LCN}}$; by Definition 8 again (applied to $p^{(\pi^\star)}$, whose table has this same row) $g_s(p^{(\pi^\star)}) = g_s(q)$, and $p^{(\pi^\star)} \in \mathcal{D}_{\mathrm{LCN}}$ satisfies s. Hence $g_s(q) \in [\ell_s, u_s]$.

If s has form (R1), then $\mathrm{pa}_v = \varnothing$, so the family has the single (empty) parent configuration and its table is the whole marginal $P(\mathrm{ch}_v) = P_{v,\varnothing}$. The same argument applies verbatim with $\pi^\star$ the empty configuration.

In both cases the key point is that realizability makes g_s a function of *one row*, and a row realized inside $\mathcal{D}_{\mathrm{LCN}}$ (Lemma 6) carries its constraint value with it — so the free, independent choice of the *other* rows and families cannot disturb it. Therefore every sentence holds at q, giving $q \in \mathcal{D}_{\mathrm{LCN}}$. □

We now give an example of a loopy LCN on which CVE is exact.

Example 6. *Single-connectedness is not necessary for exactness. Consider the following family-realizable k-tree LCN with n=2 atoms and k=3:*

$$s_0\text{:}\ 0.084382 \le P(x_1) \le 0.684382 \qquad s_2\text{:}\ 0.092785 \le P(x_3 \mid x_1 \wedge x_2) \le 0.692785$$
$$s_1\text{:}\ 0.01 \le P(\neg x_2 \mid \neg x_1) \le 0.572656 \quad s_3\text{:}\ 0.068242 \le P(x_0 \mid x_1 \wedge x_3) \le 0.668242$$

*The seed 3-clique is $\{x_1, x_2, x_3\}$ and x_0 attaches to the 2-clique $\{x_1, x_3\}$, so the moralized graph contains the triangle $x_1x_2x_3$: the instance is **loopy**, decidedly not singly connected. It is also fully realizable (each ψ is a full conjunction over its family's parents; s_0 is a root marginal). Measured, with all eight SCIP solves* `confirmed` *at gap* 0*:*

engine	$P(x_0)$	$P(x_1)$	$P(x_2)$	$P(x_3)$
`Exact` *(global)*	$[0, 1]$	$[0.0844, 0.6844]$	$[0.1349, 0.9968]$	$[0, 1]$
`CJT` *(SCIP)*	$[0, 1]$	$[0.0844, 0.6844]$	$[0.1349, 0.9968]$	$[0, 1]$
CVE	$[0, 1]$	$[0.0844, 0.6844]$	$[0.1349, 0.9968]$	$[0, 1]$

CVE is exact on every atom *of a loopy LCN.*

Remark 8 (What loopiness *does* cost)**.** *Loopiness is not free — but its price is paid in* cost*, not accuracy. On a five-atom k-tree LCN of Example 5, the credal network builds essentially instantly and is tiny ($8 \cdot 8 \cdot 4 \cdot 1 \cdot 2 = 512$ vertex combinations across the five families), yet CVE did not terminate within* 200 *s: it stalls inside the pruning stage during elimination.* `CJT` *solves the same instance in about a second. So on bounded-treewidth loopy classes the practical recommendation — prefer* `CJT` *— stands, but the reason is tractability, not looseness.*

F Detailed Description of Credal Variable Elimination

We present next three inference algorithms that leverage the chain graph factorization of an LCN to compute marginal probability bounds. The first, Credal Variable Elimination (CVE), extends classical variable elimination Dechter (1996) to credal networks by propagating credal potentials, represented as finite sets of non-negative functions Mauá & Cozman (2020).

F.1 Credal Networks

A **credal network** is a pair $\langle G, \mathcal{K} \rangle$ where G is a directed acyclic graph (DAG) over variables $\mathbf{V} = \{V_1, \ldots, V_m\}$ with finite domains $\mathcal{D} = \{D_1, \ldots, D_n\}$, and $\mathcal{K}$ is a collection of conditional credal sets $K(V_i \mid \mathrm{pa}(V_i) = \pi)$, one for each variable V_i and each configuration π of its parents $\mathrm{pa}(V_i)$ in G.

Each credal set $K(V_i \mid \pi)$ is a closed convex set of probability distributions over V_i, specified by its *extreme points* (vertices):

$$\mathrm{ext}\, K(V_i \mid \pi) = \{P_1(V_i \mid \pi), \ldots, P_v(V_i \mid \pi)\}.$$

F.2 Strong Extension

The **strong extension** of a separately specified credal network is the convex hull of all joint distributions formed by combining one extreme point per local credal set:

$$D_{\mathrm{SE}}(\mathbf{V}) = \mathrm{CH}\Big\{P(\mathbf{x}) = \prod_{i=1}^{n} P(x_i \mid \pi_i) : P(V_i \mid \pi_i) \in \operatorname{ext} K(V_i \mid \pi_i)\Big\}. \tag{26}$$

Every extreme point of the strong extension is such a product of local extreme points (Mauá & Cozman, 2020) (not every product need be extreme). Thus the strong extension can be built directly from the local extreme points.

F.3 Marginal Inference

Given an atom a and evidence $\mathbf{y}$ with $\underline{P}(\mathbf{y}) > 0$, the goal is to compute

$$\underline{P}(a \mid \mathbf{y}) = \min_{P} \frac{\sum_{\mathbf{v} \models a \wedge \mathbf{y}} \prod_i P(v_i \mid \pi_i)}{\sum_{\mathbf{v} \models \mathbf{y}} \prod_i P(v_i \mid \pi_i)}, \tag{27}$$

where P ranges over the products of one extreme point per local credal set, and $\overline{P}(a \mid \mathbf{y})$ is defined likewise with $\max$. Since the ratio is linear-fractional in each local conditional, these are also the bounds over $\mathcal{D}_{\mathrm{SE}}$ (Lemma 2).

F.4 Pair Potentials

Definition 9 (Pair potential)**.** *A **potential** $\phi(\mathbf{Y})$ over variables $\mathbf{Y}$ is a finite set of pairs (p, q) of non-negative functions on the joint domain of $\mathbf{Y}$. Each pair corresponds to one choice of extreme points of the local credal sets that have been multiplied into it (Mauá et al., 2012; Mauá & Cozman, 2020).*

Product and sum-marginal. The product of $\phi(\mathbf{Y})$ and $\psi(\mathbf{Z})$ is $\phi \cdot \psi = \{(pr,\, qs) : (p, q) \in \phi,\ (r, s) \in \psi\}$ over $\mathbf{Y} \cup \mathbf{Z}$, and the sum-marginal sums both components, $\sum_X \phi = \{(\sum_x p,\ \sum_x q) : (p, q) \in \phi\}$.

Input potentials. A family V_i with parent configurations $\pi_1, \ldots, \pi_r$ contributes $\phi_{V_i} = \{(f, f)\}$, one function f per element of $\operatorname{ext} \mathcal{K}(V_i \mid \pi_1) \times \cdots \times \operatorname{ext} \mathcal{K}(V_i \mid \pi_r)$ (a complete CPT choosing one extreme point in every column). The potential is therefore an explicit enumeration of the family's contribution to the strong extension. Evidence $Y = y$ on an atom multiplies the indicator δ_y into both components of its variable's potential. For an atom a of the variable Z with $0/1$ indicator t over Z's states, the *query potential* is $\{(1 - t,\ t)\}$.

Read-off. After every variable has been eliminated, each pair is a pair of numbers $p = P(\neg a, \mathbf{y})$ and $q = P(a, \mathbf{y})$ for one selection of extreme points, so

$$\underline{P}(a \mid \mathbf{y}) = \min_{(p,q),\ p+q>0} \frac{q}{p+q}, \qquad \overline{P}(a \mid \mathbf{y}) = \max_{(p,q),\ p+q>0} \frac{q}{p+q}.$$

The ratio is taken per pair, which is the conditioning step: it is *not* the invalid ratio $\min P(a, \mathbf{y}) / \max P(\mathbf{y})$ of a separately optimized numerator and denominator. Pairs with $p + q = 0$ give the evidence probability zero and are dropped.

Definition 10 (Pareto pruning)**.** *For the upper bound, (p, q) is* dominated *when another pair (r, s) has $r \le p$ and $s \ge q$ pointwise. For the lower bound the order is reversed.* $\mathrm{prune}(\phi)$ *removes the pairs that are dominated in* both *orders, so one elimination serves both bounds.*

Definition 11 (Hull pruning)**.** *For a potential whose pairs all have $p = q$ (no query multiplied in yet),* $\mathrm{prune}_{\mathrm{hull}}(\phi)$ *keeps the pairs (f, f) whose ray $f / \sum f$ is a vertex of the convex hull of $\{f / \sum f : (f, f) \in \phi\}$. The kept pairs are stored rescaled (to $\max f = 1$), which keeps long products of probabilities in floating-point range and changes no ratio $q/(p + q)$.*

Definition 12 (α-quotient (Mauá et al., 2012))**.** *For $\alpha > 1$, two pairs are* α-equivalent *when, after scaling each pair to $\sum(p + q) = 1$, every coordinate of both components is zero in both or positive in both with equal $\lfloor \log_\alpha \cdot \rfloor$. The quotient ϕ/α keeps one member (a real pair) of each equivalence class.*

Algorithm 2 Credal Variable Elimination (CVE) and its FPTAS CVE_ε

Require: Extreme points $\operatorname{ext}\mathcal{K}(V_i \mid \mathrm{pa}(V_i))$ for each variable V_i; query variable Z; evidence $\mathbf{y}$; $\varepsilon \geq 0$
Ensure: $[\underline{P}(a \mid \mathbf{y}),\ \overline{P}(a \mid \mathbf{y})]$ for each atom a of Z

$\Gamma \leftarrow \{\phi_{V_i} : V_i \in \mathbf{V}\}$ with evidence indicators multiplied in
$\mathbf{o} \leftarrow$ an elimination ordering of $\mathbf{V} \setminus \{Z\}$ (min-fill)
$N \leftarrow$ number of pairwise products and summations below, plus one; $\alpha \leftarrow (1+\varepsilon)^{1/(2N)}$ ▷ $\alpha = 1$: exact
for each V_i in ordering $\mathbf{o}$ **do**
  $\Gamma_{V_i} \leftarrow \{\phi \in \Gamma : V_i \in \mathrm{scope}(\phi)\}$; $\Gamma \leftarrow \Gamma \setminus \Gamma_{V_i}$
  $\psi \leftarrow$ the product of Γ_{V_i}, one pairwise product at a time, each followed by $/\alpha$
  $\psi \leftarrow \mathrm{prune}_{\mathrm{hull}}\big((\sum_{V_i} \psi)/\alpha\big)$ ▷ $p = q$ throughout
  $\Gamma \leftarrow \Gamma \cup \{\psi\}$
$\phi_Z \leftarrow$ the product of Γ (each pairwise product followed by $/\alpha$ and $\mathrm{prune}_{\mathrm{hull}}$) ▷ scope $\{Z\}$
for each atom a of Z not in the evidence **do**
  $\phi \leftarrow \mathrm{prune}\big(\big(\sum_Z \phi_Z \cdot \{(1-t_a,\, t_a)\}\big)/\alpha\big)$
  report $\big[\min_\phi \frac{q}{p+q},\ \max_\phi \frac{q}{p+q}\big]$

F.5 Algorithm

CVE answers all atom marginals of one variable Z by a single bucket-elimination pass in which Z is eliminated *last* (Algorithm 2). The variables of the credal network are chain-graph node families, not atoms: a variable carries the atoms $\mathrm{atoms}(Z)$ and has $2^{|\mathrm{atoms}(Z)|}$ states (most significant bit first). Because the query enters only after every other variable is gone, all of Z's atoms share the elimination. Each atom then costs only the final product with its query potential, so a compound variable needs no projection of per-state bounds onto its atoms.

All marginals of the network are obtained by running Algorithm 2 once per variable, each with its own ordering. Nothing is shared between variables.

F.6 Elimination Ordering

Two heuristics are supported; both hold Z out of the ordering.

- **Reverse topological order** of the DAG, restricted to $\mathbf{V} \setminus \{Z\}$.
- **Min-fill**: the greedy interaction-graph heuristic, eliminating at each step the variable that adds the fewest fill-in edges, with an alphabetical tiebreak. Z still counts as a neighbour when fill is scored. This aims at a small induced width w^*.

F.7 Running Example

Example 7 (Alarm Network)**.** *The LCN* `examples/alarm.lcn` *over atoms A, B, C, D, E factorizes into four credal-network nodes: the singletons B (Burglary), E (Earthquake), A (Alarm), and the* compound *node C-D carrying the two atoms C, D. They are merged because the sentence $0.5 \leq P(\neg(C \oplus D) \mid A) \leq 0.7$ constrains them jointly. The compound node has $2^2 = 4$ states, ordered $CD \in \{00, 01, 10, 11\}$ most-significant bit first. The DAG is*

$$B \to A, \quad E \to A, \quad A \to C\text{-}D.$$

The local credal sets (extreme points from LRS vertex enumeration) are:

Node	*Parent Config*	*#*	*Vertices*
B	—	*2*	$[0.8, 0.2]$, $[0.9, 0.1]$
E	—	*2*	$[0.9, 0.1]$, $[0.95, 0.05]$
A	B=0, E=0	*2*	$[0.956, 0.044]$, $[1.0, 0.0]$
A	B=1, E=0	*2*	$[0.0, 1.0]$, $[1.0, 0.0]$
A	B=0, E=1	*2*	$[0.0, 1.0]$, $[1.0, 0.0]$
A	B=1, E=1	*2*	$[0.0, 1.0]$, $[1.0, 0.0]$
C-D	A=0	*4*	$[0, 0, 0, 1]$, $[0, 0, 1, 0]$, $[0, 1, 0, 0]$, $[1, 0, 0, 0]$
C-D	A=1	*11*	$[0, 0, 0.3, 0.7]$, $[0, 0.3, 0, 0.7]$, …, $[0.7, 0, 0, 0.3]$

Initial potentials. *The potential sizes are the* products *of the counts above:* $|\phi_B| = 2$, $|\phi_E| = 2$, $|\phi_A| = 2^4 = 16$, $|\phi_{C\text{-}D}| = 4 \times 11 = 44$.

Query: the atom B (no evidence). *Min-fill with B held out gives* $\mathbf{o} = (C\text{-}D, A, E)$.

***Step 1: eliminate** C**-**D**.** *The bucket is $\phi_{C\text{-}D}$ alone. Summing out C-D maps each of its* 44 *functions to the constant* 1 *over* $\{A\}$ *(each is a conditional table), and hull pruning leaves a single function.*

***Step 2: eliminate** A**.** *The bucket is ϕ_A (*16 *functions over* $\{A, B, E\}$*) and the constant potential. Their product has* 16 *functions, and summing out A maps all of them to the constant* 1 *over* $\{B, E\}$*. Hull pruning therefore keeps a single function, up to floating-point rounding: the enumerated vertices sum to one only to about* $4 \cdot 10^{-7}$*, and a near-duplicate kept for this reason is harmless.*

***Step 3: eliminate** E**.** *The product with ϕ_E is again summed to the constant* 1 *over* $\{B\}$*, so the query step sees ϕ_B alone.*

***Result.** The product with ϕ_B and the query potential* $\{(1 - t_B,\, t_B)\}$*, summed over B, gives pairs whose ratios $q/(p+q)$ range over*

$$\underline{P}(B{=}1) = 0.10, \qquad \overline{P}(B{=}1) = 0.20,$$

exactly the interval of the sentence $0.1 \le P(B) \le 0.2$*, as it must be since B is a root. The largest potential in this pass has* 16 *elements. Plain Pareto pruning of single functions removes nothing here, because no conditional table dominates another (Lemma 8); along the forward topological order it keeps* 328 *functions in the last step.*

All marginals. *The four passes give* $P(A{=}1) \in [0, 0.312]$, $P(B{=}1) \in [0.1, 0.2]$, $P(E{=}1) \in [0.05, 0.1]$ *and* $P(C{=}1), P(D{=}1) \in [0, 1]$. *The last two come directly from the query indicators of the compound node, and are vacuous because A=0 leaves C-D unconstrained. With evidence C=1 the same code returns the posterior bounds over the strong extension. CVE$_\varepsilon$ with* $\varepsilon = 0.1$ *returns the same intervals to four decimals.*

F.8 Proofs of Theorems 3, 4 and Proposition 3

The complexity bound of Theorem 3 follows from Mauá & Cozman (2020): each bucket combines at most C pairs over at most $w^* + 1$ variables of domain size at most d, and there are m buckets. The polynomial bound on C for CVE$_\varepsilon$ is Proposition 7 and Theorem 11 of Mauá et al. (2012): the number of α-equivalence classes over a scope of size $|\Omega|$ is at most $(1 - \lfloor \log_\alpha t \rfloor)^{2|\Omega|}$, where $t \ge 2^{-bn}$ is the smallest positive entry and b the input size in bits, which is polynomial in b and $1/\varepsilon$ when $|\Omega| \le d^{w^*+1}$ is bounded. The extra exact prunings only remove pairs.

Exactness rests on one invariant, shared by the three prunings. For non-negative weight vectors u, v on a scope's domain and a pair (p, q) with $\langle u, q\rangle + \langle v, p\rangle > 0$, let

$$R_{u,v}(p, q) = \frac{\langle u, q\rangle}{\langle u, q\rangle + \langle v, p\rangle}.$$

The weights stand for everything the rest of the elimination does to a pair: fix one element of every other potential, and the final pair of numbers is $(\langle v, p\rangle, \langle u, q\rangle)$, with v and u the products and sums contributed by those elements. Its read-off $q/(p+q)$ is $R_{u,v}(p, q)$.

Definition 13 (Covering)**.** *For $c \geq 1$, a set $S' \subseteq S$ of pairs* c-covers *S if for all $u, v \geq 0$ and every $(p, q) \in S$ with $R_{u,v}(p, q)$ defined there are $(r, s), (r', s') \in S'$ with $R_{u,v}$ defined and*

$$R_{u,v}(r, s) \;\geq\; R_{u,v}(p, q)/c, \qquad R_{u,v}(r', s') \;\leq\; c\, R_{u,v}(p, q).$$

Lemma 7 (Covering calculus)**.** *1.* Pareto pruning *(Def. 10), removal of exact duplicates and of pairs with $p = q = 0$, and the rescaling of query-free pairs* 1*-cover.*

2. Hull pruning *(Def. 11) of a potential with $p = q$* 1*-covers.*

3. The α-quotient *(Def. 12)* α^2*-covers.*

4. If S' c-covers S and T' d-covers T, then $S' \cdot T'$ cd-covers $S \cdot T$. Sum-marginals preserve the factor, and if S'' c'-covers S', which c-covers S, then S'' cc'-covers S.

Proof. (1) $R_{u,v}$ is non-decreasing in q and non-increasing in p, so a pair (r, s) with $r \leq p$, $s \geq q$ has $R_{u,v}(r, s) \geq R_{u,v}(p, q)$, and the reverse order serves the minimum. A pair is removed only when it is dominated in both orders. Pairs with $p = q = 0$ have no defined $R_{u,v}$. Duplicates and positive rescalings of a query-free pair (f, f), which give (cf, cf), leave $R_{u,v}$ unchanged, because the factor cancels; with $p = q$ the denominator is $\langle u + v, f\rangle$. (2) With $p = q = f$, $R_{u,v}(f, f) = \langle u, f\rangle/\langle u + v, f\rangle$ is homogeneous of degree 0, so it depends only on the ray $\hat{f} = f/\sum f$. If $\hat{f} = \sum_k \lambda_k \hat{f}_k$ is a convex combination of kept rays, then $R_{u,v}(\hat{f}) = \sum_k \mu_k R_{u,v}(\hat{f}_k)$ with $\mu_k \propto \lambda_k \langle u+v, \hat{f}_k\rangle \geq 0$, summing over the k with $\mu_k > 0$, for which $R_{u,v}$ is defined. These are the weights of Lemma 2. So some kept pair has at least, and some at most, the removed pair's value. (3) After scaling both pairs to $\sum(p + q) = 1$, which changes no $R_{u,v}$ because both components carry the same factor, an α-equivalent kept pair (r, s) has $r \leq \alpha p$, $s \geq q/\alpha$, $r \geq p/\alpha$ and $s \leq \alpha q$ pointwise. Write $x = \langle u, q\rangle$ and $y = \langle v, p\rangle$. Then $R_{u,v}(r, s) \geq \frac{x/\alpha}{x/\alpha+\alpha y} = \frac{x}{x+\alpha^2 y} \geq \frac{1}{\alpha^2}\frac{x}{x+y}$ and $R_{u,v}(r, s) \leq \frac{\alpha x}{\alpha x+y/\alpha} = \frac{\alpha^2 x}{\alpha^2 x+y} \leq \alpha^2 \frac{x}{x+y}$. In particular $R_{u,v}(r, s)$ is defined whenever $R_{u,v}(p, q)$ is, since zero patterns agree. (4) For a product, fix $\tau = (p_\tau, q_\tau) \in T$. Then $R_{u,v}(\phi \cdot \tau) = R_{u \circ q_\tau,\, v \circ p_\tau}(\phi)$, so some $\psi \in S'$ achieves the factor c. Fixing ψ, the same argument in τ gives the factor d. A sum-marginal replaces (u, v) by their extensions, which are constant along the summed variable. Transitivity is immediate. □

Every operation of Algorithm 2 after the input potentials is a product, a sum-marginal or one of the reductions (1)–(3). Hence, by induction over the operations, with (4) for products and sums, the final set of a pass c-covers the set that brute-force elimination without any reduction would produce, the set of pairs $(P(\neg a, \mathbf{y}), P(a, \mathbf{y}))$ of all vertex products, where c is the product of the factors of the reductions applied.

Proof of Theorem 4. With $\varepsilon = 0$, CVE applies only the reductions (1) and (2), so the final set 1-covers the brute-force set. Its minimum and maximum of $q/(p + q)$ are therefore the minimum and maximum of $P(a \mid \mathbf{y})$ over the vertex products; $P(\mathbf{y}) > 0$ for all of them because $\underline{P}(\mathbf{y}) > 0$. $P(a \mid \mathbf{y})$ is linear-fractional in each local conditional with a positive denominator, so these are the extrema over $\mathcal{P}_{\mathrm{SE}}$ (optimize one local conditional at a time, which ends at a vertex), hence over $\mathcal{D}_{\mathrm{SE}}$ by Lemma 2. By Proposition 6, which needs consistency, factorization and full realizability, $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$, so these are the bounds over $\mathcal{D}_{\mathrm{LCN}}$. The query is the indicator of the atom over its variable's states, so atoms of compound variables are handled directly. If only $\overline{P}(\mathbf{y}) > 0$, the same computation returns the extrema over the vertex products with $P(\mathbf{y}) > 0$ (regular conditioning at the vertices, Mauá & Cozman 2020). □

Proof of Proposition 3. Algorithm 2 applies N quotients: one after every pairwise product and every summation, including the final products of the remaining potentials, and one in the query step. With the exact reductions, which have factor 1, the final set α^{2N}-covers the brute-force set, i.e. it $(1 + \varepsilon)$-covers it. Take the vertex product attaining $\overline{P}$: some final pair has $q/(p + q) \geq \overline{P}/(1 + \varepsilon)$; symmetrically some final pair has $q/(p + q) \leq (1 + \varepsilon)\underline{P}$. Every final pair is a real vertex product, since the reductions keep members and never synthesize pairs, so $\underline{P} \leq \underline{P}_\varepsilon$ and $\overline{P}_\varepsilon \leq \overline{P}$. □

Lemma 8 below explains why the reduction before the query enters comes from hull pruning rather than from dominance: without evidence every intermediate function is a conditional table, and distinct conditional tables never dominate one another.

Lemma 8 (Evidence-free potentials are conditional tables)**.** *Run bucket elimination on the credal network with no evidence, along any elimination order, starting from the potentials $\phi_X = \{P(X \mid \mathrm{pa}(X))\}$ whose functions are the vertex choices of X's local credal sets. For every potential ϕ created during elimination there is a partition of its scope into a* child block C_ϕ *and a* parent block R_ϕ*, determined by the elimination history alone, such that every $f \in \phi$ satisfies $\sum_{\mathbf{c}} f(\mathbf{c}, \mathbf{r}) = 1$ for every assignment $\mathbf{r}$ of R_ϕ. Consequently no function of ϕ is strictly dominated by another.*

Proof. Call a function *conditional* on (C, R) if it sums to 1 over C for each assignment of R, and assign to every potential the set $\mathcal{A}_\phi$ of variables whose own CPT has been multiplied into it; let $C_\phi = \mathcal{A}_\phi \cap \mathrm{scope}(\phi)$ and $R_\phi = \mathrm{scope}(\phi) \setminus C_\phi$. Initially ϕ_X is conditional on $(\{X\}, \mathrm{pa}(X))$.

Product. Each CPT enters exactly one bucket, so two potentials in a bucket have disjoint sets $\mathcal{A}_1, \mathcal{A}_2$ of absorbed CPTs. Every function of ϕ_i is a sum, over the eliminated variables of $\mathcal{A}_i$, of the product $\prod_{X \in \mathcal{A}_i} P(X \mid \mathrm{pa}(X))$. Fix a topological order of the DAG and sum the full product $\prod_{X \in \mathcal{A}_1 \cup \mathcal{A}_2} P(X \mid \mathrm{pa}(X))$ over all its child variables $\mathcal{A}_1 \cup \mathcal{A}_2$ from the topologically last to the first: each step sums one CPT over its own child while every other remaining factor is constant in it (no remaining factor has a later variable as parent), contributing 1. Hence a product of CPTs is conditional on (its child variables, the rest), and summing out some of its child variables — which is all elimination ever does, see below — preserves this. So every function of the bucket product is conditional on $(C_1 \cup C_2, (R_1 \cup R_2) \setminus (C_1 \cup C_2))$, with the same split for all functions.

Elimination. When the bucket of v is processed, every potential mentioning v is in it, including the one carrying v's own CPT, so $v \in C$ of the bucket product. Summing out a child-block variable preserves conditionality, with child block $C \setminus \{v\}$.

By induction every created potential is conditional on (C_ϕ, R_ϕ), the same split for all its functions. If $g \geq f$ pointwise with $g \neq f$, pick $\mathbf{r}$ with $g(\cdot, \mathbf{r}) \neq f(\cdot, \mathbf{r})$; then $\sum_{\mathbf{c}} g(\mathbf{c}, \mathbf{r}) > \sum_{\mathbf{c}} f(\mathbf{c}, \mathbf{r}) = 1$, a contradiction. □

G Detailed Description: Interval Belief Propagation

G.1 Factor Graph and Credal Messages

Given a credal network $\langle G, \mathcal{K} \rangle$, the factor graph has one variable node per variable V (cardinality $|D_V|$, which is $2^{|\mathrm{atoms}(V)|}$ for a chain-graph component), one *family factor* f_V with scope $\{V\} \cup \mathrm{pa}(V)$ storing the vertex sets $\mathrm{ext}K(V \mid \pi)$ of every row, and one *evidence factor* e_V for every V with observed atoms, storing the $0/1$ indicator δ_V of the observed atom values over D_V (for a compound variable, the states that agree with every observed atom of V).

Definition 14 (Credal message)**.** *A **credal message** on a variable V is a finite set $M \subset \Delta(D_V)$ of normalized vectors, stored as the vertices of its convex hull. It represents every message μ with $\mu / \sum \mu \in \mathrm{conv}(M)$. For a binary variable it is an interval on the second coordinate.*

For a fixed choice of local conditionals, the sum-product messages equation 6 are defined up to scale. As the local conditionals range over their credal sets, the normalized messages range over a set, which IVBP represents by its hull vertices. All messages start at the neutral message $\{\mathbf{1}/|D_V|\}$; an evidence factor always sends $\{\delta_V / \sum \delta_V\}$.

G.2 Message Updates

Write $\mathrm{nrm}(m) = m / \sum m$ and $\mathrm{hull}(\cdot)$ for the reduction to hull vertices. Vectors m with $\sum m = 0$ are dropped (regular conditioning: the evidence has probability zero for that choice).

Variable to factor (product). For the incoming credal messages $M_1, \ldots, M_r$ on V from the factors other than the receiving one,

$$M_{V \to f} = \mathrm{hull}\big\{\mathrm{nrm}(m_1 \circ \cdots \circ m_r) : m_j \in M_j\big\},$$

computed pairwise with a hull reduction after each product.

Algorithm 3 Interval Belief Propagation (IVBP)

Require: credal network $\langle G, \mathcal{K} \rangle$ (from the chain-graph factorization of $\mathcal{L}$); evidence $\mathbf{y}$; sweeps T, tolerance τ
Ensure: $[\underline{P}(a \mid \mathbf{y}), \overline{P}(a \mid \mathbf{y})]$ for every non-evidence atom a
build the factor graph: family factors f_V, evidence factors e_V
$M_{f \to v} \leftarrow \{\mathbf{1}/|D_v|\}$ for every edge
for $t = 1, \ldots, T$ **do**
  for each V in topological order (t odd) or reverse order (t even) **do**
    $M_{U \to f_V} \leftarrow$ product of the messages into U except from f_V, for $U \in \mathrm{pa}(V)$, and $M_{V \to f_V}$ likewise
    $M_{f_V \to V} \leftarrow \pi$-message; $M_{f_V \to U} \leftarrow \lambda$-message, for $U \in \mathrm{pa}(V)$
  if no message changed by more than τ (10^{-12} on a polytree) **then**
    break
$B_V \leftarrow$ product of all messages into V, including e_V
return $\left[\min_{b \in B_V} \sum_{s \models a} b(s),\ \max_{b \in B_V} \sum_{s \models a} b(s)\right]$ for each atom a of each V

Factor to child (π-message). For each *corner* $(q_1, \ldots, q_k)$, one vertex of the message of each parent $U_1, \ldots, U_k$, the weights $w(\pi) = \prod_j q_j(\pi_j)$ sum to one, and

$$\mu_{f_V \to V} = \sum_{\pi} w(\pi)\, P(V \mid \pi), \qquad P(V \mid \pi) \in \mathrm{ext}K(V \mid \pi),$$

so the set of these messages is the Minkowski sum $\sum_\pi w(\pi)\, \mathrm{ext}K(V \mid \pi)$, already normalized. Its vertices are sums of row vertices and are computed incrementally, one row at a time. $M_{f_V \to V}$ is the hull of the union over corners.

Factor to parent (λ-message). For the parent $U = U_i$ and each corner (λ, q_{-i}), one vertex λ of the message from V into f_V (the product of V's other incoming messages, including evidence) and one vertex of each other parent's message,

$$m(u) = \sum_{\pi : \pi_i = u} w_{-i}(\pi)\, \langle \lambda, P(V \mid \pi) \rangle, \qquad w_{-i}(\pi) = \prod_{j \neq i} q_j(\pi_j).$$

Each coordinate $m(u)$ depends only on the rows with $\pi_i = u$, so as the rows range over their vertices, m ranges over the box $\prod_u [\underline{m}(u), \overline{m}(u)]$, with $\underline{m}(u) = \sum_{\pi : \pi_i = u} w_{-i}(\pi) \min_{P \in \mathrm{ext}K(V \mid \pi)} \langle \lambda, P \rangle$ and $\overline{m}(u)$ with $\max$. $M_{f_V \to U}$ is the hull of the normalized box corners over all corners (λ, q_{-i}). If every corner gives $m = 0$, the evidence below the edge is impossible for every choice, and the message is neutral.

Beliefs and atom bounds. The belief of V is the product of all its incoming credal messages, including evidence, and the bounds of an atom a of V are $\min_b \sum_{s \models a} b(s)$ and $\max_b \sum_{s \models a} b(s)$ over the belief's vertices b. The functional is linear, so its extrema over the hull are attained at vertices, and compound variables need no further projection.

Schedule. The factors are updated in topological order, then in reverse topological order, alternating, until a sweep changes no message (Hausdorff distance of the vertex sets below 10^{-12}). On a loopy graph the tolerance is τ and at most T sweeps are run.

G.3 Running Example

Consider the chain $A \to B \to C$ over binary atoms with $P(A) \in [0.4, 0.6]$, $P(B \mid A) \in [0.8, 0.9]$, $P(B \mid \neg A) \in [0.1, 0.2]$, $P(C \mid B) \in [0.5, 0.7]$ and $P(C \mid \neg B) \in [0.2, 0.3]$. Every row has two vertices, the endpoints of its interval, and all messages are intervals on $P(\cdot{=}1)$.

No evidence. The π-messages are $f_A \to A = [0.4, 0.6]$, $f_B \to B = [0.1 + 0.7 \cdot 0.4,\ 0.2 + 0.7 \cdot 0.6] = [0.38, 0.62]$ and $f_C \to C = [0.314, 0.548]$, the extremes of $P(C{=}1 \mid \neg B) + (P(C{=}1 \mid B) - P(C{=}1 \mid \neg B))\, P(B{=}1)$ over the endpoints. Each λ-message is the neutral $\{(\frac{1}{2}, \frac{1}{2})\}$, because every row sums to one, so the beliefs are the π-messages: the exact intervals.

Evidence $C{=}1$. The evidence factor sends $(0,1)$ into C, so $\lambda = (0,1)$ in f_C, and $m(b) = P(C{=}1 \mid b)$. The box is $[0.2, 0.3] \times [0.5, 0.7]$, and its normalized corners give $f_C \to B = [0.5/0.8,\ 0.7/0.9] = [0.625, 0.778]$. In f_B the vertex λ of this message gives $m(a) = \langle \lambda, P(B \mid a) \rangle$, and the corners give $f_B \to A = [0.575, 0.722]$. The beliefs are the products with the π-messages:

$$P(A{=}1 \mid C{=}1) \in [0.474, 0.796], \qquad P(B{=}1 \mid C{=}1) \in [0.505, 0.851].$$

The lower bound of A combines the lower endpoints, $\frac{0.4 \cdot 0.575}{0.4 \cdot 0.575 + 0.6 \cdot 0.425} = 0.474$, and the upper one the upper endpoints. Both agree with the enumeration of all $2 \cdot 4 \cdot 4$ vertex products. The previous downward-only scheme returned $P(A{=}1 \mid C{=}1) \in [0.4, 0.6]$.

Compound variables. In `examples/ivbp_compound_mid.lcn` the chain-graph component $\{B, C\}$ is a 4-state variable with parent U and children D, E. Its row given $U{=}1$ has 12 vertices, and the π-message into it has 22. With evidence $C{=}0$ the evidence factor of B-C selects the states $\{00, 10\}$, and the λ-message to U moves $P(U{=}1)$ from $[0.3, 0.6]$ to $[0.104, 1]$, matching the enumeration of all vertex products. The examples `examples/ivbp_compound_parents.lcn` (two compound parents of one child) and `alarm.lcn` (compound leaf C-D) are checked in the test suite in the same way.

G.4 Proofs of Theorems 5 and 6

The complexity bound of Theorem 5 follows from the updates of Section G.2. A π-message enumerates the corners of the parent messages and adds the rows one at a time. A λ-message enumerates the corners of the child and of the other parents, evaluates one inner product per row vertex, and normalizes at most $2^{|D_U|}$ box corners for each parent U. The number of sweeps is bounded in the proof of Theorem 6 (last paragraph). Alternating the direction of the sweeps lets one sweep finalize a whole chain in its direction; on all polytree benchmarks ($n \leq 100$, factor-graph diameter up to 36) and on a zigzag polytree of colliders we observed at most four sweeps, the last confirming the fixed point.

Lemma 9 (Projective images of hulls)**.** *Let $X_1, \ldots, X_r$ be finite sets of non-negative vectors and F a map, linear in each argument separately, with non-negative values and $\sum F(x_1, \ldots, x_r) > 0$ on $\mathrm{conv}X_1 \times \cdots \times \mathrm{conv}X_r$. With $g = \mathrm{nrm} \circ F$,*

$$\mathrm{conv}\, g(\mathrm{conv}X_1 \times \cdots \times \mathrm{conv}X_r) = \mathrm{conv}\, g(X_1 \times \cdots \times X_r).$$

Proof. For $r = 1$: if $x = \sum_k \lambda_k x_k$ with $x_k \in X_1$, then $g(x) = \sum_k \mu_k g(x_k)$ with $\mu_k = \lambda_k \sum F(x_k) / \sum_j \lambda_j \sum F(x_j) \geq 0$ summing to one, as in Lemma 2; so $g(\mathrm{conv}X_1) \subseteq \mathrm{conv}\, g(X_1)$, and the reverse inclusion is trivial. For $r > 1$, fix $x_2, \ldots, x_r$ in the hulls and apply the case $r = 1$ in the first argument; then fix $x_1 \in X_1$ and repeat in the second argument, and so on. Each step replaces one hull by its vertex set without changing the hull of the image, because the hull of a union of sets equals the hull of the union of their hulls. □

Proof of Theorem 6. Let the compiled credal network be a polytree, and assume $\underline{P}(\mathbf{y}) > 0$. Write θ for a choice of one distribution in every local credal set $K(V \mid \pi)$; since the credal network is separately specified, the rows of θ are chosen independently. For fixed θ let μ^θ be the precise sum-product messages equation 6, including the evidence factors.

Parameter blocks. Removing the edge between a factor f and a variable V splits the factor graph, a tree, in two. Let $\Theta_{f \to V}$ be the rows of the family factors on f's side, f included, and $\Theta_{V \to f}$ those on V's side. $\mu^\theta_{f \to V}$ depends only on $\Theta_{f \to V}$ and $\mu^\theta_{V \to f}$ only on $\Theta_{V \to f}$. At a factor f with neighbours $V, W_1, \ldots, W_k$, the blocks $\Theta_{W_j \to f}$ and the rows of f are pairwise disjoint, and their union is $\Theta_{f \to V}$. At a variable V with neighbouring factors $g_1, \ldots, g_l$ the blocks $\Theta_{g_j \to V}$ are pairwise disjoint. Hence, as θ ranges over all choices, the inputs of every update range independently over their own sets.

Scale. Every update equation 6 is linear in each incoming message, so rescaling an input rescales the output. The normalized output is therefore a function of the normalized inputs. Let $S_e = \{\mathrm{nrm}(\mu^\theta_e)\}$ be the set of normalized precise messages on the directed edge e, as θ ranges over all choices. Normalization is well defined: a message of zero mass on some choice gives the evidence probability zero for that choice combined with any choice on the other side of the edge, which $\underline{P}(\mathbf{y}) > 0$ excludes.

Algorithm 4 Coordinate Descent with Variable Elimination (CDVE)

Require: credal network $\langle G, \mathcal{K} \rangle$ (from the chain-graph factorization of $\mathcal{L}$); evidence $\mathbf{y}$; sweeps T; restarts R
Ensure: inner bounds $[\underline{p}_a, \overline{p}_a]$ on $P(a \mid \mathbf{y})$ for every non-evidence atom a

for each variable Z and each non-evidence atom a of Z **do**
  $\mathcal{R} \leftarrow$ ancestors of $\{Z\} \cup \{\text{evidence variables}\}$, themselves included
  for sense $\in \{\min, \max\}$ and start $j = 0, \ldots, R$ **do**
    $r \leftarrow$ centroids ($j = 0$) or random vertices ($j > 0$) of every row
    for sweep $t = 1, \ldots, T$ **do**
      for each $V \in \mathcal{R}$ with a row of more than one vertex **do**
        $T_V \leftarrow$ VE of all factors but f_V, with the evidence indicators, onto $\{V\} \cup \mathrm{pa}(V) \cup \{Z\}$
        $n_\pi \leftarrow \sum_{s \models a} T_V(\cdot, \pi, s), \quad d_\pi \leftarrow \sum_s T_V(\cdot, \pi, s)$
        $r_V \leftarrow$ Dinkelbach step on $\sum_\pi \langle n_\pi, r_\pi \rangle / \sum_\pi \langle d_\pi, r_\pi \rangle$ (sense)
      if no family changed **then break**
    $p_j \leftarrow P_r(a \mid \mathbf{y})$
  $\underline{p}_a \leftarrow \min_j p_j$ (sense min), $\quad \overline{p}_a \leftarrow \max_j p_j$ (sense max)

Induction. We show $\operatorname{conv} M_e = \operatorname{conv} S_e$ for every directed edge e, by induction on the depth of e (the length of the longest path on its side). Messages from evidence factors are points, and the neutral initial messages of leaf variables are their precise values. For an update with inputs $x_1, \ldots, x_r$ (incoming messages and, for a factor, its rows), by the independence of the blocks the set S_e is the image $g(S_1 \times \cdots \times S_r \times K_1 \times \cdots)$ of the product of the input sets and the factor's credal sets K, where $g = \mathrm{nrm} \circ F$ with F the update, linear in each argument. By the induction hypothesis and Lemma 9, $\operatorname{conv} S_e = \operatorname{conv} g(M_1 \times \cdots \times M_r \times \mathrm{ext}K_1 \times \cdots)$.

It remains to check that IVBP computes the vertices of this hull. For the product update this is the set of normalized vertex products. For the π-message, with the parent vertices fixed, it is the Minkowski sum of the weighted row-vertex sets, whose vertices are sums of row vertices. For the λ-message to U, with λ and the other parents' vertices fixed, the coordinate $m(u)$ is a sum of linear functions of the rows with $\pi_U = u$, and distinct u use distinct rows. So as the rows range over their credal sets, m ranges over the whole box $\prod_u [\underline{m}(u), \overline{m}(u)]$. The box is the hull of its corners, so by Lemma 9 (with F the identity on the box) the normalized image is the hull of the normalized corners. Under $\underline{P}(\mathbf{y}) > 0$ no corner has zero mass, since every corner is attained by a choice of rows.

Beliefs. On a tree, $b_V^\theta \propto \prod_g \mu_{g \to V}^\theta$ is $P_\theta(V \mid \mathbf{y})$, and the incoming messages of V depend on disjoint blocks. By the same argument, the hull of $\{P_\theta(V \mid \mathbf{y})\}_\theta$ is the hull of the computed belief vertices. The functional $\sum_{s \models a} b(s)$ is linear, so its extrema over the computed vertices are $\min_\theta P_\theta(a \mid \mathbf{y})$ and $\max_\theta P_\theta(a \mid \mathbf{y})$, the bounds over $\mathcal{P}_{\mathrm{SE}}$. Every member of $\mathcal{P}_{\mathrm{SE}}$ is such a θ, and $P(\mathbf{y}) > 0$ on all of $\mathcal{D}_{\mathrm{SE}}$, so by Lemma 2 these are the bounds over $\mathcal{D}_{\mathrm{SE}}$. By Proposition 6, which needs consistency, factorization and family realizability, $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$, so they are the bounds over $\mathcal{D}_{\mathrm{LCN}}$.

Fixed point. A message of depth t is computed from messages of depth less than t, and every sweep recomputes every message. So after t sweeps all messages of depth less than t equal their limits, and $D + 1$ sweeps suffice. The stopping test (no message changes in a sweep) is then met, and conversely a sweep without change on a tree certifies the fixed point, which by the induction is unique. □

Each hypothesis is used. On a loopy structure the inputs of an update share parameters and cannot be chosen independently, so the computed sets are neither inner nor outer in general. With multi-valued variables the vertex sets of the messages can grow, consistently with the NP-hardness of exact polytree inference for ternary variables (Mauá et al., 2014); for binary variables every message has at most two vertices and IVBP coincides with 2U. If $\underline{P}(\mathbf{y}) = 0$, IVBP drops the vectors of zero mass (regular conditioning); we do not claim exactness in that case.

H Detailed Description: Coordinate Descent with Variable Elimination

CDVE (Algorithm 4) is a coordinate-descent method over the local credal sets in the spirit of ApproxLP (Antonucci et al., 2015), which iteratively solves the linear program obtained by fixing all local credal sets but one.

Objective. For an atom a of Z (a state set $S_a \subseteq D_Z$ for a compound Z) and evidence $\mathbf{y}$ (an indicator over each observed variable's states, which for a compound variable selects the states that agree with its observed atoms),

$$P(a \mid \mathbf{y}) = \frac{\sum_{s \in S_a} P(Z{=}s, \mathbf{y})}{\sum_{s \in D_Z} P(Z{=}s, \mathbf{y})}.$$

Families outside the ancestral set $\mathcal{R}$ of $\{Z\} \cup \{\text{evidence variables}\}$ sum out to one, so they are dropped.

The family step. Every term of $P(Z, \mathbf{y})$ contains exactly one entry of V's table, so with the other families fixed

$$P(Z{=}s, \mathbf{y}) = \sum_{x,\pi} P(x \mid \pi)\, T_V(x, \pi, s),$$

with T_V the product of all other factors and of the evidence indicators, eliminated onto $\{V\} \cup \mathrm{pa}(V) \cup \{Z\}$ (if $Z = V$ or $Z \in \mathrm{pa}(V)$, that axis is shared). The objective is the ratio $N(r)/D(r)$ of $N = \sum_\pi \langle n_\pi, r_\pi \rangle$ and $D = \sum_\pi \langle d_\pi, r_\pi \rangle$, with $0 \le n_\pi \le d_\pi$, over the rows $r_\pi \in K(V \mid \pi)$. Since the network is separately specified, the rows range independently. Starting from the current ratio t_0, Dinkelbach's iteration sets every row to the vertex maximizing $\langle n_\pi - t\, d_\pi, v \rangle$ (minimizing, for $\min$) and $t \leftarrow N/D$, until $F(t) = \max_r (N - tD) \le 0$. This is the optimum of the local program, attained at a vertex of every row. A positive F implies $N > 0$ and hence $D > 0$, so the ratio stays defined. The move is accepted if it strictly improves t_0.

Starts. The first start puts every row at the centroid of its credal set. $P(\mathbf{y})$ is multilinear in the independently chosen rows, so its value at the centroids is the average of its values over all vertex products. The evidence therefore has positive probability at the start whenever some member of $\mathcal{P}_{\mathrm{SE}}$ gives it positive probability. At this symmetric point an uninformative child row (for instance $P(C \mid B)$ at $(\frac{1}{2}, \frac{1}{2})$) can hide every move of its parent's family, so a family whose rows are not yet vertices moves to a best vertex choice even when the ratio stays equal. Each row does so at most once, so this cannot cycle. The other R starts choose one vertex per row uniformly at random, with a fixed seed. Each start ends at a member of $\mathcal{P}_{\mathrm{SE}}$, so the best value over the starts remains an inner bound. Coordinate descent can stop at a local optimum: reaching the optimum may require changing two families at once.

Output. $[\underline{p}_a, \overline{p}_a]$ for every non-evidence atom; for a compound variable, the per-state range of the posteriors $P_r(Z \mid \mathbf{y})$ at the optima found.

H.1 Detailed Example

Consider the chain $A \to B \to C$ of the IVBP example, with $P(A) \in [0.4, 0.6]$, $P(B \mid A) \in [0.8, 0.9]$, $P(B \mid \neg A) \in [0.1, 0.2]$, $P(C \mid B) \in [0.5, 0.7]$, $P(C \mid \neg B) \in [0.2, 0.3]$ and evidence $C{=}1$. All three families are ancestors of $\{A, C\}$. We trace the start at the centroids, $P(A{=}1) = 0.5$, $P(B{=}1 \mid A) = 0.85$, $P(B{=}1 \mid \neg A) = 0.15$, $P(C{=}1 \mid B) = 0.6$, $P(C{=}1 \mid \neg B) = 0.25$.

Sense $\max$**, sweep 1.**

- *Family A.* $T_A(a) = P(C{=}1 \mid a)$ at the current rows, so $n = (0,\, 0.5475)$ and $d = (0.3025,\, 0.5475)$ over $A = (0, 1)$. Of the two vertices, $P(A) = (0.4, 0.6)$ gives the larger ratio, $0.6 \cdot 0.5475/(0.4 \cdot 0.3025 + 0.6 \cdot 0.5475) = 0.7308$.
- *Family B.* The coefficients make the step pick the rows $P(B{=}1 \mid A) = 0.9$ and $P(B{=}1 \mid \neg A) = 0.1$, giving 0.7483.
- *Family C.* The step picks $P(C{=}1 \mid B) = 0.7$ and $P(C{=}1 \mid \neg B) = 0.2$, giving 0.7959.

Sweep 2. No family improves, so the run stops with $\overline{p}_A = 0.7959$. Sense $\min$ picks the other vertices and stops at $\underline{p}_A = 0.4742$.

Result. $P(A{=}1 \mid C{=}1) \in [0.4742, 0.7959]$, which here is the exact interval: enumerating all $2 \cdot 4 \cdot 4$ vertex products gives the same values. Without evidence the same run returns the exact $P(A{=}1) \in [0.4, 0.6]$ after one improving sweep and one confirming sweep.

H.2 Proof of Theorem 7

Each family step costs one variable elimination over the n factors, in time $O(n\,k^{w^*+1})$, and I Dinkelbach iterations, each evaluating $\langle n_\pi - t\,d_\pi, v\rangle$ for every vertex of every row, in time $O(E_V\,k)$. A sweep performs at most one step per family. There are at most T sweeps per start, $R{+}1$ starts per sense, two senses and A atoms. The space is that of one elimination. Dinkelbach's iteration terminates because t increases strictly and takes values among the finitely many ratios of vertex choices.

I Detailed Experimental Evaluation

This appendix reports every measurement behind Section 5: all four random families at both scales, and the complete per-instance results for the two realistic suites. Throughout, the reference is CredalJT and the budget is 3600 seconds and 30 GB per run.

I.1 A reference that is exact by construction

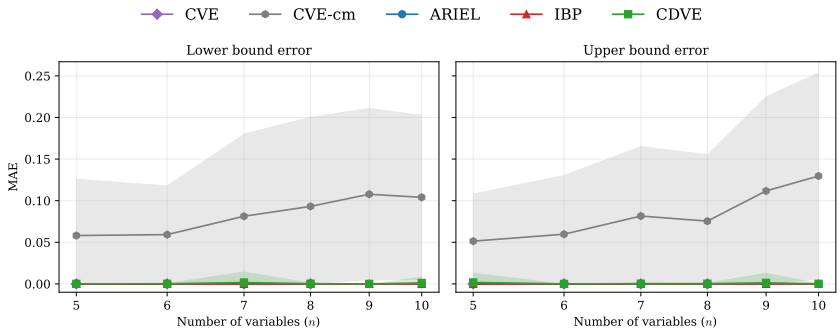
All approximate methods are scored against CJT, so the generator is required to produce instances on which CJT is *provably* exact. A candidate that passes the consistency check is kept only if (i) the junction tree that CJT builds for it satisfies H1–H4 of Theorem 8 — every sentence hosted, every LMC assertion discharged by separator-entailment, an exact reduction or the ordered rows, with nothing relaxed (Section B.2) — and its largest cluster has at most 12 atoms; and (ii) every endpoint of every marginal is certified by SCIP (proven optimal, or primal and dual bounds within 10^{-4}) within 60 s per solve. Condition (i) costs no solve and is decided in well under a second per instance; condition (ii) is the reference computation itself. Every saved instance records the outcome in a header comment. With the triage of Section B.2, condition (i) holds on all 556 random and realistic instances; a candidate is rejected only when its tree exceeds the width budget or an assertion would have to be relaxed — for example when an LMC assertion splits an undirected chain component, as in the `smokers` example.

I.2 Coverage and exclusions

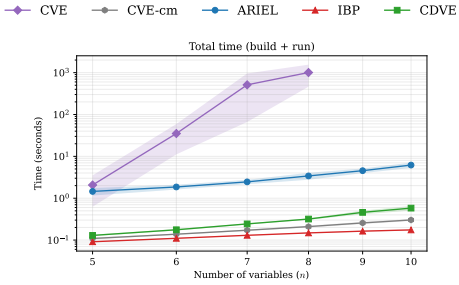
Not every algorithm completes on every instance, and reading the figures without this in mind would overstate coverage, since a curve simply stops where its algorithm ran out of budget. Over the 480 random instances, ARIEL and IVBP complete essentially all of them, and CredalJT's `ipopt` backend completes all of the instances it was run on. The exceptions are concentrated in the CVE family. Plain CVE, without clustering, is the least scalable: it solves $35/60$ of `tree_small`, $30/60$ of `polytree_small` and $24/60$ of `dag_small`, but *none* of `ktree_small` (21 timeouts and 39 errors) and none of `polytree_large`; on the other three large families it was not run at all. It is therefore absent from most plots, and where present it covers only the smallest sizes. Clustering restores coverage at small scale — CVE_{cm} and CVE_{cp} solve all 60 instances of every small family — but degrades on the large loopy ones, where CVE_{cm} reaches only $14/60$ on `ktree_large` and $30/60$ on `dag_large`. Exact inference over the 2^n joint is viable only at the very smallest sizes ($15/60$ on `tree_small`, $13/60$ on `polytree_small`, $2/10$ on the Bayesian-network instances, none of the Supreme Court ones), which is why it is not used as the reference. Three bookkeeping gaps are worth recording: the `ktree_large` CDVE log holds 58 records rather than 60, the Supreme Court exact-inference log holds 12 rather than 16, and the Bayesian-network CVE log holds 7 rather than 10; the corresponding instances are simply missing rather than failed, and are excluded from the aggregates.

I.3 RANDOM LCNS

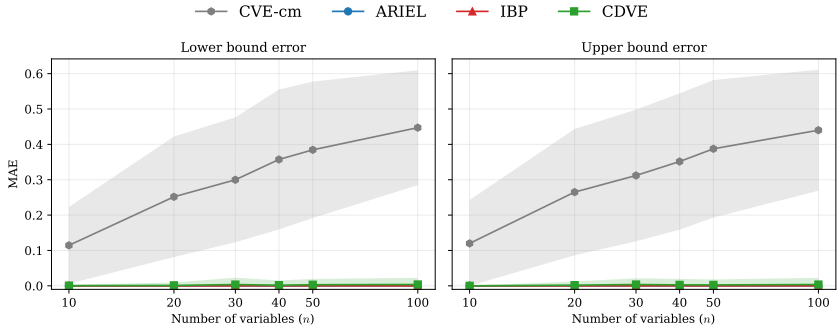
Figures 5–8 give the error and timing curves for all four families at both scales. The tables that follow them give the underlying per-size aggregates; for each family and scale the first table reports means and the second the corresponding standard deviations over the ten instances at each size.



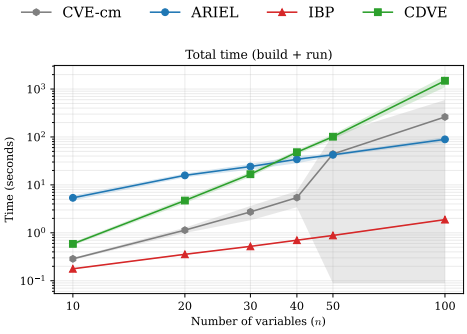
(a) small, error



(b) small, time

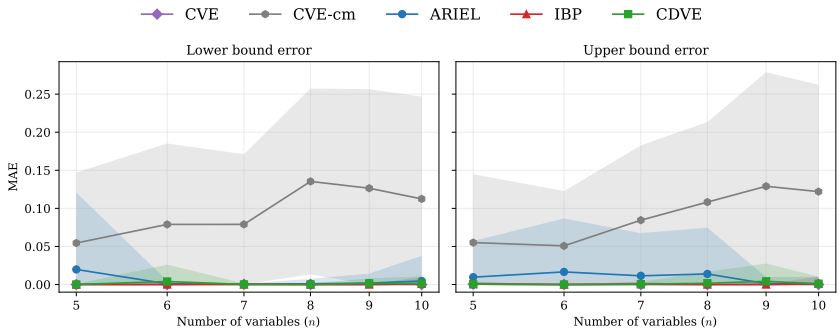


(c) large, error

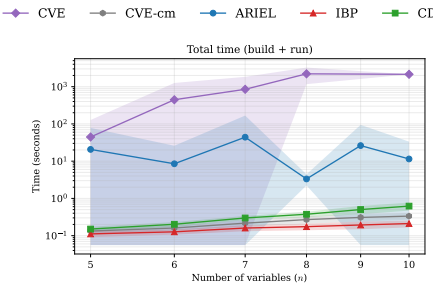


(d) large, time

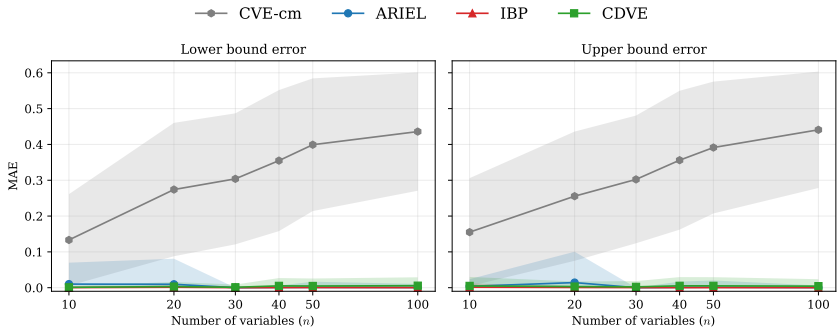
Figure 5: Random `tree` LCNs. Singly connected and fully realizable, so the credal-network relaxation is tight: ARIEL and IVBP are indistinguishable from the reference.



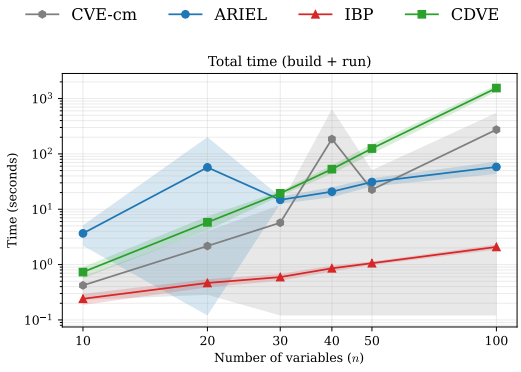
(a) small, error



(b) small, time

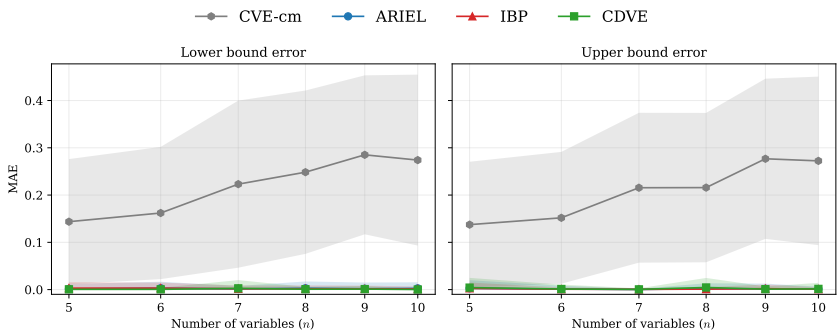


(c) large, error

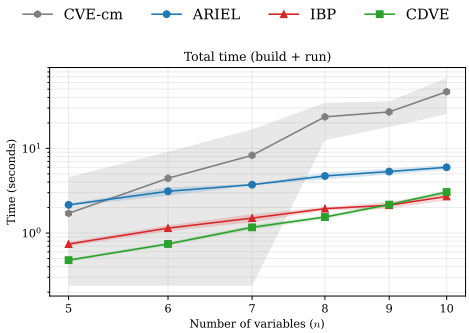


(d) large, time

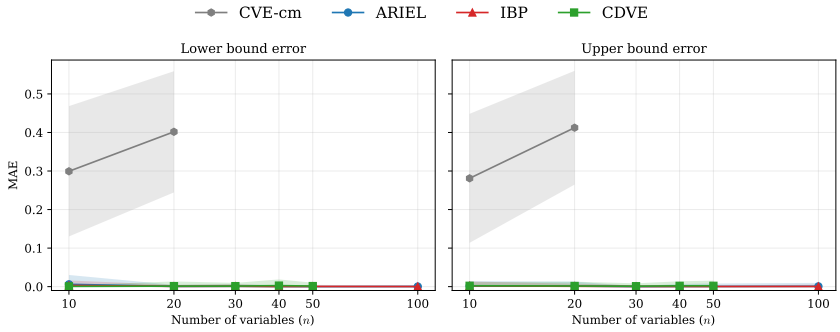
Figure 6: Random `polytree` LCNs. Also singly connected; the residual ARIEL error at the small scale comes from instances where the reference itself is the tighter engine.



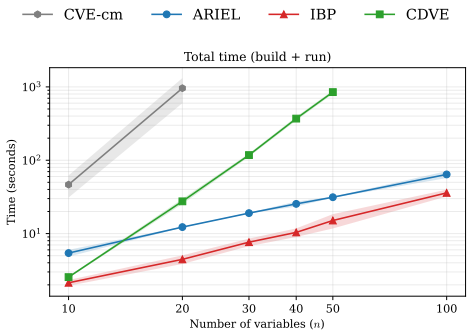
(a) small, error



(b) small, time

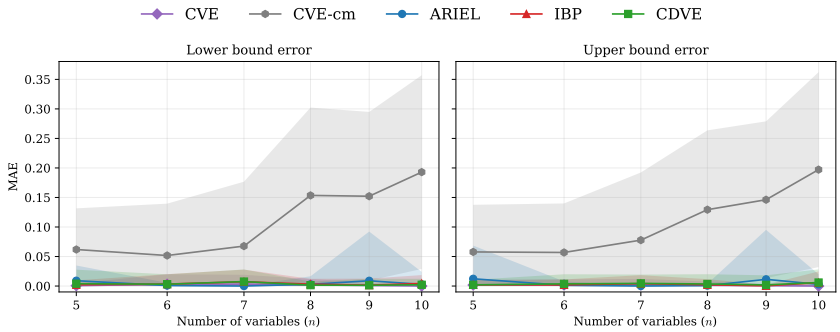


(c) large, error

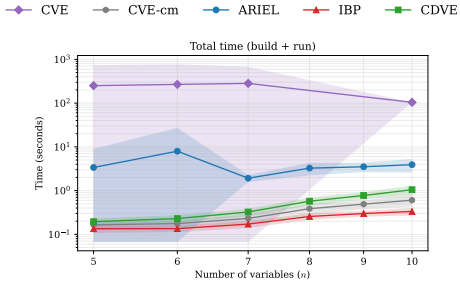


(d) large, time

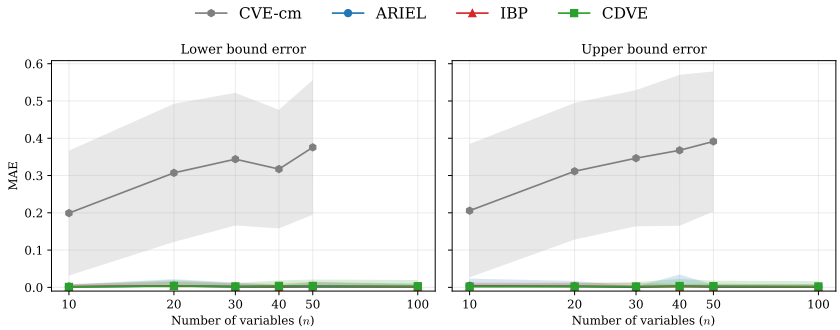
Figure 7: Random `ktree` LCNs (k=3). Loopy despite bounded treewidth, so all credal-network methods give outer bounds only; plain CVE solves none of these instances and CVE$_{cm}$ stops after n=20.



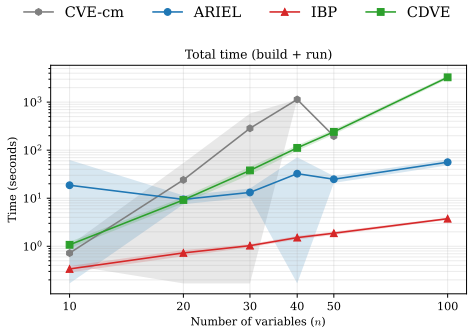
(a) small, error



(b) small, time



(c) large, error



(d) large, time

Figure 8: Random `dag` LCNs (moralized treewidth ≤ 3, at most two parents). The least structured family; CDVE's runtime grows fastest here, exceeding 600 seconds at n=100.

Table 5: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
tree_fr	5	CDVE	0.0001	0.0017	0.08	0.04	0.13	0.31	1.00
tree_fr	5	ARIEL	0.0000	0.0000	0.00	1.45	1.45	0.31	–
tree_fr	5	CVE	0.0000	0.0000	0.10	1.97	2.07	0.31	1.00
tree_fr	5	CVE_{cm}	0.0581	0.0514	0.09	0.02	0.11	0.31	1.00
tree_fr	5	IBP	0.0000	0.0000	0.09	0.00	0.09	0.31	1.00
tree_fr	6	CDVE	0.0001	0.0000	0.10	0.08	0.18	0.28	1.00
tree_fr	6	ARIEL	0.0000	0.0000	0.00	1.86	1.86	0.28	–
tree_fr	6	CVE	0.0000	0.0000	0.12	35.01	35.13	0.28	1.00
tree_fr	6	CVE_{cm}	0.0592	0.0597	0.10	0.04	0.14	0.28	1.00
tree_fr	6	IBP	0.0000	0.0000	0.11	0.00	0.11	0.28	1.00
tree_fr	7	CDVE	0.0016	0.0002	0.12	0.12	0.24	0.33	1.00
tree_fr	7	ARIEL	0.0000	0.0000	0.00	2.46	2.46	0.33	–
tree_fr	7	CVE	0.0000	0.0000	0.12	511.13	511.25	0.33	1.00
tree_fr	7	CVE_{cm}	0.0813	0.0815	0.12	0.05	0.17	0.33	1.00
tree_fr	7	IBP	0.0000	0.0000	0.13	0.00	0.13	0.33	1.00
tree_fr	8	CDVE	0.0002	0.0002	0.14	0.18	0.32	0.39	1.00
tree_fr	8	ARIEL	0.0000	0.0000	0.00	3.41	3.41	0.39	–
tree_fr	8	CVE	0.0000	0.0000	0.14	1006.64	1006.78	0.39	1.00
tree_fr	8	CVE_{cm}	0.0931	0.0754	0.14	0.07	0.21	0.39	1.00
tree_fr	8	IBP	0.0000	0.0000	0.14	0.01	0.15	0.39	1.00
tree_fr	9	CDVE	0.0000	0.0013	0.15	0.31	0.46	0.44	1.00
tree_fr	9	ARIEL	0.0000	0.0000	0.00	4.55	4.55	0.44	–
tree_fr	9	CVE_{cm}	0.1078	0.1117	0.16	0.10	0.26	0.44	1.00
tree_fr	9	IBP	0.0000	0.0000	0.16	0.01	0.16	0.44	1.00
tree_fr	10	CDVE	0.0011	0.0001	0.17	0.41	0.58	0.50	1.00
tree_fr	10	ARIEL	0.0000	0.0000	0.00	6.19	6.19	0.50	–
tree_fr	10	CVE_{cm}	0.1040	0.1297	0.18	0.12	0.30	0.50	1.00
tree_fr	10	IBP	0.0000	0.0000	0.17	0.01	0.17	0.50	1.00

Table 6: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
tree_fr	5	CDVE	0.0005	0.0119	0.08	0.04	0.13	0.31	1.00
tree_fr	5	ARIEL	0.0000	0.0000	0.00	1.45	1.45	0.31	–
tree_fr	5	CVE	0.0000	0.0000	0.10	1.97	2.07	0.31	1.00
tree_fr	5	CVE_{cm}	0.0683	0.0572	0.09	0.02	0.11	0.31	1.00
tree_fr	5	IBP	0.0000	0.0000	0.09	0.00	0.09	0.31	1.00
tree_fr	6	CDVE	0.0009	0.0000	0.10	0.08	0.18	0.28	1.00
tree_fr	6	ARIEL	0.0000	0.0000	0.00	1.86	1.86	0.28	–
tree_fr	6	CVE	0.0000	0.0000	0.12	35.01	35.13	0.28	1.00
tree_fr	6	CVE_{cm}	0.0591	0.0711	0.10	0.04	0.14	0.28	1.00
tree_fr	6	IBP	0.0000	0.0000	0.11	0.00	0.11	0.28	1.00
tree_fr	7	CDVE	0.0133	0.0014	0.12	0.12	0.24	0.33	1.00
tree_fr	7	ARIEL	0.0000	0.0000	0.00	2.46	2.46	0.33	–
tree_fr	7	CVE	0.0000	0.0000	0.12	511.13	511.25	0.33	1.00
tree_fr	7	CVE_{cm}	0.0993	0.0842	0.12	0.05	0.17	0.33	1.00
tree_fr	7	IBP	0.0000	0.0000	0.13	0.00	0.13	0.33	1.00
tree_fr	8	CDVE	0.0013	0.0014	0.14	0.18	0.32	0.39	1.00
tree_fr	8	ARIEL	0.0000	0.0000	0.00	3.41	3.41	0.39	–
tree_fr	8	CVE	0.0000	0.0000	0.14	1006.64	1006.78	0.39	1.00
tree_fr	8	CVE_{cm}	0.1078	0.0804	0.14	0.07	0.21	0.39	1.00
tree_fr	8	IBP	0.0000	0.0000	0.14	0.01	0.15	0.39	1.00
tree_fr	9	CDVE	0.0000	0.0120	0.15	0.31	0.46	0.44	1.00
tree_fr	9	ARIEL	0.0000	0.0000	0.00	4.55	4.55	0.44	–
tree_fr	9	CVE_{cm}	0.1035	0.1138	0.16	0.10	0.26	0.44	1.00
tree_fr	9	IBP	0.0000	0.0000	0.16	0.01	0.16	0.44	1.00
tree_fr	10	CDVE	0.0076	0.0009	0.17	0.41	0.58	0.50	1.00
tree_fr	10	ARIEL	0.0000	0.0000	0.00	6.19	6.19	0.50	–
tree_fr	10	CVE_{cm}	0.0992	0.1246	0.18	0.12	0.30	0.50	1.00
tree_fr	10	IBP	0.0000	0.0000	0.17	0.01	0.17	0.50	1.00

Table 7: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
tree_fr	10	CDVE	0.0005	0.0000	0.18	0.41	0.59	0.64	1.00
tree_fr	10	ARIEL	0.0000	0.0000	0.00	5.37	5.37	0.64	–
tree_fr	10	CVE_{cm}	0.1145	0.1199	0.18	0.11	0.29	0.64	1.00
tree_fr	10	IBP	0.0000	0.0000	0.17	0.01	0.18	0.64	1.00
tree_fr	20	CDVE	0.0011	0.0019	0.35	4.36	4.71	1.43	1.00
tree_fr	20	ARIEL	0.0000	0.0000	0.00	15.80	15.80	1.43	–
tree_fr	20	CVE_{cm}	0.2517	0.2650	0.37	0.77	1.14	1.43	1.00
tree_fr	20	IBP	0.0000	0.0000	0.34	0.02	0.36	1.43	1.00
tree_fr	30	CDVE	0.0036	0.0037	0.50	16.18	16.68	2.58	1.00
tree_fr	30	ARIEL	0.0000	0.0000	0.00	24.13	24.13	2.58	–
tree_fr	30	CVE_{cm}	0.2998	0.3120	0.55	2.18	2.73	2.58	1.00
tree_fr	30	IBP	0.0000	0.0000	0.49	0.03	0.52	2.58	1.00
tree_fr	40	CDVE	0.0017	0.0026	0.66	47.39	48.04	3.90	1.00
tree_fr	40	ARIEL	0.0000	0.0000	0.00	34.21	34.21	3.90	–
tree_fr	40	CVE_{cm}	0.3573	0.3515	0.70	4.72	5.43	3.90	1.00
tree_fr	40	IBP	0.0000	0.0000	0.66	0.04	0.70	3.90	1.00
tree_fr	50	CDVE	0.0031	0.0026	0.83	100.39	101.22	5.95	1.00
tree_fr	50	ARIEL	0.0000	0.0000	0.00	42.41	42.41	5.95	–
tree_fr	50	CVE_{cm}	0.3845	0.3875	0.96	43.09	44.05	5.95	1.00
tree_fr	50	IBP	0.0000	0.0000	0.83	0.05	0.88	5.95	1.00
tree_fr	100	CDVE	0.0037	0.0036	1.67	1487.61	1489.29	16.24	1.00
tree_fr	100	ARIEL	0.0000	0.0000	0.00	89.09	89.09	16.24	–
tree_fr	100	CVE_{cm}	0.4473	0.4401	1.94	259.37	261.31	16.24	1.00
tree_fr	100	IBP	0.0000	0.0000	1.76	0.12	1.88	16.24	1.00

Table 8: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
tree_fr	10	CDVE	0.0026	0.0003	0.18	0.41	0.59	0.64	1.00
tree_fr	10	ARIEL	0.0000	0.0000	0.00	5.37	5.37	0.64	–
tree_fr	10	CVE_{cm}	0.1085	0.1227	0.18	0.11	0.29	0.64	1.00
tree_fr	10	IBP	0.0000	0.0000	0.17	0.01	0.18	0.64	1.00
tree_fr	20	CDVE	0.0083	0.0104	0.35	4.36	4.71	1.43	1.00
tree_fr	20	ARIEL	0.0000	0.0000	0.00	15.80	15.80	1.43	–
tree_fr	20	CVE_{cm}	0.1704	0.1786	0.37	0.77	1.14	1.43	1.00
tree_fr	20	IBP	0.0000	0.0000	0.34	0.02	0.36	1.43	1.00
tree_fr	30	CDVE	0.0195	0.0177	0.50	16.18	16.68	2.58	1.00
tree_fr	30	ARIEL	0.0000	0.0000	0.00	24.13	24.13	2.58	–
tree_fr	30	CVE_{cm}	0.1766	0.1860	0.55	2.18	2.73	2.58	1.00
tree_fr	30	IBP	0.0000	0.0000	0.49	0.03	0.52	2.58	1.00
tree_fr	40	CDVE	0.0134	0.0169	0.66	47.39	48.04	3.90	1.00
tree_fr	40	ARIEL	0.0000	0.0000	0.00	34.21	34.21	3.90	–
tree_fr	40	CVE_{cm}	0.1977	0.1929	0.70	4.72	5.43	3.90	1.00
tree_fr	40	IBP	0.0000	0.0000	0.66	0.04	0.70	3.90	1.00
tree_fr	50	CDVE	0.0164	0.0154	0.83	100.39	101.22	5.95	1.00
tree_fr	50	ARIEL	0.0000	0.0000	0.00	42.41	42.41	5.95	–
tree_fr	50	CVE_{cm}	0.1929	0.1944	0.96	43.09	44.05	5.95	1.00
tree_fr	50	IBP	0.0000	0.0000	0.83	0.05	0.88	5.95	1.00
tree_fr	100	CDVE	0.0189	0.0190	1.67	1487.61	1489.29	16.24	1.00
tree_fr	100	ARIEL	0.0000	0.0000	0.00	89.09	89.09	16.24	–
tree_fr	100	CVE_{cm}	0.1625	0.1711	1.94	259.37	261.31	16.24	1.00
tree_fr	100	IBP	0.0000	0.0000	1.76	0.12	1.88	16.24	1.00

Table 9: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
polytree_fr	5	CDVE	0.0001	0.0005	0.10	0.05	0.15	0.39	1.50
polytree_fr	5	ARIEL	0.0198	0.0096	0.00	20.68	20.68	0.39	–
polytree_fr	5	CVE	0.0000	0.0005	0.11	44.41	44.52	0.39	1.50
polytree_fr	5	CVE_{cm}	0.0545	0.0550	0.11	0.03	0.13	0.39	1.50
polytree_fr	5	IBP	0.0000	0.0005	0.11	0.01	0.11	0.39	1.50
polytree_fr	6	CDVE	0.0039	0.0000	0.12	0.08	0.20	0.44	1.40
polytree_fr	6	ARIEL	0.0010	0.0166	0.00	8.51	8.51	0.44	–
polytree_fr	6	CVE	0.0000	0.0000	0.14	445.58	445.71	0.44	1.40
polytree_fr	6	CVE_{cm}	0.0789	0.0508	0.12	0.04	0.16	0.44	1.40
polytree_fr	6	IBP	0.0000	0.0000	0.12	0.01	0.13	0.44	1.40
polytree_fr	7	CDVE	0.0002	0.0004	0.15	0.15	0.30	0.73	1.70
polytree_fr	7	ARIEL	0.0002	0.0114	0.00	44.04	44.04	0.73	–
polytree_fr	7	CVE	0.0003	0.0000	0.13	849.79	849.92	0.73	1.40
polytree_fr	7	CVE_{cm}	0.0790	0.0845	0.15	0.06	0.22	0.73	1.70
polytree_fr	7	IBP	0.0002	0.0004	0.15	0.01	0.16	0.73	1.70
polytree_fr	8	CDVE	0.0001	0.0017	0.16	0.22	0.38	0.91	1.40
polytree_fr	8	ARIEL	0.0010	0.0139	0.00	3.33	3.33	0.91	–
polytree_fr	8	CVE	0.0000	0.0000	0.15	2213.21	2213.37	0.91	1.00
polytree_fr	8	CVE_{cm}	0.1354	0.1082	0.18	0.09	0.27	0.91	1.40
polytree_fr	8	IBP	0.0000	0.0000	0.17	0.01	0.17	0.91	1.40
polytree_fr	9	CDVE	0.0015	0.0042	0.18	0.32	0.50	0.99	1.30
polytree_fr	9	ARIEL	0.0019	0.0012	0.00	26.20	26.20	0.99	–
polytree_fr	9	CVE_{cm}	0.1264	0.1290	0.19	0.12	0.31	0.99	1.30
polytree_fr	9	IBP	0.0000	0.0000	0.18	0.01	0.19	0.99	1.30
polytree_fr	10	CDVE	0.0014	0.0011	0.19	0.43	0.62	1.11	1.30
polytree_fr	10	ARIEL	0.0046	0.0011	0.00	11.47	11.47	1.11	–
polytree_fr	10	CVE	0.0000	0.0000	0.16	2149.12	2149.28	1.11	1.00
polytree_fr	10	CVE_{cm}	0.1124	0.1220	0.20	0.14	0.33	1.11	1.30
polytree_fr	10	IBP	0.0010	0.0010	0.20	0.01	0.21	1.11	1.30

Table 10: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
polytree_fr	5	CDVE	0.0004	0.0029	0.10	0.05	0.15	0.39	1.50
polytree_fr	5	ARIEL	0.1010	0.0475	0.00	20.68	20.68	0.39	–
polytree_fr	5	CVE	0.0001	0.0029	0.11	44.41	44.52	0.39	1.50
polytree_fr	5	CVE_{cm}	0.0925	0.0898	0.11	0.03	0.13	0.39	1.50
polytree_fr	5	IBP	0.0001	0.0029	0.11	0.01	0.11	0.39	1.50
polytree_fr	6	CDVE	0.0222	0.0000	0.12	0.08	0.20	0.44	1.40
polytree_fr	6	ARIEL	0.0042	0.0702	0.00	8.51	8.51	0.44	–
polytree_fr	6	CVE	0.0000	0.0000	0.14	445.58	445.71	0.44	1.40
polytree_fr	6	CVE_{cm}	0.1063	0.0718	0.12	0.04	0.16	0.44	1.40
polytree_fr	6	IBP	0.0000	0.0000	0.12	0.01	0.13	0.44	1.40
polytree_fr	7	CDVE	0.0012	0.0032	0.15	0.15	0.30	0.73	1.70
polytree_fr	7	ARIEL	0.0009	0.0560	0.00	44.04	44.04	0.73	–
polytree_fr	7	CVE	0.0017	0.0001	0.13	849.79	849.92	0.73	1.40
polytree_fr	7	CVE_{cm}	0.0922	0.0982	0.15	0.06	0.22	0.73	1.70
polytree_fr	7	IBP	0.0012	0.0032	0.15	0.01	0.16	0.73	1.70
polytree_fr	8	CDVE	0.0011	0.0150	0.16	0.22	0.38	0.91	1.40
polytree_fr	8	ARIEL	0.0064	0.0608	0.00	3.33	3.33	0.91	–
polytree_fr	8	CVE	0.0000	0.0000	0.15	2213.21	2213.37	0.91	1.00
polytree_fr	8	CVE_{cm}	0.1220	0.1051	0.18	0.09	0.27	0.91	1.40
polytree_fr	8	IBP	0.0000	0.0000	0.17	0.01	0.17	0.91	1.40
polytree_fr	9	CDVE	0.0067	0.0234	0.18	0.32	0.50	0.99	1.30
polytree_fr	9	ARIEL	0.0126	0.0088	0.00	26.20	26.20	0.99	–
polytree_fr	9	CVE_{cm}	0.1301	0.1497	0.19	0.12	0.31	0.99	1.30
polytree_fr	9	IBP	0.0000	0.0001	0.18	0.01	0.19	0.99	1.30
polytree_fr	10	CDVE	0.0093	0.0094	0.19	0.43	0.62	1.11	1.30
polytree_fr	10	ARIEL	0.0331	0.0088	0.00	11.47	11.47	1.11	–
polytree_fr	10	CVE	0.0000	0.0000	0.16	2149.12	2149.28	1.11	1.00
polytree_fr	10	CVE_{cm}	0.1343	0.1404	0.20	0.14	0.33	1.11	1.30
polytree_fr	10	IBP	0.0092	0.0094	0.20	0.01	0.21	1.11	1.30

Table 11: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
polytree_fr	10	CDVE	0.0010	0.0050	0.23	0.51	0.73	13.77	1.50
polytree_fr	10	ARIEL	0.0100	0.0038	0.00	3.66	3.66	13.77	–
polytree_fr	10	CVE_{cm}	0.1333	0.1551	0.22	0.20	0.42	13.77	1.50
polytree_fr	10	IBP	0.0003	0.0013	0.23	0.01	0.24	13.77	1.50
polytree_fr	20	CDVE	0.0031	0.0031	0.45	5.35	5.80	3.21	1.60
polytree_fr	20	ARIEL	0.0095	0.0143	0.00	57.07	57.07	3.21	–
polytree_fr	20	CVE_{cm}	0.2740	0.2555	0.44	1.73	2.17	3.21	1.60
polytree_fr	20	IBP	0.0012	0.0006	0.43	0.03	0.47	3.21	1.60
polytree_fr	30	CDVE	0.0012	0.0023	0.54	18.84	19.38	5.22	1.10
polytree_fr	30	ARIEL	0.0000	0.0000	0.00	14.78	14.78	5.22	–
polytree_fr	30	CVE_{cm}	0.3039	0.3023	0.58	5.12	5.71	5.22	1.10
polytree_fr	30	IBP	0.0001	0.0003	0.56	0.04	0.59	5.22	1.10
polytree_fr	40	CDVE	0.0051	0.0052	0.76	52.22	52.98	12.10	1.40
polytree_fr	40	ARIEL	0.0007	0.0009	0.00	20.76	20.76	12.10	–
polytree_fr	40	CVE_{cm}	0.3548	0.3560	0.78	184.60	185.37	12.10	1.40
polytree_fr	40	IBP	0.0003	0.0002	0.80	0.06	0.86	12.10	1.40
polytree_fr	50	CDVE	0.0047	0.0051	1.10	124.13	125.23	38.34	1.50
polytree_fr	50	ARIEL	0.0008	0.0012	0.00	31.07	31.07	38.34	–
polytree_fr	50	CVE_{cm}	0.3992	0.3913	0.93	21.67	22.60	38.34	1.50
polytree_fr	50	IBP	0.0005	0.0003	0.99	0.07	1.06	38.34	1.50
polytree_fr	100	CDVE	0.0056	0.0043	1.95	1545.82	1547.77	68.64	1.40
polytree_fr	100	ARIEL	0.0005	0.0003	0.00	58.08	58.08	68.64	–
polytree_fr	100	CVE_{cm}	0.4359	0.4408	1.86	272.65	274.51	68.64	1.33
polytree_fr	100	IBP	0.0002	0.0002	1.93	0.15	2.08	68.64	1.40

Table 12: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
polytree_fr	10	CDVE	0.0041	0.0251	0.23	0.51	0.73	13.77	1.50
polytree_fr	10	ARIEL	0.0598	0.0200	0.00	3.66	3.66	13.77	–
polytree_fr	10	CVE_{cm}	0.1278	0.1507	0.22	0.20	0.42	13.77	1.50
polytree_fr	10	IBP	0.0013	0.0105	0.23	0.01	0.24	13.77	1.50
polytree_fr	20	CDVE	0.0127	0.0143	0.45	5.35	5.80	3.21	1.60
polytree_fr	20	ARIEL	0.0714	0.0862	0.00	57.07	57.07	3.21	–
polytree_fr	20	CVE_{cm}	0.1862	0.1804	0.44	1.73	2.17	3.21	1.60
polytree_fr	20	IBP	0.0060	0.0043	0.43	0.03	0.47	3.21	1.60
polytree_fr	30	CDVE	0.0076	0.0161	0.54	18.84	19.38	5.22	1.10
polytree_fr	30	ARIEL	0.0000	0.0000	0.00	14.78	14.78	5.22	–
polytree_fr	30	CVE_{cm}	0.1829	0.1783	0.58	5.12	5.71	5.22	1.10
polytree_fr	30	IBP	0.0014	0.0027	0.56	0.04	0.59	5.22	1.10
polytree_fr	40	CDVE	0.0218	0.0244	0.76	52.22	52.98	12.10	1.40
polytree_fr	40	ARIEL	0.0052	0.0162	0.00	20.76	20.76	12.10	–
polytree_fr	40	CVE_{cm}	0.1969	0.1938	0.78	184.60	185.37	12.10	1.40
polytree_fr	40	IBP	0.0019	0.0019	0.80	0.06	0.86	12.10	1.40
polytree_fr	50	CDVE	0.0210	0.0242	1.10	124.13	125.23	38.34	1.50
polytree_fr	50	ARIEL	0.0160	0.0190	0.00	31.07	31.07	38.34	–
polytree_fr	50	CVE_{cm}	0.1852	0.1840	0.93	21.67	22.60	38.34	1.50
polytree_fr	50	IBP	0.0049	0.0022	0.99	0.07	1.06	38.34	1.50
polytree_fr	100	CDVE	0.0235	0.0195	1.95	1545.82	1547.77	68.64	1.40
polytree_fr	100	ARIEL	0.0100	0.0060	0.00	58.08	58.08	68.64	–
polytree_fr	100	CVE_{cm}	0.1651	0.1624	1.86	272.65	274.51	68.64	1.33
polytree_fr	100	IBP	0.0025	0.0025	1.93	0.15	2.08	68.64	1.40

Table 13: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
ktree_fr_k3	5	CDVE	0.0006	0.0042	0.33	0.14	0.48	0.29	3.00
ktree_fr_k3	5	ARIEL	0.0015	0.0028	0.00	2.15	2.15	0.29	–
ktree_fr_k3	5	CVE_{cm}	0.1437	0.1374	0.33	1.38	1.71	0.29	3.00
ktree_fr_k3	5	IBP	0.0030	0.0027	0.33	0.41	0.74	0.29	3.00
ktree_fr_k3	6	CDVE	0.0006	0.0012	0.45	0.29	0.74	0.36	3.00
ktree_fr_k3	6	ARIEL	0.0029	0.0011	0.00	3.11	3.11	0.36	–
ktree_fr_k3	6	CVE_{cm}	0.1620	0.1519	0.45	3.99	4.43	0.36	3.00
ktree_fr_k3	6	IBP	0.0030	0.0008	0.45	0.69	1.14	0.36	3.00
ktree_fr_k3	7	CDVE	0.0027	0.0005	0.57	0.59	1.17	0.42	3.00
ktree_fr_k3	7	ARIEL	0.0015	0.0002	0.00	3.72	3.72	0.42	–
ktree_fr_k3	7	CVE_{cm}	0.2231	0.2155	0.60	7.68	8.27	0.42	3.00
ktree_fr_k3	7	IBP	0.0018	0.0005	0.58	0.92	1.50	0.42	3.00
ktree_fr_k3	8	CDVE	0.0008	0.0044	0.64	0.90	1.54	0.51	3.00
ktree_fr_k3	8	ARIEL	0.0028	0.0020	0.00	4.72	4.72	0.51	–
ktree_fr_k3	8	CVE_{cm}	0.2484	0.2158	0.69	22.88	23.57	0.51	3.00
ktree_fr_k3	8	IBP	0.0018	0.0011	0.71	1.23	1.94	0.51	3.00
ktree_fr_k3	9	CDVE	0.0011	0.0009	0.78	1.39	2.17	0.60	3.00
ktree_fr_k3	9	ARIEL	0.0019	0.0017	0.00	5.32	5.32	0.60	–
ktree_fr_k3	9	CVE_{cm}	0.2851	0.2768	0.82	26.13	26.95	0.60	3.00
ktree_fr_k3	9	IBP	0.0015	0.0020	0.76	1.38	2.13	0.60	3.00
ktree_fr_k3	10	CDVE	0.0002	0.0015	0.95	2.10	3.05	0.68	3.00
ktree_fr_k3	10	ARIEL	0.0027	0.0011	0.00	5.98	5.98	0.68	–
ktree_fr_k3	10	CVE_{cm}	0.2740	0.2723	0.96	45.69	46.65	0.68	3.00
ktree_fr_k3	10	IBP	0.0013	0.0009	0.97	1.74	2.70	0.68	3.00

Table 14: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
ktree_fr_k3	5	CDVE	0.0021	0.0206	0.33	0.14	0.48	0.29	3.00
ktree_fr_k3	5	ARIEL	0.0105	0.0171	0.00	2.15	2.15	0.29	–
ktree_fr_k3	5	CVE_{cm}	0.1324	0.1330	0.33	1.38	1.71	0.29	3.00
ktree_fr_k3	5	IBP	0.0125	0.0122	0.33	0.41	0.74	0.29	3.00
ktree_fr_k3	6	CDVE	0.0034	0.0094	0.45	0.29	0.74	0.36	3.00
ktree_fr_k3	6	ARIEL	0.0137	0.0071	0.00	3.11	3.11	0.36	–
ktree_fr_k3	6	CVE_{cm}	0.1399	0.1394	0.45	3.99	4.43	0.36	3.00
ktree_fr_k3	6	IBP	0.0119	0.0037	0.45	0.69	1.14	0.36	3.00
ktree_fr_k3	7	CDVE	0.0174	0.0022	0.57	0.59	1.17	0.42	3.00
ktree_fr_k3	7	ARIEL	0.0063	0.0013	0.00	3.72	3.72	0.42	–
ktree_fr_k3	7	CVE_{cm}	0.1767	0.1585	0.60	7.68	8.27	0.42	3.00
ktree_fr_k3	7	IBP	0.0069	0.0027	0.58	0.92	1.50	0.42	3.00
ktree_fr_k3	8	CDVE	0.0050	0.0202	0.64	0.90	1.54	0.51	3.00
ktree_fr_k3	8	ARIEL	0.0144	0.0114	0.00	4.72	4.72	0.51	–
ktree_fr_k3	8	CVE_{cm}	0.1728	0.1580	0.69	22.88	23.57	0.51	3.00
ktree_fr_k3	8	IBP	0.0071	0.0041	0.71	1.23	1.94	0.51	3.00
ktree_fr_k3	9	CDVE	0.0067	0.0057	0.78	1.39	2.17	0.60	3.00
ktree_fr_k3	9	ARIEL	0.0131	0.0110	0.00	5.32	5.32	0.60	–
ktree_fr_k3	9	CVE_{cm}	0.1681	0.1694	0.82	26.13	26.95	0.60	3.00
ktree_fr_k3	9	IBP	0.0068	0.0089	0.76	1.38	2.13	0.60	3.00
ktree_fr_k3	10	CDVE	0.0012	0.0127	0.95	2.10	3.05	0.68	3.00
ktree_fr_k3	10	ARIEL	0.0129	0.0052	0.00	5.98	5.98	0.68	–
ktree_fr_k3	10	CVE_{cm}	0.1808	0.1783	0.96	45.69	46.65	0.68	3.00
ktree_fr_k3	10	IBP	0.0054	0.0039	0.97	1.74	2.70	0.68	3.00

Table 15: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
ktree_fr_k3	10	CDVE	0.0007	0.0023	0.40	2.14	2.55	0.65	3.00
ktree_fr_k3	10	ARIEL	0.0060	0.0017	0.00	5.42	5.42	0.65	–
ktree_fr_k3	10	CVE_{cm}	0.2993	0.2810	0.40	46.11	46.51	0.65	3.00
ktree_fr_k3	10	IBP	0.0032	0.0021	0.40	1.74	2.13	0.65	3.00
ktree_fr_k3	20	CDVE	0.0013	0.0017	0.85	26.67	27.52	1.57	3.00
ktree_fr_k3	20	ARIEL	0.0006	0.0014	0.00	12.26	12.26	1.57	–
ktree_fr_k3	20	CVE_{cm}	0.4017	0.4124	0.81	959.08	959.88	1.57	3.00
ktree_fr_k3	20	IBP	0.0007	0.0009	0.83	3.62	4.45	1.57	3.00
ktree_fr_k3	30	CDVE	0.0013	0.0011	1.28	116.25	117.53	2.91	3.00
ktree_fr_k3	30	ARIEL	0.0009	0.0001	0.00	19.01	19.01	2.91	–
ktree_fr_k3	30	IBP	0.0009	0.0004	1.26	6.38	7.64	2.91	3.00
ktree_fr_k3	40	CDVE	0.0027	0.0020	1.71	367.85	369.55	4.80	3.00
ktree_fr_k3	40	ARIEL	0.0003	0.0004	0.00	25.41	25.41	4.80	–
ktree_fr_k3	40	IBP	0.0003	0.0003	1.68	8.70	10.38	4.80	3.00
ktree_fr_k3	50	CDVE	0.0015	0.0022	3.68	845.10	848.78	6.89	3.00
ktree_fr_k3	50	ARIEL	0.0001	0.0004	0.00	31.32	31.32	6.89	–
ktree_fr_k3	50	IBP	0.0004	0.0003	3.64	11.44	15.08	6.89	3.00
ktree_fr_k3	100	ARIEL	0.0003	0.0007	0.00	64.03	64.03	22.71	–
ktree_fr_k3	100	IBP	0.0002	0.0005	10.55	25.34	35.89	22.71	3.00

Table 16: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
ktree_fr_k3	10	CDVE	0.0031	0.0095	0.40	2.14	2.55	0.65	3.00
ktree_fr_k3	10	ARIEL	0.0242	0.0128	0.00	5.42	5.42	0.65	–
ktree_fr_k3	10	CVE_{cm}	0.1690	0.1673	0.40	46.11	46.51	0.65	3.00
ktree_fr_k3	10	IBP	0.0128	0.0113	0.40	1.74	2.13	0.65	3.00
ktree_fr_k3	20	CDVE	0.0122	0.0111	0.85	26.67	27.52	1.57	3.00
ktree_fr_k3	20	ARIEL	0.0041	0.0116	0.00	12.26	12.26	1.57	–
ktree_fr_k3	20	CVE_{cm}	0.1572	0.1476	0.81	959.08	959.88	1.57	3.00
ktree_fr_k3	20	IBP	0.0038	0.0068	0.83	3.62	4.45	1.57	3.00
ktree_fr_k3	30	CDVE	0.0085	0.0075	1.28	116.25	117.53	2.91	3.00
ktree_fr_k3	30	ARIEL	0.0055	0.0011	0.00	19.01	19.01	2.91	–
ktree_fr_k3	30	IBP	0.0055	0.0032	1.26	6.38	7.64	2.91	3.00
ktree_fr_k3	40	CDVE	0.0161	0.0119	1.71	367.85	369.55	4.80	3.00
ktree_fr_k3	40	ARIEL	0.0034	0.0065	0.00	25.41	25.41	4.80	–
ktree_fr_k3	40	IBP	0.0029	0.0036	1.68	8.70	10.38	4.80	3.00
ktree_fr_k3	50	CDVE	0.0087	0.0133	3.68	845.10	848.78	6.89	3.00
ktree_fr_k3	50	ARIEL	0.0010	0.0077	0.00	31.32	31.32	6.89	–
ktree_fr_k3	50	IBP	0.0042	0.0030	3.64	11.44	15.08	6.89	3.00
ktree_fr_k3	100	ARIEL	0.0037	0.0090	0.00	64.03	64.03	22.71	–
ktree_fr_k3	100	IBP	0.0024	0.0048	10.55	25.34	35.89	22.71	3.00

Table 17: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
dag_fr_tw3	5	CDVE	0.0039	0.0020	0.13	0.06	0.20	0.42	2.00
dag_fr_tw3	5	ARIEL	0.0090	0.0123	0.00	3.38	3.38	0.42	–
dag_fr_tw3	5	CVE	0.0006	0.0017	0.13	250.50	250.64	0.42	2.00
dag_fr_tw3	5	CVE_{cm}	0.0619	0.0578	0.13	0.03	0.16	0.42	2.00
dag_fr_tw3	5	IBP	0.0021	0.0020	0.13	0.01	0.13	0.42	2.00
dag_fr_tw3	6	CDVE	0.0031	0.0037	0.14	0.09	0.23	0.49	1.90
dag_fr_tw3	6	ARIEL	0.0008	0.0012	0.00	7.96	7.96	0.49	–
dag_fr_tw3	6	CVE	0.0031	0.0019	0.15	267.68	267.84	0.49	1.90
dag_fr_tw3	6	CVE_{cm}	0.0517	0.0569	0.14	0.04	0.18	0.49	1.90
dag_fr_tw3	6	IBP	0.0033	0.0020	0.13	0.01	0.14	0.49	1.90
dag_fr_tw3	7	CDVE	0.0076	0.0045	0.17	0.15	0.32	0.79	1.80
dag_fr_tw3	7	ARIEL	0.0000	0.0000	0.00	1.92	1.92	0.79	–
dag_fr_tw3	7	CVE	0.0031	0.0020	0.13	282.36	282.49	0.79	1.50
dag_fr_tw3	7	CVE_{cm}	0.0677	0.0778	0.17	0.07	0.23	0.79	1.80
dag_fr_tw3	7	IBP	0.0073	0.0041	0.16	0.01	0.17	0.79	1.80
dag_fr_tw3	8	CDVE	0.0015	0.0035	0.24	0.33	0.57	1.10	2.20
dag_fr_tw3	8	ARIEL	0.0029	0.0007	0.00	3.26	3.26	1.10	–
dag_fr_tw3	8	CVE_{cm}	0.1535	0.1293	0.24	0.14	0.39	1.10	2.20
dag_fr_tw3	8	IBP	0.0032	0.0021	0.24	0.02	0.26	1.10	2.20
dag_fr_tw3	9	CDVE	0.0013	0.0019	0.28	0.49	0.77	1.23	2.20
dag_fr_tw3	9	ARIEL	0.0089	0.0114	0.00	3.50	3.50	1.23	–
dag_fr_tw3	9	CVE_{cm}	0.1521	0.1462	0.27	0.22	0.49	1.23	2.20
dag_fr_tw3	9	IBP	0.0016	0.0004	0.28	0.02	0.30	1.23	2.20
dag_fr_tw3	10	CDVE	0.0021	0.0060	0.31	0.73	1.05	1.62	2.20
dag_fr_tw3	10	ARIEL	0.0029	0.0029	0.00	3.92	3.92	1.62	–
dag_fr_tw3	10	CVE	0.0000	0.0000	0.20	104.02	104.21	1.62	2.00
dag_fr_tw3	10	CVE_{cm}	0.1930	0.1972	0.30	0.30	0.60	1.62	2.20
dag_fr_tw3	10	IBP	0.0044	0.0058	0.31	0.03	0.33	1.62	2.20

Table 18: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
dag_fr_tw3	5	CDVE	0.0238	0.0085	0.13	0.06	0.20	0.42	2.00
dag_fr_tw3	5	ARIEL	0.0258	0.0563	0.00	3.38	3.38	0.42	–
dag_fr_tw3	5	CVE	0.0041	0.0085	0.13	250.50	250.64	0.42	2.00
dag_fr_tw3	5	CVE_{cm}	0.0697	0.0798	0.13	0.03	0.16	0.42	2.00
dag_fr_tw3	5	IBP	0.0074	0.0083	0.13	0.01	0.13	0.42	2.00
dag_fr_tw3	6	CDVE	0.0168	0.0163	0.14	0.09	0.23	0.49	1.90
dag_fr_tw3	6	ARIEL	0.0047	0.0067	0.00	7.96	7.96	0.49	–
dag_fr_tw3	6	CVE	0.0168	0.0094	0.15	267.68	267.84	0.49	1.90
dag_fr_tw3	6	CVE_{cm}	0.0878	0.0830	0.14	0.04	0.18	0.49	1.90
dag_fr_tw3	6	IBP	0.0168	0.0092	0.13	0.01	0.14	0.49	1.90
dag_fr_tw3	7	CDVE	0.0205	0.0150	0.17	0.15	0.32	0.79	1.80
dag_fr_tw3	7	ARIEL	0.0000	0.0001	0.00	1.92	1.92	0.79	–
dag_fr_tw3	7	CVE	0.0160	0.0102	0.13	282.36	282.49	0.79	1.50
dag_fr_tw3	7	CVE_{cm}	0.1090	0.1145	0.17	0.07	0.23	0.79	1.80
dag_fr_tw3	7	IBP	0.0206	0.0143	0.16	0.01	0.17	0.79	1.80
dag_fr_tw3	8	CDVE	0.0061	0.0165	0.24	0.33	0.57	1.10	2.20
dag_fr_tw3	8	ARIEL	0.0136	0.0031	0.00	3.26	3.26	1.10	–
dag_fr_tw3	8	CVE_{cm}	0.1487	0.1343	0.24	0.14	0.39	1.10	2.20
dag_fr_tw3	8	IBP	0.0091	0.0096	0.24	0.02	0.26	1.10	2.20
dag_fr_tw3	9	CDVE	0.0109	0.0163	0.28	0.49	0.77	1.23	2.20
dag_fr_tw3	9	ARIEL	0.0834	0.0838	0.00	3.50	3.50	1.23	–
dag_fr_tw3	9	CVE_{cm}	0.1428	0.1327	0.27	0.22	0.49	1.23	2.20
dag_fr_tw3	9	IBP	0.0110	0.0018	0.28	0.02	0.30	1.23	2.20
dag_fr_tw3	10	CDVE	0.0101	0.0223	0.31	0.73	1.05	1.62	2.20
dag_fr_tw3	10	ARIEL	0.0211	0.0170	0.00	3.92	3.92	1.62	–
dag_fr_tw3	10	CVE	0.0000	0.0000	0.20	104.02	104.21	1.62	2.00
dag_fr_tw3	10	CVE_{cm}	0.1640	0.1650	0.30	0.30	0.60	1.62	2.20
dag_fr_tw3	10	IBP	0.0142	0.0190	0.31	0.03	0.33	1.62	2.20

Table 19: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
dag_fr_tw3	10	CDVE	0.0012	0.0020	0.32	0.76	1.07	1.53	2.40
dag_fr_tw3	10	ARIEL	0.0013	0.0038	0.00	18.64	18.64	1.53	–
dag_fr_tw3	10	CVE_{cm}	0.1992	0.2057	0.32	0.41	0.72	1.53	2.40
dag_fr_tw3	10	IBP	0.0019	0.0030	0.31	0.03	0.34	1.53	2.40
dag_fr_tw3	20	CDVE	0.0039	0.0023	0.67	8.53	9.20	4.17	2.60
dag_fr_tw3	20	ARIEL	0.0040	0.0026	0.00	9.46	9.46	4.17	–
dag_fr_tw3	20	CVE_{cm}	0.3072	0.3114	0.65	23.43	24.08	4.17	2.60
dag_fr_tw3	20	IBP	0.0037	0.0026	0.66	0.07	0.73	4.17	2.60
dag_fr_tw3	30	CDVE	0.0025	0.0023	1.03	36.88	37.90	10.25	2.80
dag_fr_tw3	30	ARIEL	0.0013	0.0001	0.00	13.20	13.20	10.25	–
dag_fr_tw3	30	CVE_{cm}	0.3439	0.3465	0.95	284.01	284.96	10.25	2.75
dag_fr_tw3	30	IBP	0.0021	0.0018	0.94	0.09	1.03	10.25	2.80
dag_fr_tw3	40	CDVE	0.0039	0.0036	1.43	110.19	111.62	14.22	3.00
dag_fr_tw3	40	ARIEL	0.0009	0.0034	0.00	32.43	32.43	14.22	–
dag_fr_tw3	40	CVE_{cm}	0.3173	0.3677	1.09	1135.19	1136.29	14.22	3.00
dag_fr_tw3	40	IBP	0.0019	0.0018	1.35	0.17	1.51	14.22	3.00
dag_fr_tw3	50	CDVE	0.0037	0.0032	1.65	238.85	240.50	23.21	3.00
dag_fr_tw3	50	ARIEL	0.0011	0.0008	0.00	24.86	24.86	23.21	–
dag_fr_tw3	50	CVE_{cm}	0.3756	0.3915	1.54	194.40	195.94	23.21	3.00
dag_fr_tw3	50	IBP	0.0027	0.0020	1.67	0.20	1.87	23.21	3.00
dag_fr_tw3	100	CDVE	0.0030	0.0024	3.00	3278.99	3281.98	80.21	3.00
dag_fr_tw3	100	ARIEL	0.0013	0.0010	0.00	56.21	56.21	80.21	–
dag_fr_tw3	100	IBP	0.0013	0.0012	3.34	0.40	3.75	80.21	3.00

Table 20: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
dag_fr_tw3	10	CDVE	0.0060	0.0079	0.32	0.76	1.07	1.53	2.40
dag_fr_tw3	10	ARIEL	0.0070	0.0196	0.00	18.64	18.64	1.53	–
dag_fr_tw3	10	CVE_{cm}	0.1675	0.1789	0.32	0.41	0.72	1.53	2.40
dag_fr_tw3	10	IBP	0.0066	0.0099	0.31	0.03	0.34	1.53	2.40
dag_fr_tw3	20	CDVE	0.0155	0.0093	0.67	8.53	9.20	4.17	2.60
dag_fr_tw3	20	ARIEL	0.0180	0.0147	0.00	9.46	9.46	4.17	–
dag_fr_tw3	20	CVE_{cm}	0.1853	0.1833	0.65	23.43	24.08	4.17	2.60
dag_fr_tw3	20	IBP	0.0115	0.0099	0.66	0.07	0.73	4.17	2.60
dag_fr_tw3	30	CDVE	0.0092	0.0122	1.03	36.88	37.90	10.25	2.80
dag_fr_tw3	30	ARIEL	0.0118	0.0022	0.00	13.20	13.20	10.25	–
dag_fr_tw3	30	CVE_{cm}	0.1779	0.1827	0.95	284.01	284.96	10.25	2.75
dag_fr_tw3	30	IBP	0.0085	0.0100	0.94	0.09	1.03	10.25	2.80
dag_fr_tw3	40	CDVE	0.0148	0.0189	1.43	110.19	111.62	14.22	3.00
dag_fr_tw3	40	ARIEL	0.0046	0.0312	0.00	32.43	32.43	14.22	–
dag_fr_tw3	40	CVE_{cm}	0.1591	0.2025	1.09	1135.19	1136.29	14.22	3.00
dag_fr_tw3	40	IBP	0.0081	0.0073	1.35	0.17	1.51	14.22	3.00
dag_fr_tw3	50	CDVE	0.0173	0.0148	1.65	238.85	240.50	23.21	3.00
dag_fr_tw3	50	ARIEL	0.0122	0.0063	0.00	24.86	24.86	23.21	–
dag_fr_tw3	50	CVE_{cm}	0.1807	0.1878	1.54	194.40	195.94	23.21	3.00
dag_fr_tw3	50	IBP	0.0124	0.0080	1.67	0.20	1.87	23.21	3.00
dag_fr_tw3	100	CDVE	0.0167	0.0142	3.00	3278.99	3281.98	80.21	3.00
dag_fr_tw3	100	ARIEL	0.0096	0.0083	0.00	56.21	56.21	80.21	–
dag_fr_tw3	100	IBP	0.0062	0.0064	3.34	0.40	3.75	80.21	3.00

I.4 REALISTIC LCNS

The two tables below give the complete per-instance results for the ten Bayesian-network-derived LCNs and the sixteen Supreme Court chain-graph LCNs, including the nine dense `supreme_court` instances omitted from Table 1. On those nine, only ARIEL and the CredalJT reference complete: with n=10 atoms but 345–1009 sentences, the size of the constraint system rather than the treewidth is the binding limit, and IVBP, CDVE and all CVE variants exhaust the budget. One instance in the suite, `benchmark_scaling_directions_n24_2`, is a repaired variant of the original benchmark: in the chain graph the four reviewers of a stage are mutually independent, so the probability of their conjunction is forced to the product of their marginals, which the original's `component` sentences contradicted; the three incompatible sentences were dropped and the repaired instance is consistent, verified by a junction-tree feasibility check.

Table 21: Mean error metrics vs. CJT reference

Instance	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
alarm_consistent.lcn	CDVE	0.1008	0.0146	0.10	0.59	0.69	0.72	2.00
alarm_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.92	3.92	0.72	–
alarm_consistent.lcn	CVE_{cm}	0.0866	0.0670	0.10	0.14	0.24	0.72	2.00
alarm_consistent.lcn	IBP	0.1008	0.0149	0.10	0.03	0.13	0.72	2.00
asia_consistent.lcn	CDVE	0.0033	0.0073	0.08	0.24	0.32	0.45	2.00
asia_consistent.lcn	ARIEL	0.0000	0.0000	0.00	2.86	2.86	0.45	–
asia_consistent.lcn	CVE_{cm}	0.0323	0.0291	0.08	0.08	0.16	0.45	2.00
asia_consistent.lcn	IBP	0.0035	0.0063	0.08	0.01	0.10	0.45	2.00
cancer_consistent.lcn	CDVE	0.0653	0.0272	0.06	0.06	0.12	0.37	2.00
cancer_consistent.lcn	ARIEL	0.0013	0.0139	0.00	2.10	2.10	0.37	–
cancer_consistent.lcn	CVE	0.0653	0.0272	0.06	42.46	42.52	0.37	2.00
cancer_consistent.lcn	CVE_{cm}	0.0491	0.0267	0.06	0.03	0.09	0.37	2.00
cancer_consistent.lcn	IBP	0.0639	0.0392	0.06	0.01	0.07	0.37	2.00
credit_consistent.lcn	CDVE	0.2570	0.0000	0.11	0.58	0.69	1.15	3.00
credit_consistent.lcn	ARIEL	0.0496	0.0000	0.00	3.18	3.18	1.15	–
credit_consistent.lcn	CVE_{cm}	0.1079	0.1650	0.11	1.62	1.73	1.15	3.00
credit_consistent.lcn	IBP	0.2570	0.0000	0.11	0.13	0.24	1.15	3.00
earthquake_consistent.lcn	CDVE	0.0041	0.0000	0.05	0.05	0.10	0.53	2.00
earthquake_consistent.lcn	ARIEL	0.0002	0.0000	0.00	1.88	1.88	0.53	–
earthquake_consistent.lcn	CVE	0.0041	0.0000	0.05	7.63	7.68	0.53	2.00
earthquake_consistent.lcn	CVE_{cm}	0.0019	0.0036	0.05	0.03	0.08	0.53	2.00
earthquake_consistent.lcn	IBP	0.0041	0.0000	0.05	0.01	0.06	0.53	2.00
engine_consistent.lcn	CDVE	0.0591	0.0104	0.10	0.43	0.53	0.53	2.00
engine_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.25	3.25	0.53	–
engine_consistent.lcn	CVE_{cm}	0.1234	0.0839	0.10	0.32	0.42	0.53	2.00
engine_consistent.lcn	IBP	0.0591	0.0104	0.10	0.01	0.12	0.53	2.00
hepatitis_consistent.lcn	CDVE	0.0703	0.0851	0.24	0.90	1.15	0.56	3.00
hepatitis_consistent.lcn	ARIEL	0.0084	0.0075	0.00	3.91	3.91	0.56	–
hepatitis_consistent.lcn	CVE_{cm}	0.1137	0.1838	0.24	16.29	16.53	0.56	3.00
hepatitis_consistent.lcn	IBP	0.0686	0.0797	0.24	0.50	0.75	0.56	3.00
suicide_consistent.lcn	CDVE	0.0849	0.0302	0.10	0.52	0.62	0.87	2.00
suicide_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.74	3.74	0.87	–
suicide_consistent.lcn	CVE_{cm}	0.0970	0.0513	0.10	0.13	0.23	0.87	2.00
suicide_consistent.lcn	IBP	0.0849	0.0302	0.10	0.02	0.12	0.87	2.00
tank_consistent.lcn	CDVE	0.1149	0.0751	0.30	0.87	1.18	0.77	3.00
tank_consistent.lcn	ARIEL	0.0048	0.0034	0.00	5.02	5.02	0.77	–
tank_consistent.lcn	CVE_{cm}	0.0653	0.0802	0.31	0.19	0.50	0.77	3.00
tank_consistent.lcn	IBP	0.1163	0.0739	0.30	0.02	0.32	0.77	3.00
toy_consistent.lcn	CDVE	0.0891	0.0745	0.09	0.03	0.12	0.16	2.00
toy_consistent.lcn	ARIEL	0.0000	0.0000	0.00	1.60	1.60	0.16	–
toy_consistent.lcn	CVE_{cm}	0.0742	0.0294	0.09	0.01	0.10	0.16	2.00
toy_consistent.lcn	IBP	0.0891	0.0745	0.09	0.01	0.09	0.16	2.00

Table 22: Standard deviation of error metrics vs. CJT reference

Instance	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
alarm_consistent.lcn	CDVE	0.1476	0.0203	0.10	0.59	0.69	0.72	2.00
alarm_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.92	3.92	0.72	–
alarm_consistent.lcn	CVE_{cm}	0.1279	0.0854	0.10	0.14	0.24	0.72	2.00
alarm_consistent.lcn	IBP	0.1476	0.0208	0.10	0.03	0.13	0.72	2.00
asia_consistent.lcn	CDVE	0.0043	0.0172	0.08	0.24	0.32	0.45	2.00
asia_consistent.lcn	ARIEL	0.0000	0.0000	0.00	2.86	2.86	0.45	–
asia_consistent.lcn	CVE_{cm}	0.0308	0.0359	0.08	0.08	0.16	0.45	2.00
asia_consistent.lcn	IBP	0.0047	0.0144	0.08	0.01	0.10	0.45	2.00
cancer_consistent.lcn	CDVE	0.0374	0.0320	0.06	0.06	0.12	0.37	2.00
cancer_consistent.lcn	ARIEL	0.0026	0.0278	0.00	2.10	2.10	0.37	–
cancer_consistent.lcn	CVE	0.0374	0.0320	0.06	42.46	42.52	0.37	2.00
cancer_consistent.lcn	CVE_{cm}	0.0397	0.0233	0.06	0.03	0.09	0.37	2.00
cancer_consistent.lcn	IBP	0.0369	0.0323	0.06	0.01	0.07	0.37	2.00
credit_consistent.lcn	CDVE	0.2149	0.0000	0.11	0.58	0.69	1.15	3.00
credit_consistent.lcn	ARIEL	0.0414	0.0000	0.00	3.18	3.18	1.15	–
credit_consistent.lcn	CVE_{cm}	0.1401	0.1170	0.11	1.62	1.73	1.15	3.00
credit_consistent.lcn	IBP	0.2149	0.0000	0.11	0.13	0.24	1.15	3.00
earthquake_consistent.lcn	CDVE	0.0035	0.0000	0.05	0.05	0.10	0.53	2.00
earthquake_consistent.lcn	ARIEL	0.0003	0.0000	0.00	1.88	1.88	0.53	–
earthquake_consistent.lcn	CVE	0.0035	0.0000	0.05	7.63	7.68	0.53	2.00
earthquake_consistent.lcn	CVE_{cm}	0.0029	0.0043	0.05	0.03	0.08	0.53	2.00
earthquake_consistent.lcn	IBP	0.0035	0.0000	0.05	0.01	0.06	0.53	2.00
engine_consistent.lcn	CDVE	0.1131	0.0131	0.10	0.43	0.53	0.53	2.00
engine_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.25	3.25	0.53	–
engine_consistent.lcn	CVE_{cm}	0.1318	0.0677	0.10	0.32	0.42	0.53	2.00
engine_consistent.lcn	IBP	0.1131	0.0131	0.10	0.01	0.12	0.53	2.00
hepatitis_consistent.lcn	CDVE	0.1232	0.1429	0.24	0.90	1.15	0.56	3.00
hepatitis_consistent.lcn	ARIEL	0.0252	0.0167	0.00	3.91	3.91	0.56	–
hepatitis_consistent.lcn	CVE_{cm}	0.1580	0.1630	0.24	16.29	16.53	0.56	3.00
hepatitis_consistent.lcn	IBP	0.1236	0.1445	0.24	0.50	0.75	0.56	3.00
suicide_consistent.lcn	CDVE	0.1160	0.0430	0.10	0.52	0.62	0.87	2.00
suicide_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.74	3.74	0.87	–
suicide_consistent.lcn	CVE_{cm}	0.1147	0.0637	0.10	0.13	0.23	0.87	2.00
suicide_consistent.lcn	IBP	0.1160	0.0430	0.10	0.02	0.12	0.87	2.00
tank_consistent.lcn	CDVE	0.1337	0.1096	0.30	0.87	1.18	0.77	3.00
tank_consistent.lcn	ARIEL	0.0144	0.0102	0.00	5.02	5.02	0.77	–
tank_consistent.lcn	CVE_{cm}	0.0659	0.0555	0.31	0.19	0.50	0.77	3.00
tank_consistent.lcn	IBP	0.1351	0.1089	0.30	0.02	0.32	0.77	3.00
toy_consistent.lcn	CDVE	0.1363	0.0751	0.09	0.03	0.12	0.16	2.00
toy_consistent.lcn	ARIEL	0.0000	0.0000	0.00	1.60	1.60	0.16	–
toy_consistent.lcn	CVE_{cm}	0.1190	0.0345	0.09	0.01	0.10	0.16	2.00
toy_consistent.lcn	IBP	0.1363	0.0751	0.09	0.01	0.09	0.16	2.00

Table 23: Mean error metrics vs. CJT reference

Instance	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
benchmark_scaling_directions_n24_2.lcn	CDVE	0.1215	0.4370	1.78	22.80	24.57	1258.24	4.00
benchmark_scaling_directions_n24_2.lcn	ARIEL	0.0000	0.0000	0.00	13.42	13.42	1258.24	–
benchmark_scaling_directions_n24_2.lcn	CVE_{cm}	0.1123	0.1749	1.86	2206.67	2208.54	1258.24	4.00
benchmark_scaling_directions_n24_2.lcn	IBP	0.1215	0.4370	1.91	685.00	686.91	1258.24	4.00
benchmark_scaling_directions_n32_1.lcn	CDVE	0.1148	0.2247	0.57	19.98	20.55	2.42	1.00
benchmark_scaling_directions_n32_1.lcn	ARIEL	0.0000	0.0000	0.00	19.80	19.80	2.42	–
benchmark_scaling_directions_n32_1.lcn	CVE_{cm}	0.0643	0.0716	0.59	8.59	9.18	2.42	1.00
benchmark_scaling_directions_n32_1.lcn	IBP	0.1148	0.2247	0.64	0.05	0.69	2.42	1.00
maintenance_scaling_n12_3.lcn	CDVE	0.1287	0.5013	0.29	0.94	1.23	0.66	2.00
maintenance_scaling_n12_3.lcn	ARIEL	0.0284	0.0000	0.00	8.13	8.13	0.66	–
maintenance_scaling_n12_3.lcn	CVE_{cm}	0.0793	0.2270	0.31	0.25	0.57	0.66	2.00
maintenance_scaling_n12_3.lcn	IBP	0.1286	0.4926	0.31	0.04	0.35	0.66	2.00
maintenance_scaling_n6_1.lcn	CDVE	0.1029	0.3972	0.17	0.10	0.27	0.28	2.00
maintenance_scaling_n6_1.lcn	ARIEL	0.0000	0.0000	0.00	4.16	4.16	0.28	–
maintenance_scaling_n6_1.lcn	CVE	0.1029	0.4017	0.14	789.77	789.91	0.28	2.00
maintenance_scaling_n6_1.lcn	CVE_{cm}	0.1021	0.2685	0.14	0.05	0.19	0.28	2.00
maintenance_scaling_n6_1.lcn	IBP	0.1029	0.4017	0.15	0.01	0.16	0.28	2.00
maintenance_scaling_n9_2.lcn	CDVE	0.1447	0.4748	0.21	0.48	0.69	0.44	2.00
maintenance_scaling_n9_2.lcn	ARIEL	0.0398	0.0000	0.00	6.09	6.09	0.44	–
maintenance_scaling_n9_2.lcn	CVE_{cm}	0.1095	0.2437	0.20	0.12	0.32	0.44	2.00
maintenance_scaling_n9_2.lcn	IBP	0.1447	0.4662	0.23	0.02	0.25	0.44	2.00
paper_scaling_directions_n35_2.lcn	CDVE	0.1506	0.4467	7.59	190.48	198.07	1342.14	5.00
paper_scaling_directions_n35_2.lcn	ARIEL	0.0064	0.0047	0.00	36.58	36.58	1342.14	–
paper_scaling_directions_n40_1.lcn	CDVE	0.1288	0.2631	0.70	50.55	51.25	3.27	1.00
paper_scaling_directions_n40_1.lcn	ARIEL	0.0000	0.0000	0.00	34.72	34.72	3.27	–
paper_scaling_directions_n40_1.lcn	CVE_{cm}	0.0696	0.0566	0.74	658.10	658.84	3.27	1.00
paper_scaling_directions_n40_1.lcn	IBP	0.1288	0.2631	0.82	0.09	0.91	3.27	1.00
supreme_court_judicial_power_credal_source_comparison_n10_1.lcn	ARIEL	0.0532	0.1234	0.00	115.72	115.72	348.07	–
supreme_court_judicial_power_credal_source_comparison_n10_2.lcn	ARIEL	0.0321	0.1107	0.00	76.72	76.72	337.23	–
supreme_court_judicial_power_credal_source_comparison_n10_3.lcn	ARIEL	0.0259	0.0972	0.00	527.92	527.92	426.93	–
supreme_court_judicial_power_dual_source_credal_n10_1.lcn	ARIEL	0.0259	0.0972	0.00	523.64	523.64	435.71	–
supreme_court_judicial_power_release_comparison_n10_1.lcn	ARIEL	0.0317	0.0860	0.00	68.03	68.03	346.18	–
supreme_court_judicial_power_release_comparison_n10_2.lcn	ARIEL	0.0532	0.1234	0.00	117.31	117.31	337.95	–
supreme_court_judicial_power_release_comparison_n10_3.lcn	ARIEL	0.0259	0.0972	0.00	562.73	562.73	426.77	–
supreme_court_judicial_power_source_variants_n10_1.lcn	ARIEL	0.0317	0.0860	0.00	71.64	71.64	338.65	–
supreme_court_judicial_power_source_variants_n10_2.lcn	ARIEL	0.0176	0.0913	0.00	70.58	70.58	357.82	–

Table 24: Standard deviation of error metrics vs. CJT reference

Instance	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
benchmark_scaling_directions_n24_2.lcn	CDVE	0.0648	0.1928	1.78	22.80	24.57	1258.24	4.00
benchmark_scaling_directions_n24_2.lcn	ARIEL	0.0000	0.0000	0.00	13.42	13.42	1258.24	–
benchmark_scaling_directions_n24_2.lcn	CVE_{cm}	0.0780	0.1686	1.86	2206.67	2208.54	1258.24	4.00
benchmark_scaling_directions_n24_2.lcn	IBP	0.0648	0.1928	1.91	685.00	686.91	1258.24	4.00
benchmark_scaling_directions_n32_1.lcn	CDVE	0.0657	0.1463	0.57	19.98	20.55	2.42	1.00
benchmark_scaling_directions_n32_1.lcn	ARIEL	0.0000	0.0000	0.00	19.80	19.80	2.42	–
benchmark_scaling_directions_n32_1.lcn	CVE_{cm}	0.0430	0.0738	0.59	8.59	9.18	2.42	1.00
benchmark_scaling_directions_n32_1.lcn	IBP	0.0657	0.1463	0.64	0.05	0.69	2.42	1.00
maintenance_scaling_n12_3.lcn	CDVE	0.0795	0.2363	0.29	0.94	1.23	0.66	2.00
maintenance_scaling_n12_3.lcn	ARIEL	0.0514	0.0000	0.00	8.13	8.13	0.66	–
maintenance_scaling_n12_3.lcn	CVE_{cm}	0.0491	0.1572	0.31	0.25	0.57	0.66	2.00
maintenance_scaling_n12_3.lcn	IBP	0.0794	0.2313	0.31	0.04	0.35	0.66	2.00
maintenance_scaling_n6_1.lcn	CDVE	0.0729	0.2197	0.17	0.10	0.27	0.28	2.00
maintenance_scaling_n6_1.lcn	ARIEL	0.0000	0.0000	0.00	4.16	4.16	0.28	–
maintenance_scaling_n6_1.lcn	CVE	0.0729	0.2232	0.14	789.77	789.91	0.28	2.00
maintenance_scaling_n6_1.lcn	CVE_{cm}	0.0710	0.1599	0.14	0.05	0.19	0.28	2.00
maintenance_scaling_n6_1.lcn	IBP	0.0729	0.2232	0.15	0.01	0.16	0.28	2.00
maintenance_scaling_n9_2.lcn	CDVE	0.0953	0.2482	0.21	0.48	0.69	0.44	2.00
maintenance_scaling_n9_2.lcn	ARIEL	0.0496	0.0000	0.00	6.09	6.09	0.44	–
maintenance_scaling_n9_2.lcn	CVE_{cm}	0.0699	0.1858	0.20	0.12	0.32	0.44	2.00
maintenance_scaling_n9_2.lcn	IBP	0.0952	0.2428	0.23	0.02	0.25	0.44	2.00
paper_scaling_directions_n35_2.lcn	CDVE	0.0682	0.1408	7.59	190.48	198.07	1342.14	5.00
paper_scaling_directions_n35_2.lcn	ARIEL	0.0250	0.0276	0.00	36.58	36.58	1342.14	–
paper_scaling_directions_n40_1.lcn	CDVE	0.0656	0.1334	0.70	50.55	51.25	3.27	1.00
paper_scaling_directions_n40_1.lcn	ARIEL	0.0000	0.0000	0.00	34.72	34.72	3.27	–
paper_scaling_directions_n40_1.lcn	CVE_{cm}	0.0672	0.0365	0.74	658.10	658.84	3.27	1.00
paper_scaling_directions_n40_1.lcn	IBP	0.0656	0.1334	0.82	0.09	0.91	3.27	1.00
supreme_court_judicial_power_credal_source_comparison_n10_1.lcn	ARIEL	0.0620	0.2451	0.00	115.72	115.72	348.07	–
supreme_court_judicial_power_credal_source_comparison_n10_2.lcn	ARIEL	0.0407	0.2477	0.00	76.72	76.72	337.23	–
supreme_court_judicial_power_credal_source_comparison_n10_3.lcn	ARIEL	0.0414	0.2514	0.00	527.92	527.92	426.93	–
supreme_court_judicial_power_dual_source_credal_n10_1.lcn	ARIEL	0.0414	0.2514	0.00	523.64	523.64	435.71	–
supreme_court_judicial_power_release_comparison_n10_1.lcn	ARIEL	0.0624	0.2550	0.00	68.03	68.03	346.18	–
supreme_court_judicial_power_release_comparison_n10_2.lcn	ARIEL	0.0620	0.2451	0.00	117.31	117.31	337.95	–
supreme_court_judicial_power_release_comparison_n10_3.lcn	ARIEL	0.0414	0.2514	0.00	562.73	562.73	426.77	–
supreme_court_judicial_power_source_variants_n10_1.lcn	ARIEL	0.0624	0.2550	0.00	71.64	71.64	338.65	–
supreme_court_judicial_power_source_variants_n10_2.lcn	ARIEL	0.0442	0.2536	0.00	70.58	70.58	357.82	–